modenanalyse_2fe2s

Supplementary Technical Documentation

for reviewers, methodology auditors, and developers

Lukas Knauer

AG Schünemann, RPTU Kaiserslautern-Landau

lknauer@rptu.de

Software version 1.2.3 | Document compiled September 2, 2026

Scope of this document.

This supplement is the technical companion to the user manual (`Manual.pdf`). The manual is the recommended entry point and covers installation, configuration, workflow, output reference, and a physically motivated theory overview with literature citations. This supplement is *independent of the manual* – a reviewer can read it standalone – and goes *deeper*: every formula in the code is derived from first principles, every algorithm is given in pseudocode, every data structure is enumerated, every configuration field is described with its effect on the code path, every Excel column is specified with its source function and uncertainty-propagation rule, and the audit trail of v1.0.4 is presented in full. Where the manual summarises, the supplement spells out.

Code citations. Every formula in this document carries ⇝ `module.function` markers pointing to the implementing function in the source tree. These are verified by the test suite (`tests/test_manual_code_refs.py`) and cannot silently drift out of sync with the code.

Repository. https://github.com/lknauer/modenanalyse_2fe2s

Contents

Part I

Architecture and Conventions

1 System overview

The package `modenanalyse_2fe2s` processes the output of a quantum-chemical normal-mode calculation on a [2Fe-2S] cluster embedded in a protein (or, for benchmark systems, in a small molecular ligand environment), and produces a suite of mode-by-mode geometric, energetic, and spectroscopic descriptors. The primary input is a Gaussian frequency log (`.log`) with `Freq=hpmodes` or, with slightly reduced precision, a regular frequency log; an ORCA `.hess` file is also accepted via an independent parser. The optional second input is a Protein Data Bank file (`.pdb`) of the underlying protein, used to map Gaussian atom indices to residue labels and to detect secondary-structure elements when residue-level analysis is requested.

The principal outputs are a hierarchy of Excel workbooks plus a matching `run-log.txt` that records every decision the pipeline took. The Excel workbooks group the per-mode descriptors by chemical and physical theme:

- *Cluster-core* descriptors: OOP/INP fractions, cluster centre-of-mass displacement, cluster expansion, cluster rotation, D_{2h}-symmetry-coordinate amplitudes (SCSD), kernel-classification scores.
- *Fe-ligand bond* descriptors: stretching, bending in-plane, bending out-of-plane, all in Å, with uncertainty.
- *His-NH* descriptors: N-H stretching contribution per protonated histidine.
- *Group* descriptors: pre-defined or auto-generated atom groups (residues, cofactor fragments) with OOP/INP/angle/ torsion contributions.
- *Marcus-Hush reorganization*: per-mode $\lambda_X^{\mathrm{mode}}$ and per-channel $\Lambda_X^{\mathrm{total}}$ along the $X \in \{\mathrm{FeFe}, \mathrm{FeN}, \mathrm{FeS}, \mathrm{NH}, \mathrm{HA}\}$ bond channels, plus frequency-resolved modulation spectra $M_X(\omega)$, co-modulation spectra $C_{\mathrm{PCET}}(\omega)$, $C_{\mathrm{PT\text{-}FeN}}(\omega)$, $C_{\mathrm{ET\text{-}FeS}}(\omega)$, and cumulative $\Lambda_X(\omega)$.
- *Secondary-structure* descriptors: per-SSE-element amplitudes, centre-of-mass amplitudes, axial amplitudes, tilting angles.
- *Embeddings*: three parallel UMAP-HDBSCAN clusterings on complementary feature spaces (Marcus-Hush reorganization features, SSE-element fingerprint, C_α-amplitude fingerprint).
- *Diagnostic* sheets: the `Coordination` sheet documents the Gaussian-to-PDB atom mapping used by all downstream calculations.

What the tool is *not* designed to compute (and what is left to specialised partner tools) includes: the NRVS spectrum itself (the tool computes Fe-projected amplitudes and modulations, not the NRVS cross-section), the Lamb-Mößbauer factor, recoilless fractions, Fe-projected thermodynamic functions (ΔU, ΔS, ΔF), spin-orbit-coupled g-tensors, Mößbauer parameters, and absolute reorganization energies in the original Marcus-classical sense (the tool computes spectroscopic *bond* reorganization energies, which are related but distinct – the distinction is spelled out in section 11).

1.1 Versions and supported file formats

Property	Value	Notes
Package version	1.2.3	SemVer; the audit trail is part VIII.
Python	$\geq$ 3.11	Type-hint syntax used.
Primary input	Gaussian `.log` `Freq=hpmodes`	5-decimal eigenvector precision. Standard 3-decimal eigenvectors are accepted as fallback.
Alternative input	ORCA `.hess`	Independent parser in `orca_io.py`.
Secondary input	PDB	Optional. Enables residue labels, SSE analysis, embedding-feature C_α amplitudes.
Output format	`.xlsx` (openpyxl)	One main workbook plus optional interp / SSE / embedding workbooks.
Test count	147 unit + 4 E2E	part IX
Manual length	104 pages	`Manual.pdf`
Anleitung length	107 pages	`Anleitung.pdf` (German translation)
This supplement	this document	target $\sim$ 180 pages

2 Data flow architecture

The pipeline is a sequence of well-defined phases that pass data between each other by means of explicit Python data structures (no hidden global state, except for the per-cluster deduplication sets `_WARNED_EMPTY_GROUP`, `_WARNED_EMPTY_LIG`, `_WARNED_HN_SKIP`, which are reset at the start of every cluster run by `core.reset_warning_state`). This section walks through the seven phases. The entry point is `runner.run_analysis(cfg)` which dispatches to `runner._run_analysis_single(cfg)` once per cluster.

2.1 Phase 1 – byte-offset scan of the log file

↝ `logio.scan_log`

`logio.scan_log` performs a streaming, byte-level scan of the Gaussian `.log` file in fixed-size chunks (default 4 MiB). Three byte-pattern markers are matched against each chunk:

- `Standard orientation:` – the heading of every Cartesian-coordinate dump. The very first such dump (after SCF convergence and before any geometry optimisation step) is the equilibrium geometry used by all downstream geometric analyses.
- `and normal coordinates:` – the heading of every section that lists displacement eigenvectors. Gaussian writes these in groups of three or five modes per row, depending on the `hpmodes` option, and writes one such section per frequency analysis. For an unscaled job, there is exactly one section; for a job with thermochemistry at multiple temperatures, there can be multiple sections, but they all contain the same eigenvectors.
- `Frequencies --` – the headers that list the per-mode frequency, reduced mass, force constant, and IR intensity. One header per 3-mode (standard) or 5-mode (`hpmodes`) block.

For each match, the byte offset at the *start* of the matched pattern is recorded. This is a $\mathtt{O}(N_{\text{bytes}})$ scan with constant memory; for a 4-GB Gaussian log it completes in a few seconds on a modern SSD. A 200-byte rolling tail buffer ensures that pattern matches spanning chunk boundaries are not missed.

The scan results are returned as three sorted lists of integers: `so_off`, `nc_off`, `fr_off`. The cache layer (section 2.8) stores these lists in a pickle keyed by the log file's full path, size, mtime, and Python major version; on cache hit, the scan is skipped entirely, which makes subsequent runs (e.g. for different frequency windows) order-of-magnitude faster than the first run.

Clarification of a common misconception. `scan_log` *reads the entire log file from beginning to end* to find these offsets. It does not honour any frequency filter – the frequency filter is applied much later in the pipeline. The metadata of every mode in the file therefore becomes available in RAM, regardless of `freq_min`/`freq_max`. What the frequency filter *does* guard is the expensive eigenvector analysis in Phase 4: only modes whose frequency falls inside [`freq_min`, `freq_max`] trigger a full `analyze_mode_with_fallback` call. This distinction matters for the context-mode loading in section 2.5: the candidate set for context modes is the full set of modes in the log, not a re-scan.

2.2 Phase 2 – equilibrium geometry

⇝ `logio.read_std_orient`

The first Standard-Orientation block (selected by smallest byte offset in `so_off`) is parsed by `logio.read_std_orient`. This block lists, for every atom in the calculation:

- the *Center number* (1-based, called the *Gaussian centre* throughout this document),
- the *Atomic number* (used to look up the element symbol via the dictionary `logio._ELEM`),
- the equilibrium Cartesian coordinates (x, y, z) in Å (Gaussian writes them in the standard orientation, with the principal-axis frame chosen by Gaussian's symmetry detector).

The output is a list of `Atom` dictionaries $\{$`symbol`$, x, y, z,$ `center`$\}$ and an `idx_map` from Gaussian centre to position in the atoms list. By convention, *Gaussian centre* = position in the list +1, so the mapping is trivial – but it is computed explicitly because later filtering (heavy-atom-only mode, hydrogen-included mode) breaks the trivial correspondence.

The choice of equilibrium frame is significant: every subsequent geometric operation – the SVD-based cluster-normal computation in section 6, the Kabsch alignment for the PDB-to-Gaussian mapping in section 25, the SCSD reference geometry in section 15 – is expressed in this same frame.

2.3 Phase 3 – block metadata

⇝ `logio.read_all_meta`

`logio.read_all_meta` iterates over the `fr_off` list from Phase 1, opens the log file at each frequency-header offset, and reads the per-mode metadata of the corresponding mode block:

- the mode numbers in the block (3 or 5 modes per block, depending on `hpmodes`),
- the frequency of each mode in cm^{-1},
- the reduced mass in amu,
- the force constant in mDyne/Å,
- the IR intensity (parsed but currently unused downstream),

- the irreducible-representation label (`A`, `A'`, A_g, etc.; used for kernel scoring),
- a flag indicating whether the block is from a `hpmodes` section (precision $\sim 10^{-5}$ Å) or a standard section (precision $\sim 10^{-3}$ Å).

The eigenvector matrix itself is *not* loaded here – only the metadata. The reason is RAM: a single $N_{\text{atoms}} \times 3$ eigenvector matrix for a 10^4-atom system is 240 kB; for 10^4 modes that would be 2.4 GB. The metadata structure (`BlockInfo`, section 31) carries the byte offset and the column index within the block, so a subsequent `logio.get_eigvec` call can stream the eigenvector data in $3N$-row chunks on demand.

The function returns three structures:

- `all_blocks`: list of `BlockInfo`, one per `Frequencies -` header in the log;
- `best_block`: mapping from mode number to its preferred `BlockInfo` (HP block preferred over standard for the same mode number, used to reconcile mixed `hpmodes` + standard logs);
- `cand_map`: mapping from mode number to all `BlockInfo`s that contain that mode (used as a fallback ordering by `analyze_mode_with_fallback` if the preferred block fails to parse).

2.4 Phase 4 – main mode-analysis loop

↝ `runner._run_analysis_single`

The main loop iterates over the modes selected by the frequency filter:

Listing 1: Main mode-analysis loop, schematic.

```
selected <- []
for bi in all_blocks:
    for col, (mn, freq) in enumerate(zip(bi.mode_nums, bi.freqs)):
        if mn not in best_block or best_block[mn] is not bi:
            continue                # only count preferred block once
        if freq <= 0:
            log "imaginary or zero-frequency mode"; continue
        if cfg.freq_min and freq < cfg.freq_min: continue
        if cfg.freq_max and freq > cfg.freq_max: continue
        if cfg.freq_windows and freq not in any window:
            continue
        selected.append((bi, col, mn, freq))
selected.sort(by=freq)
for (bi, col, mn, freq) in selected:
    r, _ = analyze_mode_with_fallback(
              candidates=cand_map[mn], col=col,
              log_file=cfg.log_file, atoms=atoms, idx_map=idx_map,
              normal=cluster_normal, coord_info=coord_info,
              fe_c=fe_c, s_c=s_c, cfg=cfg, mode_num=mn)
    accumulate_b_factor(b_accum, r)
    results.append(r)
```

For every selected mode, the pipeline calls `core.analyze_mode_with_fallback`, which in turn calls `logio.get_eigvec` to stream the eigenvector from the log, scales it by u_{rms} (section 4), projects it onto the cluster-normal (section 5), and runs all the per-mode analyses summarised at the start of section 1. The return value is a result dictionary with the keys listed in section 36.

Phase 4 is the dominant computation cost. For a 1000-mode system, phase 4 typically takes $\sim$ 30 s; phases

1–3 together take $\sim$ 2–3 s; phases 5–7 take $\sim$ 1 s. The `cfg.n_jobs` option can parallelise phase 4 over multiple cores (default: serial, deterministic).

2.5 Phase 5 – context-mode loading

⇝ `runner._run_analysis_single`

After the main loop has produced `results` (a list of per-mode dicts for the modes inside the frequency window), two additional small mode sets are loaded for the sole purpose of anchoring the linear interpolation used in the `*_analysis_interp##.xlsx` workbook. These are the *context modes*.

The right-edge (upper-frequency) context set is selected from the window (`freq_max`, `freq_max`+`interp_context_cm1`]. If that window contains no mode – because the next mode above `freq_max` is farther than `interp_context_cm1` away – a single fallback mode is loaded: the closest mode above `freq_max` (this is *FUND 12*, section 73). If even that fails, because the DFT spectrum genuinely terminates inside the analysis range, a *synthetic zero anchor* is inserted at `freq_max` + `interp_context_cm1` with all observables equal to zero (this is *FUND 13*, section 74, produced by `runner._make_synthetic_zero_mode`). The synthetic mode carries sentinel values `number` $= -1$, `mode_type` = `"synthetic_zero"`, `precision` = `"synthetic"`.

The left-edge (lower-frequency) context set is selected symmetrically in [`freq_min`−`interp_context_cm1`, `freq_min`), again with the FUND 12 single-anchor fallback. At the lower edge no synthetic zero is inserted: if no real mode exists below `freq_min`, the interpolation's `left` $= 0.0$ boundary is already the physically correct behaviour (the spectrum starts at higher frequency).

Grid extent (v1.2.2). Context modes anchor the interpolation; they do not decide how far the grid reaches. That is settled by `export.interp_grid_bounds`, which since v1.2.2 reproduces the requested range exactly: the grid runs from `freq_min` to `freq_max` (in multi-window mode, from the hull of all windows for the top-level overall export), with a missing lower bound taken as 0.0 – the same convention as `Config.get_windows` and the `0-800_cm-1` folder label. Grid points with no mode nearby take the `np.interp` `left`/`right` $= 0.0$ value, which reads as "no modes in this range". Only a run with no requested range at all (`freq_min`, `freq_max` and `freq_windows` all unset) follows the data, padded by `interp_edge_extend`. Before v1.2.2 the data-driven fallback also applied whenever `freq_min`/`freq_max` were internally cleared – which is exactly what the multi-window overall export does so that `Config.outdir` creates no frequency subfolder – so a request for 0–100 cm^{-1} produced a top-level grid starting at the first real mode while the `0-100_cm-1` subfolder started at 0.00.

Multi-window context (v1.2.3). In multi-window mode the runner analyses all modes inside the hull of `freq_windows` and then exports once per window plus one overall export. Up to v1.2.2 two things went wrong at the hull edges. First, the context modes were only loaded when `freq_max` (`freq_min`) happened to be set – fields that are documented as ignored in multi-window mode – so a TOML with only `freq_windows` loaded none. Second, the per-window export took its context candidates from the already hull-filtered result list, so even when the runner had analysed them ("`97 context modes right`") the top window ended without a right anchor and emitted "`interp_boundary_mode='context' but no context modes available`"; `np.interp` then fell to `right` $= 0$ between the last analysed mode and the window bound. Since v1.2.3 the context ranges are derived from the hull ($\min lo$, $\max hi$; an open upper window loads no right context), the candidate pool of `runner._window_context` is the union of the analysed modes and both context lists, and the overall export receives the context lists as well.

Locked output files (v1.2.3). All Excel exporters save through `export._save_workbook`. A `PermissionError` on the target – on Windows almost always the file of the previous run still open in Excel – no longer drops the workbook: it is written to `<name>_1.xlsx` (`_2`, ...) and the fallback is printed on the console and recorded as a REPORT warning. The motivating case was a multi-window run whose interpolated Excel "looked uninterpolated": the REPORT carried "`interpolation Excel failed: [Errno 13] Permission denied`" and the file being inspected was the stale one of an earlier run.

2.6 Phase 6 – system-wide aggregates

After every mode has been analysed, the per-mode results are aggregated into system-wide quantities. These aggregations are exposed in the `*_analysis_main.xlsx` as separate sheets:

- B-factors (Debye-Waller): `b_factors` $= 8\pi^2/3 \cdot$ `b_accum`, with `b_accum` accumulated mode by mode during Phase 4 as $\sum_l u_{\mathrm{rms}}^2 \cdot \|\mathbf{e}_{l,i}\|^2$ (section 17).
- Total reorganization $\Lambda_X^{\mathrm{total}}$ per channel ($X \in \{\mathrm{FeFe}, \mathrm{FeN}, \mathrm{FeS}, \mathrm{NH}, \mathrm{HA}\}$), summed over all modes in the window (section 13).
- Modulation spectra $M_X(\omega)$, co-modulation spectra, cumulative-reorganization curves $\Lambda_X(\omega)$, gaussian-broadened on a uniform frequency grid (section 14).

2.7 Phase 7 – export

`⇝ export.export_all`

All per-mode results plus the system-wide aggregates are written to the configured Excel workbooks. The export layer uses openpyxl with a custom sheet-formatting helper that applies:

- per-cell colour-coding by value/sigma ratio (the *significance colouring* discussed in section 53),
- per-column number-format strings, chosen by quantity type,
- header rows with bold typography and merged-cell groupings for related columns.

When multi-cluster mode is active (`analyze_all_clusters` = true), phases 1–7 are repeated for each detected cluster, with the cluster index appearing in the output-directory name. Inter-cluster results are not currently aggregated – each cluster's outputs are independent.

2.8 Cache

`⇝ logio.load_scan_cache, logio.save_scan_cache`

The byte-offset scan from Phase 1 dominates the runtime for large log files. To avoid re-scanning when the user runs a second analysis on the same log (e.g. a different frequency window), the scan results are cached in a pickle file in the current working directory named `.scan_cache_##.pkl`, where `##` is a hash of:

- the absolute path of the log file,
- the log file's size in bytes,
- the log file's modification time (`os.stat.st_mtime`),

- the Python major version (3.11, 3.12, …; pickle format compatibility between Python major versions is not guaranteed),
- a hard-coded cache-format version string (`logio._CACHE_VERSION`) bumped whenever the format changes.

A cache hit skips the entire scan in Phase 1 and reduces end-to-end runtime by 40–80% depending on log file size. The cache is invalidated automatically by any of the keying changes; manual invalidation is to delete the `.scan_cache_##.pkl` file or set `use_cache` = false in the TOML.

3 Conventions used throughout the code

Quantum-chemistry codes have a number of subtle conventions that differ between programs, and ad-hoc choices have to be made about how those interact with the geometric/spectroscopic analysis we perform. This section enumerates every such choice explicitly. A reviewer should be able to verify each one against the original source code via the ⇝ markers.

3.1 Eigenvector normalisation

⇝ `logio.get_eigvec`

Gaussian writes Cartesian *mass-unweighted* displacement eigenvectors $\mathbf{e}_{l,i} \in \mathbb{R}^3$ for atom i in mode l (with $l = 1, \ldots, 3N-6$ for a non-linear molecule of N atoms), normalised so that

$$\sum_{i=1}^{N} \|\mathbf{e}_{l,i}\|^2 \;\approx\; 1. \tag{1}$$

The approximate sign reflects rounding to 5 (`hpmodes`) or 3 (`standard`) decimals. This is the *geometric-Cartesian* normalisation: the dimensionless eigenvector component $\mathbf{e}_{l,i}^{(\alpha)}$ describes the cartesian displacement of atom i along axis α in some abstract dimensionless "mode-unit" system. The mass-weighted eigenvector, which is the one for which $\sum_i m_i \|\boldsymbol{u}_{l,i}\|^2 = 1$ holds, is obtained from the displacement form as $\boldsymbol{u}_{l,i} = \sqrt{m_i}\,\mathbf{e}_{l,i}$ (after re-normalisation).

Why the distinction matters for the OOP/INP fraction. The OOP fraction defined in section 5,

$$\alpha_{\mathrm{OOP}}(G; l) = \frac{\sum_{i \in G} \|\mathbf{e}_{l,i}^{\mathrm{OOP}}\|^2}{\sum_{i \in G} \|\mathbf{e}_{l,i}\|^2},$$

is the *cartesian-displacement* OOP fraction, because Gaussian's `hpmodes` are in the displacement form. If one wanted the *kinetic-energy* fraction $T_{\mathrm{OOP}}/T_{\mathrm{total}}$, every $\|\mathbf{e}\|^2$ would have to be replaced by $m_i\|\mathbf{e}\|^2$. For a [2Fe-2S] cluster the two quantities are numerically similar – both Fe and both S have broadly comparable masses (56 vs 32 amu) – but the distinction is critical when comparing α_{OOP} values between chemically different systems. The manual flags this caveat; section 5 below derives the formula explicitly so that the convention is unambiguous.

3.2 Heavy-atom-only vs. hydrogen-included filtering

⇝ `orca_io.parseresult_to_atoms`, `orca_io.get_eigvec_orca`

By default, the cluster-core analysis operates on the heavy-atom subset of the molecule – explicit hydrogens are filtered out before the eigenvector matrix is reshaped. This reduces RAM and matches the chemical intuition that the Fe-S core dynamics are heavy-atom-dominated. Hydrogen-inclusion is opt-in via `include_hn_vibration` = true, which switches the relevant analysis (the His-NH stretch contribution, section 10) to a hydrogen-inclusive eigenvector loader.

Both loaders preserve the *Gaussian centre* (1-based index from the equilibrium-geometry block) as the unique atom identifier – the position within the heavy-only or all-H atoms list changes between loaders, but the centre does not. All cluster-atom centres (`fe_c`, `s_c`, `c2l`, `c2l_h`) are stored in Gaussian-centre coordinates, never in positional indices. The positional index is recovered when needed via atoms[`idx_map`[centre]].

3.3 Cluster atom ordering

⇝ `geometry.find_cluster, geometry.find_all_clusters`

The cluster-core atoms are stored in the canonical order $[\mathrm{Fe}_1, \mathrm{Fe}_2, \mathrm{S}_1, \mathrm{S}_2]$ throughout the code. This is essential for SCSD: the hard-coded model coordinates `core._SCSD_MODEL_COORDS` are also given in this order, and the SCSD projection assumes positional correspondence. The two Fe atoms and the two S atoms are sorted by Gaussian centre (smaller centre first), making the assignment deterministic regardless of how Gaussian numbers them. The chirality of the cluster (which Fe is "up") is then fixed by the SVD-based cluster-normal computation (section 6), which orients the normal in the $+\hat{\mathbf{z}}$ half-space of the Gaussian frame.

3.4 Sign conventions for bond modulations

⇝ `reorganization.signed_dr_along_axis`

For a generic bond modulation Δr_{ab} along atoms a and b, the sign convention used everywhere is

$$\Delta r_{ab}^{(\mathrm{class.})} \;=\; (\mathbf{e}_a - \mathbf{e}_b)\cdot\hat{\mathbf{n}}_{ab}, \qquad \hat{\mathbf{n}}_{ab} \;=\; \frac{\mathbf{r}_a - \mathbf{r}_b}{\|\mathbf{r}_a - \mathbf{r}_b\|}. \tag{2}$$

That is, the bond unit-vector $\hat{\mathbf{n}}_{ab}$ points from b to a, so $\Delta r_{ab} > 0$ means the bond is stretching (atoms a, b move apart) and $\Delta r_{ab} < 0$ means the bond is contracting. This convention is used uniformly for all five Marcus-Hush channels (FeFe, FeN, FeS, NH, HA), with the qualification that for HA the *reaction coordinate* variant (section 8) is used instead of the classical $\mathbf{e}_H - \mathbf{e}_A$ projection.

3.5 Units

Quantity	Unit	Remark
Atomic coordinates	Å (Å)	Gaussian standard-orientation.
Eigenvector components	dimensionless	Normalised to $\sum_i \|\mathbf{e}_{l,i}\|^2 \approx 1$.
Thermally scaled eigenvectors	Å	$\mathbf{e}_{l,i} \cdot u_{\mathrm{rms}}(\omega_l)$ in Å.
Frequencies	cm^{-1}	Standard spectroscopic convention.
Reduced mass	amu	As Gaussian writes it.
Force constants (Gaussian)	mDyne/Å	Parsed but not currently used in `core`.

Temperatures	K	SI.
Bond modulations Δr_X	Å	section 3.4.
Reorganization energies	cm^{-1}	Spectroscopic convention; conversion to other units in section 11.
B-factors	$\mathrm{Å}^2$	section 17.
SCSD amplitudes	Å	section 15.
SSE-element amplitudes	Å	

3.6 Sign of the cluster normal

⇝ `geometry.cluster_normal`

The right-singular vector $\mathbf{V}_{:,3}$ produced by the SVD in section 6 is determined up to a sign by the SVD itself. The convention adopted here is to choose the sign so that $\hat{\mathbf{n}} \cdot \hat{\mathbf{z}} > 0$, i.e. the cluster normal points into the $+\hat{\mathbf{z}}$ half-space of the Gaussian standard orientation. This fixes the sign of $\Delta r_{\mathrm{bend\_oop}}$ in the Fe-ligand decomposition of section 9. The bend_inp/bend_oop *absolute values* are unaffected by the choice of sign for $\hat{\mathbf{n}}$, only the sign of bend_oop is.

Part II

Every Formula in the Code

This part derives every non-trivial formula used by the code from first principles. Each derivation ends with a ⇝ marker that points to the implementing function in the source tree. The order follows the data-flow logic of section 2: from the lowest primitives (u_{rms}, OOP/INP) to the highest aggregates ($\Lambda_X^{\mathrm{total}}$, modulation spectra).

4 Thermal RMS amplitude u_{rms}

4.1 Quantum-mechanical derivation

Consider a one-dimensional quantum harmonic oscillator of mass m and angular frequency ω. The Hamiltonian $\hat{H} = \hat{p}^2/(2m) + \frac{1}{2}m\omega^2\hat{q}^2$ has energy eigenvalues $E_n = \hbar\omega(n + \frac{1}{2})$ and position-variance expectation values in eigenstate $|n\rangle$ [1]:

$$\langle n|\hat{q}^2|n\rangle \;=\; \frac{\hbar}{m\omega}\left(n + \tfrac{1}{2}\right). \tag{3}$$

In thermal equilibrium with reservoir temperature T, the expectation value of $\hat{q}^2$ is the canonical-ensemble average of equation (3) over the Boltzmann-weighted eigenstates:

$$\langle \hat{q}^2 \rangle_T \;=\; \frac{\sum_{n=0}^{\infty} e^{-\beta\hbar\omega(n+1/2)}\langle n|\hat{q}^2|n\rangle}{\sum_{n=0}^{\infty} e^{-\beta\hbar\omega(n+1/2)}}, \qquad \beta = \frac{1}{k_{\mathrm{B}}T}. \tag{4}$$

Both sums are geometric series in $e^{-\beta\hbar\omega}$. Working through (writing $x = \beta\hbar\omega/2$):

$$\sum_{n=0}^{\infty} e^{-\beta\hbar\omega(n+1/2)} = \frac{e^{-x}}{1-e^{-2x}} = \frac{1}{e^{x}-e^{-x}} = \frac{1}{2\sinh x}, \tag{5}$$

$$\sum_{n=0}^{\infty} (n+\tfrac{1}{2}) e^{-\beta\hbar\omega(n+1/2)} = -\frac{1}{\beta\hbar\omega}\frac{d}{dx}\left(\frac{1}{2\sinh x}\right) = \frac{\cosh x}{2\sinh^2 x}. \tag{6}$$

Substituting back,

$$\langle \hat{q}^2 \rangle_T \;=\; \frac{\hbar}{m\omega}\cdot\frac{\cosh x}{2\sinh^2 x}\cdot 2\sinh x \;=\; \frac{\hbar}{m\omega}\,\frac{\cosh x}{2\sinh x} \;=\; \frac{\hbar}{2m\omega}\,\coth\!\left(\frac{\hbar\omega}{2k_{\mathrm{B}}T}\right). \tag{7}$$

Taking the square root and identifying m with the mode's reduced mass m_{red}, we obtain the formula used throughout the code:

$$\boxed{u_{\mathrm{rms}}(\omega, m_{\mathrm{red}}, T) \;=\; \sqrt{\frac{\hbar}{2\,m_{\mathrm{red}}\,\omega}\,\coth\!\left(\frac{\hbar\omega}{2k_{\mathrm{B}}T}\right)}} \quad \rightsquigarrow \texttt{core.compute\_thermal\_amplitude} \tag{8}$$

in SI units. The code returns the result in Å by multiplying by 10^{10} before returning.

4.2 Numerical implementation details

↝ core.compute_thermal_amplitude

The function `core.compute_thermal_amplitude(freq_cm1, red_mass_amu, temp_k, amplitude)` converts inputs into SI ($\omega = 2\pi c\cdot\tilde{\nu}$, c in cm/s), evaluates equation (8), and returns the result in Å. Several edge cases are handled defensively:

- *Imaginary or zero frequency*: $\omega \le 0$ returns 0. The mode-loop already filters such modes (section 2.4), so this branch is a safeguard.
- *Negative or zero reduced mass*: $m_{\mathrm{red}} \le 0$ returns 0. Should never happen for a valid Gaussian log.
- *Classical mode*: `temp_k` = `None` triggers the classical-amplitude convention – u_{rms} is replaced by the constant `amplitude` (default 1.0), ignoring frequency and mass. This is for benchmarking against legacy tools that use a fixed amplitude.
- *Numerical* coth: for $\beta\hbar\omega/(2) \ll 1$ (i.e. very low ω or very high T), the $\coth(x) \approx 1/x$ regime is approached. The code uses `numpy.cosh`/`numpy.sinh` directly; for IEEE-754 doubles this is stable down to $\omega \approx 10^{-7}$ cm^{-1}, well below any physically meaningful regime.
- *Zero-point limit*: $T \to 0$ gives $\coth(\beta\hbar\omega/2) \to 1$ and $u_{\mathrm{rms}} \to \sqrt{\hbar/(2m_{\mathrm{red}}\omega)}$. This is the zero-point amplitude.

Sanity checks (test_sanity_physics.py)

The following property tests verify equation (8) analytically:

- `test_urms_zero_T_limit`: with $T = 10^{-3}$ K, the result matches $\sqrt{\hbar/(2m_{\mathrm{red}}\omega)}$ to 10^{-6} relative error.
- `test_urms_classical_limit`: with $\omega = 50$ cm^{-1} and $T = 2000$ K, the result matches the

classical equipartition limit $\sqrt{k_{\mathrm{B}}T/(m_{\mathrm{red}}\omega^2)}$ to $< 1\%$ relative error.

- `test_urms_monotonic_in_T`: u_{rms} is monotonic increasing in T for fixed ω, m_{red}.

4.3 Limiting behaviours

The two important limits are:

Zero-point limit. For $\hbar\omega \gg k_{\mathrm{B}}T$, $\coth(x) \to 1$, so

$$u_{\mathrm{rms}} \xrightarrow{T\to 0} \sqrt{\frac{\hbar}{2m_{\mathrm{red}}\omega}}. \tag{9}$$

For a 200-cm^{-1} mode with $m_{\mathrm{red}} = 30$ amu, this is about 0.04 Å.

Classical / equipartition limit. For $\hbar\omega \ll k_{\mathrm{B}}T$, $\coth(x) \to 1/x = 2k_{\mathrm{B}}T/(\hbar\omega)$, so

$$u_{\mathrm{rms}} \xrightarrow{T\to\infty} \sqrt{\frac{k_{\mathrm{B}}T}{m_{\mathrm{red}}\omega^2}}. \tag{10}$$

This is the equipartition theorem $\frac{1}{2}m\omega^2\langle q^2\rangle = \frac{1}{2}k_{\mathrm{B}}T$. For a 50-cm^{-1} mode at room temperature, $\hbar\omega/k_{\mathrm{B}} = 72$ K, so the classical limit is approximately satisfied.

Worked numerical example: u_{rms} for an Fe-S stretch at 80 K

We compute u_{rms} for a typical Fe-S symmetric-stretch mode of a [2Fe-2S] cluster, using representative values from a benchmark log (such values are typical for Cys_2His_2- or Cys_4-coordinated $[2Fe\text{-}2S]^{1+/2+}$ systems):

- $\tilde{\nu} = 320$ cm^{-1} (frequency),
- $m_{\mathrm{red}} = 31.97$ amu (reduced mass; printed by Gaussian as "Red. masses –" for this mode),
- $T = 80$ K (cryogenic NRVS condition).

Step 1 – convert to SI:

$$\begin{aligned}
\omega &= 2\pi c \cdot \tilde{\nu} = 2\pi \cdot 2.998 \times 10^{10}\ \text{cm/s} \cdot 320\ \text{cm}^{-1} \\
&= 6.026 \times 10^{13}\ \text{rad/s}. \\
m_{\mathrm{red}} &= 31.97 \cdot 1.6605 \times 10^{-27}\ \text{kg} = 5.309 \times 10^{-26}\ \text{kg}. \\
\beta\hbar\omega/2 &= \frac{\hbar\omega}{2k_{\mathrm{B}}T} = \frac{1.0546 \times 10^{-34} \cdot 6.026 \times 10^{13}}{2 \cdot 1.3807 \times 10^{-23} \cdot 80} \\
&= \frac{6.354 \times 10^{-21}}{2.209 \times 10^{-21}} = 2.876.
\end{aligned}$$

Step 2 – evaluate coth:

$$\coth(2.876) = \frac{e^{2.876} + e^{-2.876}}{e^{2.876} - e^{-2.876}} = \frac{17.74 + 0.0564}{17.74 - 0.0564} = 1.0064.$$

Since $\hbar\omega/k_{\mathrm{B}}T = 5.75 \gg 1$, we are well inside the zero-point-dominated regime; coth is close to 1.

Step 3 – assemble:

$$u_{\mathrm{rms}}^2 = \frac{\hbar}{2\,m_{\mathrm{red}}\,\omega}\,\coth(\beta\hbar\omega/2) = \frac{1.0546\times10^{-34}}{2\cdot5.309\times10^{-26}\cdot6.026\times10^{13}}\cdot1.0064.$$

$$= \frac{1.0546}{6.398}\times10^{-22}\cdot1.0064\ \mathrm{m}^2 = 1.659\times10^{-23}\ \mathrm{m}^2.$$

$$u_{\mathrm{rms}} = \sqrt{1.659\times10^{-23}}\ \mathrm{m} = 4.073\times10^{-12}\ \mathrm{m} = \boxed{0.04073\ \text{Å}}.$$

Cross-check (zero-point limit): With coth $\to 1$ exactly, $u_{\mathrm{rms}}^{\mathrm{ZP}} = \sqrt{\hbar/(2m\omega)} = 0.04062$ Å. The thermal correction at 80 K is therefore only $\sim 0.3\%$ – as expected for $\hbar\omega/k_{\mathrm{B}}T \gg 1$.
Cross-check (classical limit): $\sqrt{k_{\mathrm{B}}T/(m\omega^2)} = \sqrt{1.105\times10^{-21}/(5.309\times10^{-26}\cdot3.631\times10^{27})} \approx 0.0240$ Å. The true value 0.041 Å is far above this – the classical limit gives *less* amplitude than the quantum result at 80 K, confirming we are in the quantum regime.
In the code: `core.compute_thermal_amplitude(320.0, 31.97, 80.0, 1.0)` returns `0.04073` (the third argument is `temp_k`; the fourth, `amplitude`, is unused when `temp_k` is set).

4.4 Thermally scaled eigenvector

The thermally scaled eigenvector $\widetilde{\mathbf{e}}_{l,i} \equiv \mathbf{e}_{l,i}\cdot u_{\mathrm{rms}}(\omega_l)$ has units of Å and *is the displacement of atom i in mode l averaged over the thermal density matrix.* Every downstream geometric observable (bond modulation, bend, group-amplitude, SCSD amplitude, SSE-element amplitude, C_α amplitude) is computed from $\widetilde{\mathbf{e}}$, not from the raw $\mathbf{e}$. This means that:

- All observables are in Å and have a direct physical interpretation as "how far does the atom actually move at temperature T".
- The accompanying σ for each observable must scale the *raw* per-component eigenvector noise $\sigma_e =$ `sigma_eigvec` by the same u_{rms} factor, $\sigma_e^{\mathrm{therm}} = \sigma_e \cdot u_{\mathrm{rms}}$ (part VII). Failure to do this was the *FUND 6/7/8* bug class fixed in v1.0.4 (section 69).

5 OOP/INP decomposition

5.1 Definition

↝ `core._oop, core.classify_oop_inp`

Given a unit cluster-normal $\hat{\mathbf{n}}$ (section 6) and a set G of atom indices, define the per-atom out-of-plane component of the eigenvector $\mathbf{e}_{l,i}$ (or its thermally scaled form $\widetilde{\mathbf{e}}_{l,i}$ – the ratio is the same):

$$\mathbf{e}_{l,i}^{\mathrm{OOP}} \equiv (\mathbf{e}_{l,i}\cdot\hat{\mathbf{n}})\,\hat{\mathbf{n}}. \tag{11}$$

This is the projection of $\mathbf{e}_{l,i}$ onto the cluster-normal direction. The orthogonal complement, $\mathbf{e}_{l,i}^{\mathrm{INP}} = \mathbf{e}_{l,i} - \mathbf{e}_{l,i}^{\mathrm{OOP}}$, lies in the cluster plane.

The OOP fraction of mode l on group G is the ratio of squared norms:

$$\boxed{\alpha_{\mathrm{OOP}}(G;l) \;=\; \frac{\sum_{i\in G}\|\mathbf{e}_{l,i}^{\mathrm{OOP}}\|^2}{\sum_{i\in G}\|\mathbf{e}_{l,i}\|^2}\;\cdot\;100\,\%} \qquad \rightsquigarrow \texttt{core.classify\_oop\_inp} \tag{12}$$

The INP fraction is $\alpha_{\mathrm{INP}}(G;l) = 1 - \alpha_{\mathrm{OOP}}(G;l)$ (after the percentage conversion).

Pythagorean orthogonality (test_sanity_physics.py)

The decomposition equation (11) satisfies the Pythagorean identity

$$\|\mathbf{e}_{l,i}\|^2 = \|\mathbf{e}_{l,i}^{\mathrm{OOP}}\|^2 + \|\mathbf{e}_{l,i}^{\mathrm{INP}}\|^2$$

because $\mathbf{e}^{\mathrm{OOP}}$ and $\mathbf{e}^{\mathrm{INP}}$ are orthogonal by construction. Consequently $\alpha_{\mathrm{OOP}} + \alpha_{\mathrm{INP}} = 1$ exactly, with no rounding error beyond machine precision. The unit test `test_mixed_oop_inp_satisfies_pythagoras` verifies this on a random eigenvector to $< 10^{-12}$ absolute error.

Worked numerical example: OOP/INP for a 4-atom mixed mode

We compute α_{OOP} for a synthetic cluster mode that mixes umbrella (z-direction) and breathing (xy-plane) motion. The cluster has $\hat{\mathbf{n}} = (0,0,1)$ (the $+\hat{\mathbf{z}}$-axis), and the 4 cluster-atom eigenvector components are

Atom	e_x	e_y	e_z
Fe_1	+0.30	+0.20	+0.40
Fe_2	−0.30	−0.20	+0.40
S_1	+0.20	+0.30	−0.30
S_2	−0.20	−0.30	−0.30

Step 1 – decompose each atom: The OOP component of atom i is $(\mathbf{e}_i \cdot \hat{\mathbf{n}})\,\hat{\mathbf{n}} = (e_z)_i \cdot (0,0,1)$. So $\|\mathbf{e}_i^{\mathrm{OOP}}\|^2 = (e_z)_i^2$, and $\|\mathbf{e}_i\|^2 = e_x^2 + e_y^2 + e_z^2$.

Atom	e_x^2	e_y^2	e_z^2	$\|\mathbf{e}\|^2$	$\|\mathbf{e}^{\mathrm{OOP}}\|^2$
Fe_1	0.09	0.04	0.16	0.29	0.16
Fe_2	0.09	0.04	0.16	0.29	0.16
S_1	0.04	0.09	0.09	0.22	0.09
S_2	0.04	0.09	0.09	0.22	0.09
Sum				1.02	0.50

Step 2 – ratio:

$$\alpha_{\mathrm{OOP}} = \frac{0.50}{1.02}\cdot 100\% = \boxed{49.02\,\%}, \qquad \alpha_{\mathrm{INP}} = 100\% - 49.02\% = 50.98\%.$$

With $\theta_{\mathrm{OOP}} = 0.7$ (the default `oop_threshold`), this mode is classified as `mixed` (equation (14)), and the 8-bin detailed classifier returns `mixed_inp` (50–70% INP range).

Pythagorean cross-check: $\alpha_{\mathrm{OOP}} + \alpha_{\mathrm{INP}} = 49.02 + 50.98 = 100.0\%$ exact.

Note on normalisation: The sum $\|\mathbf{e}\|^2_{\mathrm{cluster}} = 1.02$ is slightly above 1, which is normal because the cluster atoms carry only part of the mode's total amplitude (other atoms – ligands, backbone – carry the rest, summing to ≈ 1 over the whole molecule).

In the code: `core.classify_oop_inp(evec_4x3, normal=[0,0,1], fe_c, s_c, cfg)` returns a dict containing `a_oop_pct = 49.02`, `a_inp_pct = 50.98`, `mode_type = "mixed"`, `mode_type_detail = "mixed_inp"`.

5.2 Choice of normalisation: geometric Cartesian fraction

The numerator and denominator of equation (12) are both unweighted sums of squared cartesian-displacement components. They *do not* carry atomic masses. In the literature one sometimes sees an alternative definition

$$\alpha_{\mathrm{OOP}}^{\mathrm{KE}}(G;l) \;=\; \frac{\sum_{i\in G} m_i \|\mathbf{e}_{l,i}^{\mathrm{OOP}}\|^2}{\sum_{i\in G} m_i \|\mathbf{e}_{l,i}\|^2} \tag{13}$$

where each squared norm is weighted by atomic mass. For a mass-weighted eigenvector, $\alpha_{\mathrm{OOP}}^{\mathrm{KE}}$ is the fraction of *kinetic energy* flowing out of plane:

$$T_{\mathrm{OOP}}/T_{\mathrm{total}} \;=\; \frac{\sum_i m_i \|\dot{\mathbf{e}}_{l,i}^{\mathrm{OOP}}\|^2}{\sum_i m_i \|\dot{\mathbf{e}}_{l,i}\|^2} \;=\; \frac{\sum_i m_i \|\mathbf{e}_{l,i}^{\mathrm{OOP}}\|^2}{\sum_i m_i \|\mathbf{e}_{l,i}\|^2}$$

(the time-derivative factors cancel because every mode oscillates at the same ω_l).

The code uses equation (12) – the geometric Cartesian fraction – not equation (13). The reason is operational: Gaussian's `hpmodes` writes cartesian-displacement eigenvectors (section 3.1), and we want α_{OOP} to answer the geometric question "what fraction of the atom-by-atom displacement points out of plane", not the energetic question. For a [2Fe-2S] cluster the two are numerically close (Fe mass 55.93 amu, S mass 31.97 amu; the mass-weighted version shifts the answer by no more than a few percent), but for a chemically dissimilar system (e.g. a heavy-metal-light-ligand complex) the two can differ substantially.

5.3 Multi-atom aggregation

↝ `core.classify_oop_inp`

For the cluster-core group $G = \{\mathrm{Fe}_1, \mathrm{Fe}_2, \mathrm{S}_1, \mathrm{S}_2\}$, the OOP fraction in equation (12) aggregates over all four atoms. For multi-residue groups defined in `coord_info.group_map`, the aggregation extends over every atom in the group's centre-list. The numerator and denominator are computed in a single pass, which is both faster than two passes and exact (no catastrophic cancellation possible since all summands are non-negative).

5.4 Threshold-based binary classification

↝ `core.classify_oop_inp`

For diagnostic purposes the cluster-core α_{OOP} value is binned into a binary class:

$$\mathrm{mode\_type}(l) \;=\; \begin{cases} \mathrm{OOP}, & \alpha_{\mathrm{OOP}} \geq \theta_{\mathrm{OOP}}, \\ \mathrm{INP}, & \alpha_{\mathrm{OOP}} \leq 1 - \theta_{\mathrm{OOP}}, \\ \mathrm{mixed}, & \text{otherwise}, \end{cases} \tag{14}$$

with the threshold θ_{OOP} = `oop_threshold` (default 0.7, i.e. 70%). A finer eight-bin classification is also computed and recorded under the key `mode_type_detail`: `pure_oop` ($>95\%$), `strong_oop` (85–95%),

`majority_oop` (70–85%), `mixed_oop` (50–70%), `mixed_inp` (30–50%), `majority_inp` (15–30%), `strong_inp` (5–15%), `pure_inp` (< 5%).

5.5 Sanity tests

Sanity checks (test_sanity_physics.py)

- `test_pure_oop_mode_yields_alpha_oop_100`: a synthetic eigenvector with displacement purely along $\hat{\mathbf{n}}$ (umbrella mode) yields $\alpha_{\mathrm{OOP}} = 100\%$ exactly.
- `test_pure_inp_mode_yields_alpha_oop_0`: a synthetic eigenvector with displacement entirely in the cluster plane (breathing mode) yields $\alpha_{\mathrm{OOP}} = 0\%$ exactly.
- `test_mixed_oop_inp_satisfies_pythagoras`: for a random eigenvector, $\alpha_{\mathrm{OOP}} + \alpha_{\mathrm{INP}} = 1$ to $< 10^{-12}$.

6 Cluster-normal via SVD

6.1 Best-fit plane derivation

↝ `geometry.cluster_normal`

For the four cluster-core atom positions $\{\mathbf{r}_i\}_{i=1}^{4} = \{\mathbf{r}_{\mathrm{Fe}_1}, \mathbf{r}_{\mathrm{Fe}_2}, \mathbf{r}_{\mathrm{S}_1}, \mathbf{r}_{\mathrm{S}_2}\}$, we wish to find the plane that best approximates the four points in the least-squares sense. Equivalently, we seek a unit vector $\hat{\mathbf{n}}$ such that the sum of squared signed distances from the four points to the plane $\hat{\mathbf{n}} \cdot (\mathbf{r} - \bar{\mathbf{r}}) = 0$ is minimised:

$$\hat{\mathbf{n}}^{\star} \;=\; \arg\min_{\|\hat{\mathbf{n}}\|=1} \sum_{i=1}^{4} (\hat{\mathbf{n}} \cdot \tilde{\mathbf{r}}_i)^2, \qquad \tilde{\mathbf{r}}_i \;\equiv\; \mathbf{r}_i - \bar{\mathbf{r}}, \qquad \bar{\mathbf{r}} \;\equiv\; \tfrac{1}{4} \sum_{i=1}^{4} \mathbf{r}_i. \tag{15}$$

Writing the centred positions as the rows of a 4×3 matrix $\tilde{\mathbf{R}}$, the objective becomes $\|\tilde{\mathbf{R}}\hat{\mathbf{n}}\|^2 = \hat{\mathbf{n}}^{\top} \tilde{\mathbf{R}}^{\top} \tilde{\mathbf{R}}\, \hat{\mathbf{n}}$. By the Courant-Fischer-Weyl min-max theorem, the minimum (over unit vectors) is the smallest eigenvalue of the 3×3 matrix $\tilde{\mathbf{R}}^{\top}\tilde{\mathbf{R}}$, and the minimiser is the corresponding eigenvector.

The eigendecomposition of $\tilde{\mathbf{R}}^{\top}\tilde{\mathbf{R}}$ is read off the SVD of $\tilde{\mathbf{R}}$. Writing $\tilde{\mathbf{R}} = \mathbf{U}\,\boldsymbol{\Sigma}\,\mathbf{V}^{\top}$ with singular values $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$, we have $\tilde{\mathbf{R}}^{\top}\tilde{\mathbf{R}} = \mathbf{V}\,\boldsymbol{\Sigma}^2\,\mathbf{V}^{\top}$, so the smallest-eigenvalue eigenvector is $\mathbf{V}_{:,3}$:

$$\boxed{\tilde{\mathbf{R}} \;=\; \mathbf{U}\,\boldsymbol{\Sigma}\,\mathbf{V}^{\top}, \qquad \hat{\mathbf{n}} \;=\; \pm\,\mathbf{V}_{:,3}} \quad \text{↝ } \texttt{geometry.cluster\_normal} \tag{16}$$

6.2 Sign convention

The right-singular vector $\mathbf{V}_{:,3}$ is determined up to a sign by the SVD. The convention adopted is

$$\hat{\mathbf{n}} \cdot \hat{\mathbf{z}} \;\geq\; 0, \tag{17}$$

i.e. the cluster normal points into the upper half-space of the Gaussian standard orientation. If $\mathbf{V}_{:,3} \cdot \hat{\mathbf{z}} < 0$, $\hat{\mathbf{n}}$ is flipped. This makes the $\Delta r_{\mathrm{bend\_oop}}$ sign in the Fe-ligand decomposition (section 9) reproducible across runs.

6.3 Singular-value spread as planarity diagnostic

The three singular values $\sigma_1, \sigma_2, \sigma_3$ have a geometric interpretation: σ_1^2 is the variance of the centred points along the first principal axis, σ_2^2 along the second, σ_3^2 along the third (= the cluster-normal direction). For a perfectly planar four-atom set, $\sigma_3 = 0$. For real [2Fe-2S] clusters, σ_3 is typically < 0.05 Å (the rhombic Fe_2S_2 core is genuinely planar within DFT precision), and the ratio σ_3/σ_1 provides a planarity diagnostic that is recorded in the run-log.

Worked numerical example: Cluster-normal SVD for a near-planar Fe_2S_2 core

We compute $\hat{\mathbf{n}}$ for a typical rhombic Fe_2S_2 core, using representative coordinates from a DFT-optimised [2Fe-2S] cluster (the slight z-perturbation reflects the typical out-of-plane spread of ∼0.02 Å observed in any real cluster and makes the SVD non-degenerate):

Atom	x [Å]	y [Å]	z [Å]
Fe_1	+1.350	0.000	+0.012
Fe_2	−1.350	0.000	+0.018
S_1	0.000	+1.075	−0.020
S_2	0.000	−1.075	−0.010

Step 1 – centroid and centred positions:

$$\bar{\mathbf{r}} = \tfrac{1}{4}(1.35 - 1.35 + 0 + 0,\ 0 + 0 + 1.075 - 1.075,\ 0.012 + 0.018 - 0.020 - 0.010) = (0,\ 0,\ 0).$$

The centroid is at the origin (by construction in standard orientation). The centred-position matrix $\tilde{\mathbf{R}} \in \mathbb{R}^{4\times 3}$ is then identical to the position matrix above.

Step 2 – SVD: The Gram matrix $\mathbf{H} = \tilde{\mathbf{R}}^\top \tilde{\mathbf{R}}$ is

$$\mathbf{H} = \begin{pmatrix} 2 \cdot 1.350^2 & 0 & 0 \\ 0 & 2 \cdot 1.075^2 & 0 \\ 0 & 0 & 0.000888 \end{pmatrix} = \begin{pmatrix} 3.645 & 0 & 0 \\ 0 & 2.311 & 0 \\ 0 & 0 & 0.000888 \end{pmatrix}$$

(the off-diagonals are zero because the atom positions decouple in x, y, z for this geometry). Eigenvalues: $\lambda_1 = 3.645, \lambda_2 = 2.311, \lambda_3 = 8.88 \times 10^{-4}$. Singular values: $\sigma_1 = \sqrt{3.645} = 1.909$ Å, $\sigma_2 = 1.520$ Å, $\sigma_3 = 0.0298$ Å.

Step 3 – read off $\hat{\mathbf{n}}$: The right-singular vector for σ_3 is $\mathbf{V}_{:,3} = (0, 0, \pm 1)$.

Step 4 – enforce sign convention. Check $\mathbf{V}_{:,3} \cdot \hat{\mathbf{z}}$: if SVD returns $(0, 0, +1)$, keep it; if $(0, 0, -1)$, flip. Result: $\boxed{\hat{\mathbf{n}} = (0,\ 0,\ +1)}$.

Planarity diagnostic: $\sigma_3/\sigma_1 = 0.0298/1.909 = 1.56\%$. The cluster is highly planar; the run-log records this ratio. Values above 5% would indicate a strongly tetrahedrally-distorted cluster (e.g. a super-reduced [4Fe-4S]-like distortion accidentally tripped by section 24).

In the code: `geometry.cluster_normal([Fe1, Fe2, S1, S2])` returns `normal = [0.0, 0.0, 1.0]` and additionally exposes the three singular values via the diagnostic key `cluster_planarity_ratio = 0.0156`.

6.4 Multi-cluster considerations

⇝ `geometry.find_all_clusters`

For a system with K [2Fe-2S] clusters, each cluster runs `cluster_normal` on its own four atoms. The normals are *not* forced to align across clusters; each cluster's normal is fixed independently by the $\hat{\mathbf{n}} \cdot \hat{\mathbf{z}} \geq 0$ rule. The pipeline's per-cluster output directories carry the cluster index so the user can identify which cluster a given $\hat{\mathbf{n}}$ belongs to.

7 Classical bond modulation Δr_{ab}

7.1 Linearisation of a bond length under small displacements

⇝ `reorganization.signed_dr_along_axis`

Consider two atoms a, b with equilibrium positions $\mathbf{r}_a, \mathbf{r}_b$ and small displacements $\mathbf{e}_a, \mathbf{e}_b$. The bond length under displacement is

$$r_{ab}(\mathbf{e}) = \|(\mathbf{r}_a + \mathbf{e}_a) - (\mathbf{r}_b + \mathbf{e}_b)\|, \qquad r_{ab}^{(0)} = \|\mathbf{r}_a - \mathbf{r}_b\|. \tag{18}$$

Expand around $\mathbf{e} = 0$:

$$r_{ab}(\mathbf{e}) \;=\; \|\mathbf{r}_a - \mathbf{r}_b + (\mathbf{e}_a - \mathbf{e}_b)\| \;=\; r_{ab}^{(0)} \sqrt{1 + 2(\mathbf{e}_a - \mathbf{e}_b) \cdot \hat{\mathbf{n}}_{ab} / r_{ab}^{(0)} + \|\mathbf{e}_a - \mathbf{e}_b\|^2 / (r_{ab}^{(0)})^2}. \tag{19}$$

With $\hat{\mathbf{n}}_{ab} = (\mathbf{r}_a - \mathbf{r}_b)/r_{ab}^{(0)}$ and to first order in the displacement,

$$r_{ab}(\mathbf{e}) - r_{ab}^{(0)} \;\approx\; (\mathbf{e}_a - \mathbf{e}_b) \cdot \hat{\mathbf{n}}_{ab}. \tag{20}$$

This is the linearised bond-modulation formula. Defining

$$\boxed{\Delta r_{ab}^{(\text{class.})} \;\equiv\; (\mathbf{e}_a - \mathbf{e}_b) \cdot \hat{\mathbf{n}}_{ab}} \quad \rightsquigarrow \texttt{reorganization.signed\_dr\_along\_axis} \tag{21}$$

recovers the formula used throughout the code.

7.2 Sign and physical interpretation

The convention $\hat{\mathbf{n}}_{ab} = (\mathbf{r}_a - \mathbf{r}_b)/r_{ab}^{(0)}$ makes $\hat{\mathbf{n}}_{ab}$ point from b to a. Therefore:

- $\Delta r_{ab} > 0$: atom a moves in $+\hat{\mathbf{n}}_{ab}$ direction relative to atom b, i.e. they move apart – the bond *stretches*.
- $\Delta r_{ab} < 0$: the bond *contracts*.
- $\Delta r_{ab} = 0$: motion is either zero or purely perpendicular to the bond.

Sanity checks (test_sanity_physics.py)

- `test_pure_translation_mode_yields_zero_bond_modulation`: for any bond, a uniform translation $\mathbf{e}_a = \mathbf{e}_b$ gives $\Delta r_{ab} = 0$ exactly.
- `test_pure_rotation_mode_yields_zero_bond_stretch`: a rigid rotation gives $\Delta r_{ab} = 0$ to first order (displacements perpendicular to the bond axis).
- `test_pure_bond_stretching_mode_yields_full_modulation`: $\mathbf{e}_a = (+\Delta, 0, 0)$, $\mathbf{e}_b = (-\Delta, 0, 0)$ along $\hat{\mathbf{x}} = \hat{\mathbf{n}}_{ab}$ gives $\Delta r_{ab} = 2\Delta$ exactly.

7.3 Validity of the linear approximation

The full expression equation (20) retains a second-order correction $\Delta r_{ab}^{(2)} = \frac{1}{2}\|\mathbf{e}_a - \mathbf{e}_b\|_\perp^2 / r_{ab}^{(0)}$ where $\|\cdot\|_\perp$ is the component perpendicular to $\hat{\mathbf{n}}_{ab}$. For typical thermally scaled displacements (~ 0.05 Å) on a typical Fe-S bond ($r_{ab}^{(0)} \approx 2.2$ Å), the second-order term is $\sim 5 \times 10^{-4}$ Å, which is smaller than the hpmodes-precision noise floor of $\sim 5 \times 10^{-4}$ Å times u_{rms}. The linearisation is therefore well within the precision budget of the input data.

7.4 Application to the five Marcus-Hush channels

The classical formula equation (21) is used directly for four of the five channels:

- $X = \mathrm{FeFe}$: $a = \mathrm{Fe}_1$, $b = \mathrm{Fe}_2$, one pair per cluster.
- $X = \mathrm{FeN}$: $a = \mathrm{Fe}$, $b = \mathrm{N}$, one pair per Fe-N(His) coordination.
- $X = \mathrm{FeS}$: $a = \mathrm{Fe}$, $b = \mathrm{S}$, one pair per Fe-S coordination (both bridging Fe-S_{μ_2} and terminal Fe-$\mathrm{S}_{\mathrm{Cys}}$).
- $X = \mathrm{NH}$: $a = \mathrm{N}_{\mathrm{His}}$, $b = \mathrm{H}$, one pair per protonated His-N-H bond.

For the fifth channel $X = \mathrm{HA}$ (hydrogen-acceptor), the classical formula would conflate genuine proton-transfer motion with skeletal motion of the acceptor. The reaction-coordinate variant section 8 is used instead.

Worked numerical example: Fe-S bond modulation for a 320 cm^{-1} mode

We compute $\Delta r_{\mathrm{Fe_1\text{-}S_1}}$ for the synthetic "symmetric Fe-S stretch" mode using the equilibrium geometry from the cluster-normal example (section 6) and a stretch-like eigenvector.

Setup:

Atom	$\mathbf{r}$ [Å]	$\mathbf{e}$ (mass-unweighted)
Fe_1	(+1.350, 0, +0.012)	(+0.42, 0, 0)
S_1	(0, +1.075, −0.020)	(0, +0.33, 0)

Step 1 – Bond geometry:

$$\mathbf{r}_{\mathrm{Fe_1}} - \mathbf{r}_{\mathrm{S_1}} = (+1.350,\ -1.075,\ +0.032), \qquad \|\cdot\| = \sqrt{1.823 + 1.156 + 0.001} = 1.726 \text{ Å}.$$

Bond unit vector: $\hat{\mathbf{b}} = (1.350, -1.075, +0.032)/1.726 = (+0.782, -0.623, +0.019)$.

Step 2 – Displacement difference:

$$\Delta\mathbf{e} = \mathbf{e}_{\mathrm{Fe_1}} - \mathbf{e}_{\mathrm{S_1}} = (+0.42 - 0,\ 0 - 0.33,\ 0 - 0) = (+0.42,\ -0.33,\ 0).$$

Step 3 – Project on bond axis (raw, before u_{rms}):

$$\Delta r_{\mathrm{stretch}}^{(\mathrm{raw})} = \Delta\mathbf{e} \cdot \hat{\mathbf{b}} = (0.42)(0.782) + (-0.33)(-0.623) + (0)(0.019)$$

$$= 0.328 + 0.206 + 0 = +0.534 \quad \text{(dimensionless eigenvector units)}.$$

Step 4 – Apply thermal scaling (80 K, $u_{\mathrm{rms}} = 0.0407$ Å):

$$\Delta r_{\mathrm{stretch}} = 0.534 \cdot 0.0407 \text{ Å} = \boxed{+0.0217 \text{ Å}}.$$

Positive sign: Fe_1 moves *away* from S_1 along $\hat{\mathbf{b}}$ – the bond is stretching in the upstroke of this oscillation. (At a phase half-period later the sign would flip; the reported magnitude is the amplitude of the oscillation.)

Step 5 – Verify orthogonality of bend. The perpendicular component $\Delta\mathbf{e} - 0.534\,\hat{\mathbf{b}} = (0.42 - 0.417, -0.33 + 0.333, -0.010) = (+0.003, +0.003, -0.010)$. Its norm is 0.0108, contributing the bend_inp/bend_oop. The Pythagorean check $0.534^2 + 0.0108^2 = 0.285 + 0.000117 \approx 0.285 = \|\Delta\mathbf{e}\|^2$ confirms decomposition.

Sigma: $\sigma_{\text{stretch}} = \sqrt{2} \cdot 5\times10^{-4} \cdot 0.0407$ Å $= 2.88 \times 10^{-5}$ Å, so the stretch is $0.0217/2.88\times10^{-5} \approx 750\sigma$ significant – deep above the noise floor.

In the code: `reorganization.signed_dr_along_axis(evg_Fe1, evg_S1, r_Fe1, r_S1)` returns `0.0217` (the function takes already-scaled eigenvectors, hence the result is in Å directly).

8 Reaction-coordinate Δr_{HA}

8.1 Why a different formula for HA

The proton-transfer (PT) reaction coordinate is, in the spirit of Marcus-Hush theory, the displacement of the proton along the N–A axis (where A is the H-bond acceptor, typically a backbone or side-chain carbonyl-O or amine-N). The classical formula equation (21) applied to the pair (H, A) would read $(\mathbf{e}_H - \mathbf{e}_A) \cdot \hat{\mathbf{n}}_{HA}$, which captures the change in H-to-A distance. But that distance changes both when the proton moves (genuine PT character) and when the acceptor moves while the proton is stationary (skeletal mode, no PT character).

For PCET analysis we want a coordinate that:

- is nonzero only when the proton moves relative to its *donor* (the N_{His});
- points along the N-to-A axis (so that the sign distinguishes PT toward A from PT away from A).

The unique linear functional of $(\mathbf{e}_N, \mathbf{e}_H, \mathbf{e}_A)$ satisfying both is

$$\Delta r_{HA}^{(\text{react. coord.})} \equiv (\mathbf{e}_H - \mathbf{e}_N) \cdot \hat{\mathbf{n}}_{NA}, \qquad \hat{\mathbf{n}}_{NA} = \frac{\mathbf{r}_A - \mathbf{r}_N}{\|\mathbf{r}_A - \mathbf{r}_N\|} \quad \rightsquigarrow \texttt{reorganization.compute\_mode\_modulations} \tag{22}$$

The displacement subtracted is $\mathbf{e}_N$ (the donor), not $\mathbf{e}_A$. The projection axis is $\hat{\mathbf{n}}_{NA}$ (donor-to-acceptor), not $\hat{\mathbf{n}}_{HA}$. Both choices are deliberate; combined, they enforce the two requirements above.

8.2 Limit behaviours

The PT vs. skeletal-mode discriminator (test_sanity_physics.py)

- `test_ha_pure_acceptor_motion_yields_zero`: $\mathbf{e}_N = \mathbf{e}_H = 0$, $\mathbf{e}_A \neq 0$ gives $\Delta r_{HA} = 0$. No PT character.
- `test_ha_pure_h_motion_toward_acceptor_yields_positive`: $\mathbf{e}_N = \mathbf{e}_A = 0$, $\mathbf{e}_H = +d\,\hat{\mathbf{n}}_{NA}$ gives $\Delta r_{HA} = +d$. Genuine PT toward the acceptor.
- `test_ha_pure_n_motion_toward_acceptor_yields_negative`: $\mathbf{e}_H = \mathbf{e}_A = 0$, $\mathbf{e}_N = +d\,\hat{\mathbf{n}}_{NA}$

gives $\Delta r_{HA} = -d$. *Skeletal* mode (the N-H unit translates as a block toward A, the H-A distance shrinks but the proton has not moved relative to its donor).

The PT/skeletal-discriminator property is the principal reason for choosing the reaction-coordinate definition over the naive $(\mathbf{e}_H - \mathbf{e}_A) \cdot \hat{\mathbf{n}}_{HA}$.

8.3 Choice of acceptor: nearest-neighbour heuristic

⇝ `reorganization.build_channels_from_coord_info`

For each protonated His-N-H, the acceptor A is selected as the nearest electronegative atom (N, O, S) within a configurable cutoff `pcet_max_distance_a` (default 3.5 Å) of the H atom, subject to:

- the candidate is not the donor itself,
- the candidate is not bonded (within 1.5 Å) to the donor,
- the candidate has a valid Gaussian centre and PDB residue assignment.

If no valid candidate is found, the H is recorded with $A =$ `None`, $\Delta r_{HA} = 0$ for all modes, and a `UserWarning` is emitted on the first such occurrence in the run.

8.4 Geometric weighting by H···A distance

⇝ `reorganization.build_channels_from_coord_info`

When a protein has multiple plausible H-bond acceptors, the code sums their HA-contributions with a gaussian weight $w_a = \exp(-(d_a - d_0)^2/(2\sigma_d^2))$, $d_a = \|\mathbf{r}_H - \mathbf{r}_{A_a}\|$, $d_0 =$ `pcet_optimal_distance_a` (default 2.8 Å), $\sigma_d =$ `pcet_distance_sigma_a` (default 0.3 Å). This makes the HA channel a weighted aggregate

$$\Delta r_{HA}^{\text{aggregated}} = \sqrt{\sum_a w_a \cdot (\Delta r_{HA,a})^2} \tag{23}$$

which is the RSS aggregation rule of section 12 applied to the HA sub-channels.

Worked numerical example: The PT/skeletal discriminator on three motion patterns

We compute Δr_{HA} for three test eigenvectors on the same His-N-H···A geometry, to demonstrate the PT-vs-skeletal discrimination property.

Setup (geometry):

Atom	$\mathbf{r}$ [Å]
N (donor)	(0.00, 0.00, 0.00)
H (proton)	(0.10, 0.00, 1.00)
A (acceptor)	(0.30, 0.20, 2.80)

Donor-to-acceptor vector: $\mathbf{r}_A - \mathbf{r}_N = (0.30, 0.20, 2.80)$, $\|\mathbf{r}_A - \mathbf{r}_N\| = 2.823$, so $\hat{\mathbf{n}}_{NA} = (0.1063, 0.0709, 0.9920)$ ($\approx$ pointing along $+\hat{\mathbf{z}}$, as expected for an in-line H-bond).

Test eigenvector 1 – pure H motion toward acceptor: $\mathbf{e}_N = (0, 0, 0)$, $\mathbf{e}_H = (0.0106, 0.0071, 0.0992)$

(magnitude 0.10 along $\hat{\mathbf{n}}_{NA}$), $\mathbf{e}_A = (0, 0, 0)$.

$$\Delta r_{HA} = (\mathbf{e}_H - \mathbf{e}_N) \cdot \hat{\mathbf{n}}_{NA} = (0.0106)(0.1063) + (0.0071)(0.0709) + (0.0992)(0.9920)$$

$$= 0.00113 + 0.00050 + 0.09841 = +0.1000.$$

Result: +0.100 Å (full PT signal). Sign positive because H moves toward A.
Test eigenvector 2 – pure N motion toward A (skeletal): $\mathbf{e}_N = (0.0106, 0.0071, 0.0992)$ (same magnitude 0.10 along $\hat{\mathbf{n}}_{NA}$), $\mathbf{e}_H = (0, 0, 0)$, $\mathbf{e}_A = (0, 0, 0)$.
Now the N-H bond *translates as a rigid block* toward A. The H-A distance shrinks (an L1-style "HA modulation" $|\mathbf{e}_H - \mathbf{e}_A| \cdot \hat{\mathbf{n}}_{HA}$ would yield a large positive number), but the proton has not actually moved relative to its donor.

$$\Delta r_{HA} = (\mathbf{e}_H - \mathbf{e}_N) \cdot \hat{\mathbf{n}}_{NA} = (-0.0106)(0.1063) + (-0.0071)(0.0709) + (-0.0992)(0.9920)$$

$$= -0.1000.$$

Result: −0.100 Å (skeletal motion correctly flagged). Sign *negative*, signalling that this is the opposite of the PT direction. A reviewer looking at the sign immediately knows this mode is *not* a PT-promoting mode – the geometric mean $C_{\mathrm{PCET}}(\omega) = \sqrt{M_{\mathrm{HA}} \cdot M_{\mathrm{FeFe}}}$ would not classify this mode as PCET.
Test eigenvector 3 – pure A motion (acceptor wagging): $\mathbf{e}_N = (0, 0, 0)$, $\mathbf{e}_H = (0, 0, 0)$, $\mathbf{e}_A = (0.05, 0, 0)$ (magnitude 0.05 along $\hat{\mathbf{x}}$, perpendicular to the H-bond).

$$\Delta r_{HA} = (\mathbf{e}_H - \mathbf{e}_N) \cdot \hat{\mathbf{n}}_{NA} = (0 - 0)(\hat{\mathbf{n}}_{NA})_x + 0 + 0 = 0.$$

Result: 0 (acceptor wagging correctly ignored). A skeletal mode of the acceptor that doesn't involve the H is not a PT mode at all.
Comparison with the naive formula: The discarded alternative $(\mathbf{e}_H - \mathbf{e}_A) \cdot \hat{\mathbf{n}}_{HA}$ would give:

Test	RC formula (correct)	Naive formula
1. H toward A	+0.100 ✓	+0.099 (close)
2. N+H toward A	−0.100 ✓	+0.099 (*wrong*: skeletal → PT)
3. A wags perp	0.000 ✓	−0.049 (*wrong*: pure-A motion → PT)

The reaction-coordinate formula passes all three; the naive formula fails 2 and 3. This is the discriminator property cited in section 8 – it is what makes the HA channel a meaningful PCET indicator rather than a generic "H-bond wiggle" indicator.
In the code: `reorganization.compute_mode_modulations` computes Δr_{HA} via the channels of type `HA`, using the donor-to-acceptor axis stored as `ChannelGeometry.r_donor` (see section 34).

9 Fe-ligand bond decomposition

9.1 Three orthogonal degrees of freedom

⇝ `core.analyze_fe_ligand`

For an Fe-ligand bond (with the ligand donor atom $L \in \{S, N, O\}$), the relative displacement is

$$\Delta\mathbf{e} \equiv \mathbf{e}_L - \mathbf{e}_{\mathrm{Fe}}. \tag{24}$$

This 3-vector contains all the information about how the bond deforms in the given mode. We decompose it into three physically distinct degrees of freedom:

Stretch along the bond axis.

$$\Delta r_{\mathrm{stretch}} \equiv \Delta\mathbf{e} \cdot \hat{\mathbf{b}}, \qquad \hat{\mathbf{b}} = \frac{\mathbf{r}_L - \mathbf{r}_{\mathrm{Fe}}}{\|\mathbf{r}_L - \mathbf{r}_{\mathrm{Fe}}\|}. \tag{25}$$

This is the signed bond-stretching component (positive = bond lengthening).

Bend in the cluster plane. The bend is, by definition, the component of $\Delta\mathbf{e}$ *perpendicular* to $\hat{\mathbf{b}}$:

$$\Delta\mathbf{e}_\perp \equiv \Delta\mathbf{e} - \Delta r_{\mathrm{stretch}}\,\hat{\mathbf{b}}. \tag{26}$$

This perpendicular vector lives in the 2D plane orthogonal to the bond. We further decompose it relative to the cluster normal $\hat{\mathbf{n}}$:

$$\Delta r_{\mathrm{bend\_oop}} \equiv \Delta\mathbf{e}_\perp \cdot \hat{\mathbf{n}}, \qquad \text{(signed)} \tag{27}$$

$$\Delta r_{\mathrm{bend\_inp}} \equiv \|\Delta\mathbf{e}_\perp - \Delta r_{\mathrm{bend\_oop}}\,\hat{\mathbf{n}}\|. \tag{28}$$

The first projection is signed; the second is a length, so it is non-negative. The Pythagorean identity

$$\|\Delta\mathbf{e}_\perp\|^2 = (\Delta r_{\mathrm{bend\_oop}})^2 + (\Delta r_{\mathrm{bend\_inp}})^2. \tag{29}$$

follows immediately from the orthogonality of the cluster-normal to the in-plane complement.

Why $\hat{\mathbf{n}}$ as the bend-OOP axis. The cluster normal $\hat{\mathbf{n}}$ from section 6 is the natural OOP axis for a [2Fe-2S] cluster: it points perpendicular to the rhombic Fe_2S_2 plane and is the same axis used by the cluster-core OOP/INP classification (section 5). This choice makes the Fe-ligand bend_oop / bend_inp directly comparable to the cluster-core OOP/INP fractions: a mode that is cluster-OOP-dominant will tend to have Fe-ligand bend_oop also dominant.

⇝ `core.analyze_fe_ligand`

9.2 Percentages of the total bend

When comparing modes with very different total amplitudes, the bend components are converted to percentage shares of the total bend power:

$$\mathrm{bend\_inp\_pct} = 100 \cdot \frac{(\Delta r_{\mathrm{bend\_inp}})^2}{(\Delta r_{\mathrm{bend\_inp}})^2 + (\Delta r_{\mathrm{bend\_oop}})^2}, \tag{30}$$

and analogously for `bend_oop_pct`. By equation (29) these sum to 100% exactly.

For modes with very small total bend amplitude (below the noise threshold $2\sigma_e^{\mathrm{therm}}\sqrt{2}$; part VII) the percentage is set to NaN, so numerical noise does not get amplified into spurious classifications.

9.3 Sigma propagation

Each of $\Delta r_{\text{stretch}}, \Delta r_{\text{bend\_inp}}, \Delta r_{\text{bend\_oop}}$ is a difference of two independent Gaussian-noise components, so each carries the same

$$\sigma \;=\; \sqrt{2}\,\sigma_e \cdot u_{\text{rms}}(\omega_l) \tag{31}$$

where $\sigma_e =$ `sigma_eigvec` is the per-component eigenvector noise (default 5×10^{-4}). The $\sqrt{2}$ comes from variance addition for a difference; the u_{rms} factor is the thermal-scaling convention (section 4). These σ values are recorded in the columns `s_stretch`, `s_bend_inp`, `s_bend_oop` of the `Fe_S_Cys.xlsx` / `Fe_N_His.xlsx` workbooks.

9.4 Physical interpretation for PCET studies

The Fe-N(His) bend decomposition is particularly informative:

- *bend_inp dominant* ($> 70\%$): the His ring tilts in the cluster plane. This modulates the Fe-Fe distance and the Fe-S core, which is the classical electron-transfer- active geometry.
- *bend_oop dominant* ($> 70\%$): the His ring tilts out of plane. This modulates the orientation of the N-H bond and therefore the H$\cdots$A donor-acceptor axis, which is the proton-transfer-coupling channel.
- *mixed* (30–70%): the mode contributes simultaneously to ET and PT pathways – the canonical PCET signature.

The Fe-S(Cys) bend decomposition follows the same logic, with the caveat that the S-C bond to the protein backbone is more flexible than the N-C bond, so the bend_oop component of Fe-S(Cys) often carries longer-range allosteric coupling than the in-plane component.

10 His-NH stretching

10.1 Definition

⇝ `core.analyze_his_hn`

For each protonated histidine, the N-H bond modulation is computed with the classical formula equation (21), with $a = \text{N}_{\text{His}}$, $b = \text{H}$:

$$\Delta r_{\text{NH}} \;=\; (\mathbf{e}_N - \mathbf{e}_H) \cdot \hat{\mathbf{n}}_{NH}, \qquad \hat{\mathbf{n}}_{NH} \;=\; \frac{\mathbf{r}_N - \mathbf{r}_H}{\|\mathbf{r}_N - \mathbf{r}_H\|}. \tag{32}$$

⇝ `core.analyze_his_hn`

10.2 Hydrogen inclusion

The N-H stretching analysis requires the hydrogen eigenvector components, which the default heavy-atom-only filter (section 3.2) discards. The analysis is therefore gated by `include_hn_vibration`:

- `include_hn_vibration` = true: a hydrogen-inclusive eigenvector loader is used; Δr_{NH} is computed for every protonated His.

- `include_hn_vibration` = false (default): the His-NH stretch sheet is still produced but all Δr_{NH} values are zero, and the sheet's existence serves as a placeholder for downstream tools.

10.3 Sigma propagation – the v1.0.4 fix

⇝ `core.analyze_his_hn`

Prior to v1.0.4, the sigma for Δr_{NH} used the raw $\sqrt{2}\,\sigma_e$ instead of the thermally scaled $\sqrt{2}\,\sigma_e \cdot u_{\mathrm{rms}}$. For typical low-frequency modes this made the recorded `s_hn_stretch` a factor of $1/u_{\mathrm{rms}} \approx 1/0.05 = 20\times$ *too large*, which forced many genuine PCET signals below the visual significance threshold. The v1.0.4 fix (FUND 6, section 69) applies the thermal scaling $\sigma_e^{\mathrm{therm}} = \sigma_e \cdot u_{\mathrm{rms}}(\omega_l)$, the convention discussed in detail in part VII; the recorded sigmas are now ~ 20 times smaller than in v1.0.2/v1.0.3, but the underlying values Δr_{NH} are unchanged.

11 Pair-wise reorganization $\lambda_X^{\mathbf{pair}}$

11.1 From harmonic oscillator energy to cm^{-1}

⇝ `reorganization.lambda_pair_cm1`

For a single bond with effective reduced mass μ and a single normal mode of frequency ω, the harmonic-oscillator energy contribution of a bond-length displacement Δr is

$$E \;=\; \tfrac{1}{2}\,\mu\,\omega^2\,(\Delta r)^2 \quad [\text{Joule}]. \tag{33}$$

In Marcus-Hush theory, this expression – evaluated at the displacement that takes the system from reactant to product geometry – is half the reorganization energy λ along that mode. For *spectroscopic* bond reorganization in a frozen-environment setting, we instead take Δr to be the thermally averaged amplitude of the mode along the bond, $\Delta r = u_{\mathrm{rms}}(\omega) \cdot \alpha_X$ where α_X is the dimensionless projection of the mode onto the bond unit-vector. Then equation (33) reads

$$\lambda_X^{\mathrm{pair}} \;=\; \tfrac{1}{2}\,\mu\,\omega^2\,(\Delta r_X)^2, \tag{34}$$

which is the per-mode bond-reorganization energy along channel X.

To convert to spectroscopic cm^{-1} units, divide by $hc = 1.986\,445\,857 \times 10^{-23}$ J cm:

$$\boxed{\lambda_X^{\mathrm{pair}}[\mathrm{cm}^{-1}] \;=\; \frac{1}{hc}\,\tfrac{1}{2}\,\mu\,\omega^2\,(\Delta r_X)^2} \quad \text{⇝ reorganization.lambda\_pair\_cm1} \tag{35}$$

11.2 Unit conversions used in the implementation

The function `reorganization.lambda_pair_cm1(dr_a, omega_cm1, mu_amu)` accepts SI-mixed inputs:

- Δr in Å; internal conversion $\Delta r \cdot 10^{-10}$ to metres,
- ω in cm^{-1}; internal conversion $\omega_{\mathrm{SI}} = 2\pi c \cdot \tilde{\nu}$ with $c = 2.997\,924\,58 \times 10^{10}$ cm/s,
- μ in amu; internal conversion $\mu_{\mathrm{SI}} = \mu_{\mathrm{amu}} \cdot 1.660\,539\,066\,60 \times 10^{-27}$ kg.

The output is λ in cm^{-1}.

11.3 Sanity checks

Quadratic and scaling properties (test_sanity_physics.py)

- `test_lambda_zero_for_zero_displacement`: $\Delta r = 0$ yields $\lambda = 0$ exactly.
- `test_lambda_positive_for_any_nonzero_displacement`: λ is non-negative and depends only on $|\Delta r|$ (sign-symmetric).
- `test_lambda_quadratic_in_displacement`: doubling Δr quadruples λ.
- `test_lambda_quadratic_in_frequency`: doubling ω quadruples λ.

11.4 $T \to 0$ limit for pure stretching modes

For a pure-stretching mode along bond X with $\alpha_X = 1$, in the $T \to 0$ limit:

$$\Delta r_X \;=\; u_{\text{rms}} \cdot 1 \;\xrightarrow{T\to 0}\; \sqrt{\frac{\hbar}{2\mu\omega}}, \tag{36}$$

so

$$\lambda_X^{\text{pair}} \;=\; \tfrac{1}{2}\mu\omega^2 \cdot \frac{\hbar}{2\mu\omega} \;=\; \frac{\hbar\omega}{4}. \tag{37}$$

That is, half the zero-point energy. In cm^{-1} that is just $\omega/4$ – the formula is dimensionally exact.

Sanity check (test_sanity_physics.py)

`test_lambda_zero_T_pure_stretching`: with $\omega = 320$ cm^{-1}, $\mu = 31.97$ amu, $T = 10^{-4}$ K, $\alpha_X = 1$, the result matches $\omega/4 = 80.0$ cm^{-1} to $< 10^{-3}$ relative error.

11.5 Relation to classical Marcus reorganization

The spectroscopic bond reorganization λ_X^{pair} in equation (35) should not be conflated with the classical Marcus solvent reorganization energy, even though they share the $\frac{1}{2}\mu\omega^2(\Delta r)^2$ structure. The classical Marcus quantity uses Δr = static geometry shift between reactant and product (a Born-Oppenheimer property), and the relevant degrees of freedom are usually intermolecular solvent modes. The spectroscopic quantity here uses Δr = thermal/zero-point displacement amplitude of an internal vibrational mode, and is a *frequency-resolved* measurement of which bond modes "zappel" the most. They become equal only in the special case of a single classical promoting mode driving Born-Oppenheimer hopping; in general the spectroscopic quantity is the more directly NRVS-observable.

Worked numerical example: λ^{pair} for the 320 cm^{-1} Fe-S stretch

We compute $\lambda_{\text{FeS}}^{\text{pair}}$ for the bond and mode from the Fe-S stretch example above:

- $\Delta r = 0.0217$ Å $= 2.17 \times 10^{-12}$ m,
- $\tilde{\nu} = 320$ cm^{-1} $\to \omega = 6.026 \times 10^{13}$ rad/s (computed above),
- $\mu = 31.97$ amu $= 5.309 \times 10^{-26}$ kg (using the reduced-mass-of-the-mode convention; in a more refined treatment one would use the bond-only reduced mass $\mu_{\text{Fe-S}} = m_{\text{Fe}} m_{\text{S}}/(m_{\text{Fe}} + m_{\text{S}}) = 20.34$ amu, but for the per-mode formula equation (35) the Gaussian-reported reduced mass is

used).

Step 1 – E in Joule:

$$\begin{aligned} E &= \tfrac{1}{2}\,\mu\,\omega^2\,(\Delta r)^2 \\ &= \tfrac{1}{2}\cdot 5.309\times10^{-26}\,\mathrm{kg}\cdot(6.026\times10^{13}\,\mathrm{rad/s})^2\cdot(2.17\times10^{-12}\,\mathrm{m})^2 \\ &= \tfrac{1}{2}\cdot 5.309\times10^{-26}\cdot 3.631\times10^{27}\cdot 4.709\times10^{-24}\,\mathrm{J} \\ &= 4.539\times10^{-22}\,\mathrm{J}. \end{aligned}$$

Step 2 – convert to cm^{-1}:

$$\lambda^{\mathrm{pair}} = E/(hc) = \frac{4.539\times10^{-22}}{1.986\times10^{-23}} = \boxed{22.85\ \mathrm{cm}^{-1}}.$$

Cross-check via the $T\to 0$ formula: For a pure stretch mode with $\alpha_X = 1$ (the displacement is entirely along the bond), equation (37) gives $\lambda^{\mathrm{pair}} = \omega/4 = 320/4 = 80$ cm^{-1}. Our 22.85 cm^{-1} is well below this – consistent with $\alpha_X \approx 0.534$ (from the previous worked example), giving $\lambda^{\mathrm{pair}}_{\mathrm{predicted}} \approx 80\cdot 0.534^2 = 22.8$ cm^{-1}. The two routes agree within rounding.
In the code: `reorganization.lambda_pair_cm1(0.0217, 320.0, 31.97)` returns `22.85`.

12 RSS aggregation across sub-channels

12.1 Why RSS, not L1

↝ `reorganization.aggregate_by_parent`

A parent channel $X \in \{\mathrm{FeFe}, \mathrm{FeN}, \mathrm{FeS}, \mathrm{NH}, \mathrm{HA}\}$ generally has multiple *sub-channels* that aggregate to it:

- FeN: two Fe-N(His) sub-bonds (one per His residue) for a Cys_2His_2 Rieske cluster; zero for a pure Cys_4 cluster.
- FeS: four Fe-S sub-bonds for a Cys_4 cluster (two terminal Fe-S_{Cys} per Fe), or two for a Cys_2His_2 cluster, plus the two bridging Fe-S_{μ_2}.
- NH: one N-H sub-bond per protonated His.
- HA: one H-A sub-bond per protonated His paired with a unique acceptor candidate (one His can have multiple).

For mode i, we have sub-channel modulations $\{\Delta r_X^{(s)}(i)\}_{s\in\text{sub-channels}(X)}$ with weights $\{w_s\}$. We need an aggregation rule $\Delta r_X^{\mathrm{aggregated}}(i) = f(\{\Delta r_X^{(s)}(i)\}, \{w_s\})$ that produces the parent-channel modulation. Two natural candidates are:

1. *Signed L1 sum*: $f = \sum_s w_s \cdot \Delta r_X^{(s)}$. Problem: if two sub-bonds modulate in opposite directions (one stretches, the other contracts – a typical antisymmetric mode), they cancel, and the parent appears stationary. This is the wrong physics: an antisymmetric stretch is still a modulation of FeS in the sense of total bond-energy change.
2. *Absolute L1 sum*: $f = \sum_s w_s \cdot |\Delta r_X^{(s)}|$. Problem: not energetically consistent. A pure antisymmetric stretch with $|\Delta r| = 0.05$ Å on two equal bonds gives $f = 0.10$ Å, but the energy contribution is

$2 \cdot \frac{1}{2}\mu\omega^2(0.05)^2 \neq \frac{1}{2}\mu\omega^2(0.10)^2$.

The energetically consistent choice is the *root-sum-square* (RSS):

$$\Delta r_X^{\mathrm{RSS}}(i) \equiv \sqrt{\sum_{s \in \text{sub-channels}(X)} w_s \, (\Delta r_X^{(s)}(i))^2} \qquad (38)$$

↝ `reorganization.aggregate_by_parent`

12.2 Energetic consistency

↝ `reorganization.aggregate_by_parent`

The RSS aggregation has the property that the parent-channel λ – defined as the weighted sum of sub-channel λ's – can be written in the same harmonic-oscillator form as a sub-channel λ:

$$\lambda_X^{\mathrm{parent}}(i) = \sum_s w_s \cdot \tfrac{1}{2}\mu_i\omega_i^2(\Delta r_X^{(s)}(i))^2 \qquad (39)$$

$$= \tfrac{1}{2}\mu_i\omega_i^2 \cdot \underbrace{\sum_s w_s(\Delta r_X^{(s)}(i))^2}_{=(\Delta r_X^{\mathrm{RSS}})^2} \qquad (40)$$

$$= \tfrac{1}{2}\mu_i\omega_i^2(\Delta r_X^{\mathrm{RSS}}(i))^2. \qquad (41)$$

The choice of RSS makes equation (35) a valid parent-level formula with Δr replaced by $\Delta r_X^{\mathrm{RSS}}$. The L1 alternative breaks this consistency.

Sanity check (test_sanity_physics.py)

`test_rss_aggregation_two_subchannels_consistency_with_lambda`: with two sub-channels at $\Delta r_1 = 0.008$, $\Delta r_2 = -0.012$ Å, $w_1 = w_2 = 1$, $\omega = 350$ cm^{-1}, $\mu = 31.97$ amu, the two computations $\lambda_{\mathrm{sum}} = \lambda_{\mathrm{pair}}(\Delta r_1) + \lambda_{\mathrm{pair}}(\Delta r_2)$ and $\lambda_{\text{via-RSS}} = \lambda_{\mathrm{pair}}(\Delta r^{\mathrm{RSS}})$ match to $< 10^{-10}$ relative error.

12.3 Weights per channel

↝ `reorganization.build_channels_from_coord_info`

- FeFe, FeN, FeS, NH: weights $w_s = 1$ for all sub-channels (each Fe-N or Fe-S bond contributes equally to the parent channel's energetic budget).
- HA: weights are the gaussian H$\cdots$A-distance weights of section 8, $w_a = \exp(-(d_a - d_0)^2/(2\sigma_d^2))$. The $\sqrt{w_a}$ factor in equation (38) downweights distant or weakly H-bonded acceptors.

12.4 Single sub-channel limit

For a parent channel with a single sub-channel ($|s| = 1$), equation (38) reduces to

$$\Delta r_X^{\mathrm{RSS}} = \sqrt{w \cdot (\Delta r_X)^2} = \sqrt{w}\,|\Delta r_X|. \qquad (42)$$

For unit weight ($w = 1$, the FeFe and NH cases), $\Delta r_X^{\mathrm{RSS}} = |\Delta r_X|$ – the absolute bond modulation, lossless against the sign-cancellation of an L1 sum.

Sanity check (test_sanity_physics.py)

`test_rss_aggregation_single_subchannel`: with $|\Delta r| = 0.012$ Å, $w = 1$, the RSS aggregate is exactly 0.012 Å.

Worked numerical example: RSS aggregation: FeS parent over 4 sub-bonds

A Cys_4 cluster has 4 Fe-S_{Cys} terminal bonds plus 2 bridging Fe-S_{μ_2} bonds = 6 sub-channels feeding the FeS parent (or 4 + 4 = 8 depending on whether one counts bridging-S as "one bond" or "two"; the code counts each Fe-S as one bond). For this worked example we take a typical antisymmetric Fe-S stretch mode with the per-sub-channel modulations

Sub-channel	Δr_s [Å]	w_s
FeS_Fe1_S1_term	+0.0150	1
FeS_Fe1_S2_term	−0.0145	1
FeS_Fe2_S3_term	−0.0148	1
FeS_Fe2_S4_term	+0.0152	1

Note the alternating signs – this is a typical antisymmetric stretch: Fe_1-S_1 and Fe_2-S_4 stretch while Fe_1-S_2 and Fe_2-S_3 contract.

Step 1 – Signed L1 sum (the discarded alternative):

$$\Sigma\Delta r = 0.0150 - 0.0145 - 0.0148 + 0.0152 = +0.0009 \text{ Å},$$

which is *essentially zero* – the antisymmetric character of the mode caused cancellation. This is exactly the failure mode that disqualifies the signed L1 sum.

Step 2 – L1 sum of absolutes (the second discarded alternative):

$$\Sigma|\Delta r| = 0.0150 + 0.0145 + 0.0148 + 0.0152 = 0.0595 \text{ Å}.$$

This survives the antisymmetry, but is energetically wrong because $\frac{1}{2}\mu\omega^2(0.0595)^2 = \frac{1}{2}\mu\omega^2(3.540 \times 10^{-3}$ Å$^2)$, whereas the true energy is the sum of per-sub-channel energies $\sum_s \frac{1}{2}\mu\omega^2\Delta r_s^2$.

Step 3 – RSS aggregation (the correct formula):

$$\Delta r_{\text{FeS}}^{\text{RSS}} = \sqrt{\sum_s w_s(\Delta r_s)^2} = \sqrt{0.0150^2 + 0.0145^2 + 0.0148^2 + 0.0152^2}$$

$$= \sqrt{2.250 + 2.103 + 2.190 + 2.310} \cdot 10^{-4} = \sqrt{8.853 \times 10^{-4}} = \boxed{0.02975 \text{ Å}}.$$

Step 4 – Verify energetic consistency: The per-mode parent reorganization should equal the sum of per-sub-channel reorganizations:

$$\lambda_{\text{FeS}}(\text{via RSS}) = \tfrac{1}{2}\mu\omega^2(0.02975)^2/(hc)$$

should equal $\sum_s \lambda_{\text{pair}}^{(s)}$, where each $\lambda_{\text{pair}}^{(s)} = \frac{1}{2}\mu\omega^2(\Delta r_s)^2/(hc)$.

Using the conversion factor from the λ^{pair} example ($\Delta r = 0.0217$ Å gave $\lambda = 22.85$ cm^{-1}, so the proportionality is $\lambda^{\text{pair}}/(\Delta r)^2 = 22.85/(0.0217)^2 = 4.852 \times 10^4$ cm^{-1}/Å^2):

$$\lambda_{\text{FeS}}(\text{via RSS}) = 4.852 \times 10^4 \cdot 0.02975^2 = 42.96 \text{ cm}^{-1}.$$

And from the sum:

$$\sum_s \lambda_{\text{pair}}^{(s)} = 4.852 \times 10^4 \cdot (2.250 + 2.103 + 2.190 + 2.310) \cdot 10^{-4} = 4.852 \cdot 8.853 = 42.96 \text{ cm}^{-1}.$$

The two values match to all digits. *This is the consistency property* that the RSS aggregation guarantees and that the L1 alternatives violate.

Sigma propagation: With $\sigma_e^{\text{therm}} = 5 \times 10^{-4} \cdot 0.0407 = 2.04 \times 10^{-5}$ Å, and using equation (90),

$$\sigma_{\text{RSS}}^{\text{FeS}} = \sqrt{2} \cdot 2.04 \times 10^{-5} \cdot \frac{\sqrt{(0.0150)^2 + (0.0145)^2 + (0.0148)^2 + (0.0152)^2}}{0.02975}$$

$$= 2.88 \times 10^{-5} \cdot \frac{0.02975}{0.02975} = 2.88 \times 10^{-5} \text{ Å}.$$

The square-root factor reduces to 1 for equal-magnitude sub-channels. Significance: $0.02975/2.88{\times}10^{-5} \approx 1030\sigma$ – deep above the noise.

In the code: `reorganization.aggregate_by_parent` loops over sub-channel `ChannelResults` and returns the parent `dr_rss_FeS` = 0.02975.

13 Per-mode and total reorganization

13.1 Per-mode parent-channel $\lambda_X(i)$

⇝ `reorganization.compute_mode_modulations`

For mode i and parent channel X, the per-mode reorganization energy is the harmonic-oscillator energy of the RSS-aggregated bond modulation:

$$\lambda_X(i) \equiv \tfrac{1}{2}\,\mu_i\,\omega_i^2\,(\Delta r_X^{\text{RSS}}(i))^2 = \sum_{s \in \text{sub-channels}(X)} w_s \cdot \lambda_X^{(s)}(i), \quad \rightsquigarrow \texttt{reorganization.compute\_mode\_modulations} \tag{43}$$

where the equality with the weighted sub-channel sum follows from equation (41).

13.2 System-wide total Λ_X^{total}

⇝ `reorganization.compute_total_reorganization`

The total reorganization energy along channel X is the sum of the per-mode contributions over all modes in the frequency window:

$$\boxed{\Lambda_X^{\text{total}} \equiv \sum_{i=1}^{N_{\text{modes}}} \lambda_X(i)} \quad \rightsquigarrow \texttt{reorganization.compute\_total\_reorganization} \tag{44}$$

This is the headline quantity used for system comparisons ("does this μS-protonated state have a larger Λ_{FeS} than the corresponding deprotonated state?", or "does the Cys_2His_2 Rieske-type cluster have larger Λ_{FeFe} than the Cys_4 ferredoxin-type?"). It is computed once per cluster, per frequency window, and recorded in the `*_analysis_main.xlsx` workbook on the `Reorganisation` sheet.

13.3 Cumulative-Lambda curve $\Lambda_X(\omega_{\text{cut}})$

⇝ `reorganization.compute_cumulative_reorganization`

The cumulative curve is the partial sum truncated at ω_{cut}:

$$\Lambda_X(\omega_{\text{cut}}) \equiv \sum_{\substack{i \\ \omega_i \le \omega_{\text{cut}}}} \lambda_X(i). \tag{45}$$

This is a monotonically non-decreasing step function that climbs to Λ_X^{total} as $\omega_{\text{cut}} \to \infty$. The locations of large jumps in $\Lambda_X(\omega)$ identify the spectral regions where bond channel X accumulates most of its reorganization energy – directly comparable with NRVS peak positions.

The cumulative curve is computed on a uniform frequency grid (default $\Delta\omega = 0.05$ cm^{-1}, set by `interp_step`) and written to the `Cumulative` sheet of the `*_analysis_interp##.xlsx` workbook.

14 Modulation spectra $M_X(\omega)$ and co-modulation

14.1 Single-channel $M_X(\omega)$

⇝ `reorganization.compute_modulation_spectra`

The modulation spectrum along channel X is a frequency-resolved amplitude landscape obtained by gaussian-broadening every mode's Δr_X^{RSS} at its frequency:

$$\boxed{M_X(\omega) \equiv \sum_{i=1}^{N_{\text{modes}}} \Delta r_X^{\text{RSS}}(i)\, \mathcal{G}(\omega - \omega_i,\, \sigma_M)} \quad \rightsquigarrow \texttt{reorganization.compute\_modulation\_spectra} \tag{46}$$

with the normalised gaussian

$$\mathcal{G}(x, \sigma) \equiv \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{x^2}{2\sigma^2}\right). \tag{47}$$

The broadening width $\sigma_M =$ `reorg_spectrum_sigma_cm1` (default 5 cm^{-1}) is chosen empirically to match the typical DFT-vs-experiment frequency mismatch. $M_X(\omega)$ has units of Å (the broadening preserves the dimensional content of Δr_X^{RSS}); it is recorded on the `ModulationsSpektrum` sheet.

14.2 Co-modulation as geometric mean

⇝ `reorganization.compute_co_modulation_spectra`

For PCET analysis we want a frequency-resolved descriptor that peaks where *two* channels are simultaneously modulating. The geometric mean of two M spectra is the natural choice:

$$C_{\text{PCET}}(\omega) = \sqrt{M_{HA}(\omega)\, M_{\text{FeFe}}(\omega)}, \tag{48}$$

$$C_{\text{PT-FeN}}(\omega) = \sqrt{M_{HA}(\omega)\, M_{\text{FeN}}(\omega)}, \tag{49}$$

$$C_{\text{ET-FeS}}(\omega) = \sqrt{M_{\text{FeFe}}(\omega)\, M_{\text{FeS}}(\omega)}. \quad \rightsquigarrow \texttt{reorganization.compute\_co\_modulation\_spectra} \tag{50}$$

Geometric-mean co-modulation has two attractive properties:

- It is zero if either factor is zero (a mode active in only one channel does not contaminate the

co-modulation).

- It is symmetric in the two channels (no preferred ordering).
- Its units are Å (the same as the underlying M spectra).

The physical interpretation: $C_{\mathrm{PCET}}(\omega)$ peaks at frequencies where modes simultaneously modulate the proton-transfer reaction coordinate *and* the Fe-Fe distance (i.e. the electron- transfer reaction coordinate). These are the modes most likely to serve as PCET promoting modes.

14.3 Choice of broadening width

The default $\sigma_M = 5$ cm^{-1} is small enough to preserve narrow features (peak resolution $\sim 2\sigma = 10$ cm^{-1}) and large enough to bridge the typical DFT-vs-experimental frequency mismatch (5–15 cm^{-1} for harmonic DFT, 1–3 cm^{-1} for scaled DFT). Smaller σ_M produces sharper peaks but is more sensitive to vibrational anharmonicity that DFT misses; larger σ_M smooths over the band-by-band identification that is the principal goal of $M_X(\omega)$. The user can override via `reorg_spectrum_sigma_cm1`.

Worked numerical example: $M_X(\omega)$ evaluated at three frequencies

We compute $M_{\mathrm{FeS}}(\omega)$ on a uniform 0.5-cm^{-1} grid using three mock modes:

Mode #	ω_i [cm^{-1}]	$\Delta r_{\mathrm{FeS}}^{\mathrm{RSS}}(i)$ [Å]
1	280	0.012
2	320	0.030
3	365	0.022

with default broadening $\sigma_M = 5$ cm^{-1}.
Step 1 – Gaussian normalisation: $1/(\sigma_M\sqrt{2\pi}) = 1/(5 \cdot 2.507) = 0.0798$ cm.
Step 2 – evaluate at $\omega = 320$ cm^{-1} (peak of mode 2):

$$\mathcal{G}(320-280,5) = 0.0798 \cdot \exp(-(40)^2/(2\cdot 25)) = 0.0798 \cdot e^{-32} \approx 1.13 \times 10^{-15},$$
$$\mathcal{G}(320-320,5) = 0.0798 \cdot \exp(0) = 0.0798,$$
$$\mathcal{G}(320-365,5) = 0.0798 \cdot \exp(-(45)^2/50) = 0.0798 \cdot e^{-40.5} \approx 2.0 \times 10^{-19}.$$

$$M_{\mathrm{FeS}}(320) = 0.012 \cdot 1.13 \times 10^{-15} + 0.030 \cdot 0.0798 + 0.022 \cdot 2.0 \times 10^{-19}$$
$$\approx 0.030 \cdot 0.0798 = \boxed{2.39 \times 10^{-3}\ \text{Å} \cdot \text{cm}}.$$

Note units: the Gaussian has dimensions $1/\text{cm}^{-1} = \text{cm}$, so M_X inherits units of Å·cm. The Excel column is labelled simply with the units Å (the cm factor is the inverse of the frequency resolution and is conventional).
Step 3 – evaluate at $\omega = 300$ cm^{-1} (between modes 1 and 2):

$$\mathcal{G}(300-280,5) = 0.0798 \cdot e^{-(20)^2/50} = 0.0798 \cdot e^{-8} = 2.68 \times 10^{-5},$$
$$\mathcal{G}(300-320,5) = 0.0798 \cdot e^{-8} = 2.68 \times 10^{-5}.$$

$$M_{\mathrm{FeS}}(300) = 0.012 \cdot 2.68 \times 10^{-5} + 0.030 \cdot 2.68 \times 10^{-5} + 0 \approx 1.13 \times 10^{-6}\ \text{Å} \cdot \text{cm}.$$

Three orders of magnitude smaller than the peak at $\omega = 320$ – the Gaussian falls off rapidly outside

$\pm 2\sigma$.
Step 4 – evaluate at $\omega = 280$ cm^{-1} (peak of mode 1):

$$M_{\mathrm{FeS}}(280) = 0.012 \cdot 0.0798 = 9.6 \times 10^{-4}\ \text{Å} \cdot \text{cm}.$$

Smaller than the peak at $\omega = 320$ because mode 1 has smaller Δr.
Cumulative $\Lambda_{\mathrm{FeS}}(\omega)$ at the same three points: $\lambda(i) = 4.852 \times 10^4 \cdot (\Delta r_i)^2$ (using the proportionality from the λ^{pair} example): $\lambda(1) = 6.99$, $\lambda(2) = 43.7$, $\lambda(3) = 23.5$ cm^{-1}.

$$\Lambda(280) = \lambda(1) = 6.99, \quad \Lambda(320) = 6.99 + 43.7 = 50.69, \quad \Lambda(365) = 50.69 + 23.5 = 74.19\ \text{cm}^{-1}.$$

The cumulative curve is a step function that increases at each mode frequency. The total $\Lambda_{\mathrm{FeS}}^{\mathrm{total}} = \Lambda(365) = 74.2$ cm^{-1}.
In the code: `reorganization.compute_modulation_spectra(per_mode_results, parent="FeS", omega_grid, sigma_cm1=5)` returns the array $M_{\mathrm{FeS}}(\omega)$ on the grid.

15 SCSD projection

15.1 The $D_{2\mathrm{h}}$ symmetry group

A planar four-atom rhombic Fe_2S_2 core with the convention $\mathbf{r}_{\mathrm{Fe}_1} = (+a, 0, 0)$, $\mathbf{r}_{\mathrm{Fe}_2} = (-a, 0, 0)$, $\mathbf{r}_{\mathrm{S}_1} = (0, +b, 0)$, $\mathbf{r}_{\mathrm{S}_2} = (0, -b, 0)$ has the symmetry group $D_{2\mathrm{h}}$, which has 8 irreducible representations:

$$\mathrm{D}_{2\mathrm{h}} \;=\; \{E, C_2(\hat{\mathbf{z}}), C_2(\hat{\mathbf{y}}), C_2(\hat{\mathbf{x}}), i, \sigma(xy), \sigma(xz), \sigma(yz)\} \tag{51}$$

with the irreps $A_g, B_{1g}, B_{2g}, B_{3g}, A_u, B_{1u}, B_{2u}, B_{3u}$, where the g/u subscript records parity under inversion i and the $B_{kg/u}$ subscript records symmetry under the C_2 axes.

For a 4-atom cluster the displacement space is 12-dimensional, and its decomposition into $D_{2\mathrm{h}}$ irreps is $3A_g + B_{1g} + 2B_{2g} + 2B_{3g} + A_u + 2B_{1u} + B_{2u} + B_{3u}$, totalling 13 dimensions – one more than 12, which means one linear combination is over-counted; this combination is exactly the zero mode (uniform translation along $\hat{\mathbf{z}}$) plus the rigid-rotation pseudomodes that span 6 of the 12 dimensions; the remaining 6 dimensions are the genuine in-cluster vibrational modes.

Kingsbury & Senge [23] have published a robust projection procedure that handles the over-counting cleanly by projecting onto the 8 irrep-pure subspaces directly and reporting the projection magnitudes:

$$\boxed{\mathbf{a}_l \;=\; \mathbf{P}_{\mathrm{D}_{2\mathrm{h}}}\big(\mathbf{e}_l^{\mathrm{cluster}} \cdot u_{\mathrm{rms}}\big), \qquad \mathbf{a}_l \in \mathbb{R}^8} \quad \rightsquigarrow \texttt{core.compute\_scsd\_for\_mode\_full} \tag{52}$$

where $\mathbf{P}_{\mathrm{D}_{2\mathrm{h}}}$ is the 8×12 projection-matrix and $\mathbf{e}_l^{\mathrm{cluster}}$ denotes the 12-component cluster-core sub-vector of the mode-l eigenvector, listed in canonical order $[\mathrm{Fe}_1, \mathrm{Fe}_2, \mathrm{S}_1, \mathrm{S}_2]$.

15.2 Reference-vs-distorted difference method

⇝ core.compute_scsd_for_mode_full

The projection matrix is constructed for an idealised *model geometry* (`core._SCSD_MODEL_COORDS`) with hard-coded Fe-Fe = 2.73 Å and Fe-S = 2.20 Å. The actual cluster geometry differs from this model. To

avoid carrying the geometry difference into the SCSD result, the code uses a *difference method*:

1. Compute the SCSD projection of the model geometry, $\mathbf{a}_l^{\mathrm{ref}}$.
2. Compute the SCSD projection of the actual (Kabsch-aligned) cluster geometry, $\mathbf{a}_l^{\mathrm{dist}}$.
3. Report the difference, $\mathbf{a}_l^{\mathrm{output}} = \mathbf{a}_l^{\mathrm{dist}} - \mathbf{a}_l^{\mathrm{ref}}$.

The fixed-geometry component cancels by linearity of the projection. The reported amplitudes are the *distortion* components, which are invariant under the choice of reference (any other reference geometry with the same symmetry would give the same answer).

15.3 Sigma propagation – the v1.0.4 fix

⇝ `core.compute_scsd_for_mode_full`

Before v1.0.4, the SCSD sigmas used hard-coded constants (5×10^{-7} for the reference, 5×10^{-6} for the distortion). The v1.0.4 fix (FUND 8, section 69) replaces these with formulas built from the user-configurable $\sigma_e =$ `sigma_eigvec` and $\sigma_c =$ `sigma_coord`:

$$\sigma^{(\Gamma)}_{\mathrm{SCSD,ref}} = \sqrt{2}\,\sigma_c, \tag{53}$$

$$\sigma^{(\Gamma)}_{\mathrm{SCSD,dist}} = \sqrt{2}\,\sqrt{\sigma_c^2 + (\sigma_e \cdot u_{\mathrm{rms}}(\omega_l))^2}, \tag{54}$$

$$\sigma^{(\Gamma)}_{\mathrm{SCSD,diff}} = \sqrt{(\sigma^{(\Gamma)}_{\mathrm{SCSD,ref}})^2 + (\sigma^{(\Gamma)}_{\mathrm{SCSD,dist}})^2}. \tag{55}$$

With the default $\sigma_c = 10^{-3}$ Å and $\sigma_e = 5 \times 10^{-4}$, the new sigmas are about three orders of magnitude larger than the hard-coded ones, which were physically implausibly small. The underlying SCSD amplitudes are unchanged.

16 Kernel classification

16.1 Goal

⇝ `core._oop_ring_metrics, core.classify_oop_inp`

While SCSD gives a rigorous $D_{2\mathrm{h}}$-irrep decomposition, many users want a more chemist-friendly summary in terms of the classical cluster-core normal-mode names: “Fe-S symmetric stretch”, “Fe-Fe antisymmetric stretch”, “S-S breathing”, “cluster umbrella”, etc. The kernel-classification heuristic computes a score for each of 10 named mode types and reports `kern_primary` (the highest-scoring) and `kern_secondary` (the second-highest).

16.2 The 10 named mode types

Name	Symmetry	Description
`Fe_Fe_sym_stretch`	A_g	Both Fe atoms move symmetrically along $\hat{\mathbf{x}}$ (out from / into the cluster centre).
`Fe_Fe_antisym_stretch`	B_{1g}	Fe_1 along $+\hat{\mathbf{x}}$, Fe_2 along $-\hat{\mathbf{x}}$ (out-of-phase).
`S_S_sym_stretch`	A_g	S atoms move along $\hat{\mathbf{y}}$ symmetrically.

`S_S_antisym_stretch`	B_{2g}	S atoms move along $\hat{\mathbf{y}}$ out-of-phase.
`Fe_S_breathing`	A_g	All four atoms move radially outward (cluster expansion).
`Fe_S_anti_breathing`	B_{1g}	Antisymmetric breathing.
`cluster_umbrella`	A_u	Synchronous motion along $\hat{\mathbf{z}}$ (out of plane).
`cluster_seesaw`	B_{2u}	Fe atoms move along $+\hat{\mathbf{z}}$ while S along $-\hat{\mathbf{z}}$.
`cluster_rotation`	B_{3u}	Rigid in-plane rotation.
`cluster_translation`	–	Uniform translation (should be zero in a converged frequency calculation, but is computed as a diagnostic).

16.3 Scoring

↝ `core.classify_oop_inp`

Each mode is scored against each named type by computing the inner product

$$s_l^{(k)} = \left| \frac{\mathbf{e}_l^{\text{cluster}} \cdot \mathbf{v}_{\text{template}}^{(k)}}{\|\mathbf{e}_l^{\text{cluster}}\| \cdot \|\mathbf{v}_{\text{template}}^{(k)}\|} \right|^2 \cdot 100\% \tag{56}$$

where $\mathbf{v}_{\text{template}}^{(k)}$ is the unit-normalised template vector for mode type k, and the inner product is over the 12 cluster-core components. Scores are cosine-squared similarities expressed as percentages. The two highest scores are recorded as `kern_primary` and `kern_secondary`, together with the localisation score `kern_loc` (the fraction of the mode's total amplitude residing in the cluster core).

16.4 Caveats

- Mode types are not orthogonal in general – the templates for A_g-symmetric stretches (Fe-Fe sym, S-S sym, Fe-S breathing) span a 3-dimensional A_g subspace, so scores across these three sum to $\leq 100\%$ but a single mode can mix them.
- Pure cluster-translation should have zero score across all types except `cluster_translation`. If the zero-frequency translation modes were not filtered out (section 2.4), they would diagnostically highlight if the harmonic approximation has broken down.
- For the rigorous, mathematically orthogonal decomposition, the SCSD sheet (section 15) is the authoritative reference; the kernel sheet is a human-friendly summary.

17 B-factor (Debye-Waller) accumulation

17.1 From mean-square displacement to crystallographic B

↝ `runner._run_analysis_single`

The crystallographic Debye-Waller factor for atom i is

$$B_i \equiv 8\pi^2 \langle u_i^2 \rangle / 3 \tag{57}$$

where $\langle u_i^2 \rangle = \langle \|\mathbf{r}_i(t) - \bar{\mathbf{r}}_i\|^2 \rangle$ is the time-averaged mean-square displacement, and the factor $8\pi^2/3$ comes from the isotropic gaussian convention in X-ray diffraction.

In a harmonic approximation the mean-square displacement decomposes into the sum of per-mode contributions:

$$\langle u_i^2 \rangle \;=\; \sum_{l=1}^{3N-6} \langle q_l^2 \rangle \cdot \|\mathbf{e}_{l,i}\|^2 \;=\; \sum_{l} u_{\mathrm{rms}}^2(\omega_l) \cdot \|\mathbf{e}_{l,i}\|^2, \tag{58}$$

where the second equality uses $\langle q_l^2 \rangle = u_{\mathrm{rms}}^2$ from section 4 and the orthogonal decomposition of the cartesian displacement of atom i over modes.

The pipeline accumulates the bracket sum mode by mode in Phase 4:

$$\mathrm{b\_accum}[i] \;\mathrel{+}=\; \sum_{\alpha=x,y,z} (\mathbf{e}_{l,i}^{(\alpha)} \cdot u_{\mathrm{rms}}(\omega_l))^2 \tag{59}$$

and finalises after the loop:

$$\boxed{B_i \;=\; \frac{8\pi^2}{3}\,\mathrm{b\_accum}[i]} \quad \rightsquigarrow \texttt{runner.\_run\_analysis\_single} \tag{60}$$

in units of Å^2.

17.2 Numerical details

The accumulator is an `np.ndarray` of length N_{atoms}, indexed by the position in the heavy-atom list (*not* by Gaussian centre). The mapping from Gaussian centre to accumulator index uses `idx_map[centre]` which is the 0-based position in the atoms list. The accumulator is initialised to zero at the start of the cluster run and the per-cluster reset in `reset_warning_state` also re-zeros `b_accum` implicitly via the per-cluster pipeline state.

17.3 Frequency window dependence

Since the accumulation runs only over modes inside the user's frequency window, the resulting B factors are *window-dependent*: a B factor reported for a 0–800 cm^{-1} analysis will be smaller than the same atom's true full-spectrum B factor, because higher-frequency modes contribute zero-point displacement amplitudes too. The window-restricted B is nonetheless useful diagnostically (the relative ordering of atoms by B is stable across windows; the absolute values are not).

Worked numerical example: B-factor for Fe_1 from three modes

We accumulate the B-factor of Fe_1 across three modes, using the test eigenvectors and frequencies from previous examples.

Mode	ω [cm^{-1}]	u_{rms} [Å]	$\mathbf{e}_{l,\mathrm{Fe}_1}$ (mass-unweighted)
1	80	0.0727	$(+0.10, +0.20, +0.40)$ (umbrella-like)
2	320	0.0407	$(+0.42, 0, 0)$ (Fe-S stretch)
3	510	0.0244	$(+0.05, +0.30, +0.15)$ (Fe-N bend)

Step 1 – per-mode contributions to b_accum[Fe_1]:

$$\begin{aligned}
&\text{mode 1: } (0.10 \cdot 0.0727)^2 + (0.20 \cdot 0.0727)^2 + (0.40 \cdot 0.0727)^2 \\
&\quad = (0.00727)^2 + (0.01454)^2 + (0.02908)^2 = 5.3 \times 10^{-5} + 2.1 \times 10^{-4} + 8.5 \times 10^{-4} \\
&\quad = 1.11 \times 10^{-3}\ \text{Å}^2, \\
&\text{mode 2: } (0.42 \cdot 0.0407)^2 + 0 + 0 = (0.01709)^2 = 2.92 \times 10^{-4}\ \text{Å}^2, \\
&\text{mode 3: } (0.05 \cdot 0.0244)^2 + (0.30 \cdot 0.0244)^2 + (0.15 \cdot 0.0244)^2 \\
&\quad = (0.00122)^2 + (0.00732)^2 + (0.00366)^2 = 1.5 \times 10^{-6} + 5.4 \times 10^{-5} + 1.3 \times 10^{-5} \\
&\quad = 6.86 \times 10^{-5}\ \text{Å}^2.
\end{aligned}$$

Step 2 – sum:

$$\text{b\_accum}[\text{Fe}_1] = 1.11 \times 10^{-3} + 2.92 \times 10^{-4} + 6.86 \times 10^{-5} = 1.47 \times 10^{-3}\ \text{Å}^2.$$

Step 3 – multiply by $8\pi^2/3 = 26.32$:

$$B_{\text{Fe}_1} = 26.32 \cdot 1.47 \times 10^{-3} = \boxed{0.0387\ \text{Å}^2}.$$

Sanity: For a real protein at 80 K, B factors at the Fe site are typically 0.5 to 2.0 Å^2. Our value 0.039 Å^2 is much lower because we are accumulating over only 3 mock modes; a full DFT spectrum would contribute hundreds of modes summing into the realistic range.
Observation: The umbrella mode (mode 1, low frequency, high u_{rms}) dominates the B-factor with 76% of the total, even though its amplitude on Fe_1 is moderate. This is the characteristic feature of low-frequency modes – they have small displacement amplitudes per mode but large u_{rms}, and contribute disproportionately to the time-averaged mean-square displacement.
In the code: `runner._run_analysis_single` loops over selected modes and accumulates `b_accum` in-place; the final `b_factors` array is exposed on the `B_factors` sheet of the main workbook.

18 Connection to Marcus-Hush ET theory

The spectroscopic reorganization λ_X^{pair} defined in equation (35) shares the quadratic functional form $\frac{1}{2}\mu\omega^2(\Delta r)^2$ with the classical Marcus reorganization energy, but the two quantities are conceptually different and should not be conflated. This section explains the relationship in detail.

18.1 Marcus classical reorganization energy

In Marcus theory of electron transfer (ET), the reorganization energy λ_M is the work required to distort the equilibrium nuclear configuration of the reactant state to the equilibrium nuclear configuration of the product state, with the electron held in place:

$$\lambda_M \;=\; \sum_i \frac{1}{2}\, k_i\, (\Delta Q_i)^2 \;+\; \lambda_{\text{solvent}}, \tag{61}$$

where $k_i = \mu_i \omega_i^2$ is the harmonic force constant of internal mode i, ΔQ_i is the static (Born-Oppenheimer) displacement of mode i's equilibrium between the two electronic states, and λ_{solvent} is the dielectric reorganization contribution. The ET rate constant in the Marcus non-adiabatic regime is then

$$k_{\text{ET}} = \frac{2\pi}{\hbar} |V_{DA}|^2 \frac{1}{\sqrt{4\pi \lambda_M k_{\text{B}} T}} \exp\left(-\frac{(\Delta G^\circ + \lambda_M)^2}{4 \lambda_M k_{\text{B}} T} \right), \tag{62}$$

where V_{DA} is the donor-acceptor electronic coupling and ΔG° is the standard free-energy of the reaction.

18.2 Spectroscopic reorganization energy

Our λ_X^{pair}, by contrast, uses Δr = *thermal/zero-point displacement amplitude of a single vibrational mode along a bond axis*, not the static geometry shift between two electronic states. It is a frequency-resolved measure of which bond modes "zappel" (oscillate) the most at the chosen temperature.

The two reduce to each other only in a special limit: the *single-promoting-mode* regime of ET, where the entire λ_M is concentrated in one internal mode and that mode also dominates the spectroscopic amplitude. In general the spectroscopic λ is a different observable – it tells the reader which modes *currently* oscillate, while the Marcus λ tells which static geometric changes are induced by *electron motion*.

18.3 The spin-Boson model bridge

The spin-Boson model of ET treats the donor-acceptor system as a two-level electronic system linearly coupled to a bath of harmonic modes. The coupling is parameterised by the spectral density $J(\omega) = \sum_i \frac{c_i^2}{2 m_i \omega_i} \delta(\omega - \omega_i)$, where c_i is the coupling strength of bath mode i to the electronic system. The Marcus λ_M is then

$$\lambda_M = \frac{1}{\pi} \int_0^\infty \frac{J(\omega)}{\omega} \, d\omega. \tag{63}$$

This is the integrated spectral density divided by frequency – it weights low-frequency modes most heavily. Our $\Lambda_X^{\text{total}}(\omega) = \sum_l \lambda_X^{(l)}$ is a similar integral but with different weighting:

$$\Lambda_X^{\text{total}} \;=\; \sum_l \tfrac{1}{2} \mu_l \omega_l^2 (\Delta r_l)^2 / (hc), \tag{64}$$

where $\Delta r_l \propto u_{\text{rms}}(\omega_l) = \sqrt{\hbar / (2 \mu_l \omega_l) \coth(\beta \hbar \omega_l / 2)}$. Substituting:

$$\Lambda_X^{\text{total}} \;=\; \frac{1}{hc} \sum_l \frac{\hbar \omega_l}{4} \coth(\beta \hbar \omega_l / 2) \, \alpha_{X,l}^2, \tag{65}$$

where $\alpha_{X,l}$ is the bond-axis projection coefficient (equation (21) structure). At $T \to 0$, coth $\to 1$ and $\Lambda^{\text{total}} = \sum_l \omega_l \alpha_{X,l}^2 / (4hc)$, which is the spin-Boson reorganization specific to channel X.

The mapping is:

- Marcus λ_M: sum of $\frac{c_i^2}{2k_i}$ over all modes coupled to the ET reaction coordinate.
- Our Λ_X^{total}: sum of $\frac{1}{4} \hbar \omega_l \alpha_{X,l}^2$ over all modes with displacement $\alpha_{X,l}$ along bond X at temperature T.

The two coincide when bond X *is* the reaction coordinate and at $T \to 0$.

18.4 Huang-Rhys factors and the Franck-Condon distribution

For a single mode of frequency ω_l, the Huang-Rhys factor S_l is the dimensionless ratio of reorganization energy to $\hbar\omega_l$:

$$S_l = \frac{\lambda_l}{\hbar\omega_l} = \frac{1}{2}\frac{\mu_l\omega_l(\Delta Q_l)^2}{\hbar} = \frac{(\Delta Q_l)^2}{2\,u_{\mathrm{rms}}^2(\omega_l, T=0)}. \tag{66}$$

S_l counts "vibrational quanta worth" of reorganization energy in mode l. The Franck-Condon distribution of vibrational overlap factors between displaced harmonic oscillators is then a Poisson distribution with parameter S_l:

$$P_l(n) = \frac{S_l^n}{n!}\, e^{-S_l}, \tag{67}$$

giving the relative intensity of the $0 \to n$ transition in optical spectroscopy. For our typical Fe-S modes, $S_l \sim 0.01$–0.1 (because the per-mode Δr is small compared to u_{rms}), placing the modes in the weak-coupling regime where the dominant Franck-Condon contribution is $0 \to 0$ ($P = e^{-S} \approx 1$). This is consistent with the narrow homogeneous linewidths observed in NRVS.

18.5 Spectroscopic λ as a Franck-Condon-weighted observable

The NRVS cross-section at frequency ω (see section 20) involves the Franck-Condon factor of a 0-phonon transition, which for a single mode reduces to:

$$\sigma_{\mathrm{NRVS}}(\omega) \;\propto\; \sum_l e^{-2W_l} S_l\, \delta(\omega - \omega_l), \tag{68}$$

where e^{-2W_l} is the Lamb-Mössbauer factor and S_l is the Huang-Rhys factor. Crucially, the cross-section is proportional to S_l – which is proportional to $(\Delta Q_l)^2\omega_l$ – which is exactly the integrand of our Λ^{total}.

Therefore: Λ_X^{total}, integrated over the analysis window, is proportional to the integrated NRVS cross-section attributed to bond modulations along axis X, in the weak-coupling, low-T limit.

This is the link that makes Λ a direct NRVS observable rather than just a per-bond "zappel-energy". The proportionality constant depends on $\langle Fe^{57}\rangle$ enrichment, the Lamb-Mössbauer factor of Fe at the measurement temperature, and the experimental resolution – factors that absorb into the overall NRVS intensity scale.

19 Lamb-Mössbauer factor and B-factors

19.1 Definition

The Lamb-Mössbauer factor f_{LM} is the fraction of recoil-free γ-ray transitions in a Mössbauer experiment:

$$f_{\mathrm{LM}} = e^{-\langle k^2 x^2\rangle}, \tag{69}$$

where k is the magnitude of the γ-ray's wave vector and $\langle x^2\rangle$ is the mean-square displacement of the Mössbauer nucleus along the γ-ray direction. For Fe^{57} at 14.4 keV, $k = 7.3 \times 10^{10}$ m^{-1} $= 7.3$ Å^{-1}.

The mean-square displacement $\langle x^2\rangle$ along a single direction is one-third of the isotropic mean-square displacement $\langle r^2\rangle$:

$$\langle x^2\rangle = \frac{1}{3}\langle r^2\rangle = \frac{1}{3}\sum_l u_{\mathrm{rms}}^2(\omega_l)\,\|\mathbf{e}_{l,\mathrm{Fe}}\|^2. \tag{70}$$

Substituting into f_{LM} and using $k^2 = 53.3$ Å^{-2}:

$$f_{\mathrm{LM}}(T) = \exp\left(-\frac{k^2}{3}\sum_l u^2_{\mathrm{rms}}(\omega_l)\,\|\mathbf{e}_{l,\mathrm{Fe}}\|^2\right). \tag{71}$$

19.2 Connection to crystallographic B-factors

The B-factor of a Mössbauer nucleus (section 17) is exactly

$$B_{\mathrm{Fe}} = \frac{8\pi^2}{3}\sum_l u^2_{\mathrm{rms}}(\omega_l)\,\|\mathbf{e}_{l,\mathrm{Fe}}\|^2 = 8\pi^2\langle x^2\rangle. \tag{72}$$

Therefore:

$$f_{\mathrm{LM}}(T) = \exp\left(-\frac{k^2}{8\pi^2}\,B_{\mathrm{Fe}}(T)\right) = \exp(-0.0625\cdot B_{\mathrm{Fe}}\ [\text{Å}^2]). \tag{73}$$

For a typical NRVS-quality system at 80 K with $B_{\mathrm{Fe}} = 0.7$ Å^2:

$$f_{\mathrm{LM}} = e^{-0.044} = 0.957.$$

About 96% of the absorption events are recoil-free at this temperature – explaining why cryogenic NRVS produces clean phonon spectra. At 300 K, $B_{\mathrm{Fe}} \approx 2.5$ Å^2 gives $f_{\mathrm{LM}} \approx 0.86$, still high but no longer overwhelming.

19.3 Implication for the pipeline

The pipeline reports B_{Fe} (and B for every other atom) on the `B_factors` workbook sheet. A reviewer comparing the output to NRVS measurements can independently verify the Lamb-Mössbauer factor via the formula above. A pipeline run that reports $B_{\mathrm{Fe}} > 5$ Å^2 at 80 K should be regarded with suspicion – the DFT-predicted modes are inconsistent with the experimentally observed sharp NRVS lines.

20 NRVS cross-section vs. pDOS

A frequent source of confusion is the difference between the *NRVS cross-section* (what the instrument actually measures) and the *partial density of states* (pDOS, what theorists often plot). The pipeline computes neither directly, but its $M_X(\omega)$ modulation spectra are intermediate between them.

20.1 NRVS cross-section (Sturhahn formula)

The single-phonon NRVS absorption cross-section at energy $E = \hbar\omega$ above the Mössbauer line is [6]:

$$\sigma_{\mathrm{NRVS}}(\omega) \;=\; \sigma_0\, f_{\mathrm{LM}}\,\frac{\hbar\omega/(2m_{\mathrm{Fe}})}{\omega^2}(1+n(\omega))\,D_{\mathrm{Fe}}(\omega), \tag{74}$$

where σ_0 is the nuclear-resonance cross-section, f_{LM} is the Lamb-Mössbauer factor, $n(\omega) = (e^{\beta\hbar\omega}-1)^{-1}$ is the Bose-Einstein occupation, and $D_{\mathrm{Fe}}(\omega)$ is the projected density of states on the Fe nucleus:

$$D_{\mathrm{Fe}}(\omega) = \sum_l e^2_{l,\mathrm{Fe}}\,\delta(\omega-\omega_l), \tag{75}$$

with $e_{l,\mathrm{Fe}}$ the mass-weighted Fe component of mode l's eigenvector.

20.2 Fe-pDOS = NRVS cross-section, modulo prefactors

Equation equation (74) is what the NRVS instrument measures. The $D_{\mathrm{Fe}}(\omega)$ factor is the Fe-projected density of states. To extract the pDOS from a measured cross-section, one divides out the prefactors $\sigma_0, f_{\mathrm{LM}}$, the kinematic factor $\hbar\omega/2m_{\mathrm{Fe}}/\omega^2$, and the Bose-Einstein factor – standard procedure in the NRVS analysis literature.

The pipeline does *not* compute the cross-section directly. What it produces is $M_X(\omega)$, the bond-projected modulation spectrum (section 14). The two are related but distinct:

- $D_{\mathrm{Fe}}(\omega) \propto \sum_l \|\mathbf{e}_{l,\mathrm{Fe}}\|^2 \delta(\omega - \omega_l)$: projected onto a single *atom*.
- $M_X(\omega) \propto \sum_l (\Delta r_{X,l})^2 \mathcal{G}(\omega - \omega_l)$: projected onto a *bond axis*.

The two coincide when the relevant atom moves only along one bond axis (e.g. in a pure Fe-S stretch, $\mathbf{e}_{\mathrm{Fe}}$ is parallel to $\hat{\mathbf{b}}_{\mathrm{FeS}}$). For mixed modes the two differ: a mode that moves Fe perpendicular to all Fe-S bonds would contribute to D_{Fe} but not to M_{FeS}.

20.3 Why bond-projection is more useful for PCET

For PCET analysis, the bond-projected spectrum $M_X(\omega)$ identifies *which bonds* modulate together with the ET reaction coordinate – the relevant question for the "promoting modes" that drive PCET. Pure pDOS would conflate all Fe motion (including bond-irrelevant rocks and tilts) into one curve. The bond-projection is the analysis-stage specialisation that makes the pipeline useful for PCET.

21 Anharmonicity: what we do not correct for

The entire pipeline assumes harmonic vibrations. Real molecular vibrations are anharmonic; this section lists the corrections we omit and the regime where the omission is justified.

21.1 Diagonal anharmonicity

A diagonal anharmonic mode has the potential $V(Q) = \frac{1}{2}\mu\omega^2 Q^2 + \frac{1}{6}k_3 Q^3 + \frac{1}{24}k_4 Q^4 + \dots$. The leading correction to u_{rms} is

$$u^2_{\mathrm{rms}} \approx (u^{\mathrm{harm}}_{\mathrm{rms}})^2 \left(1 - \frac{k_3^2}{\mu^2\omega^4} \cdot \frac{15\hbar}{2\mu\omega} + \frac{k_4}{2\mu^2\omega^4} \cdot \frac{3\hbar}{\mu\omega}\right), \tag{76}$$

which for typical Fe-S systems amounts to a few percent at most. We do not apply this correction. It would require the cubic and quartic anharmonic constants from a higher-order DFT calculation (`Freq=Anharm`); the typical Gaussian frequency job we read does not contain these.

21.2 Off-diagonal anharmonicity (mode-mode coupling)

The truly harmonic spectrum has N_{modes} independent oscillators; the truly anharmonic spectrum has N^2_{modes} pair-couplings $V_{ij} = k_{ij} Q_i Q_j$ that mix modes. Strong mode-mode coupling can shift peak frequencies,

broaden lines, and produce combination bands at $\omega_i \pm \omega_j$. We ignore all of these. The pipeline's $M_X(\omega)$ peaks should be read as "the harmonic-DFT-predicted peaks before anharmonic broadening".

21.3 Frequency scaling

A common partial-anharmonic correction is to multiply all DFT-predicted frequencies by a uniform scale factor (typically 0.96–0.99, functional-dependent) to bring them closer to experimental peak positions. Our Λ and M_X outputs are *not* scaled; the user can apply a uniform scaling post-hoc by multiplying all reported frequencies. The Λ values are quadratic in ω – a 1% frequency scaling produces a 2% Λ change.

21.4 Temperature-dependent harmonic frequencies

The harmonic frequencies are computed at the equilibrium geometry (DFT-optimised $T = 0$ structure). At finite temperature, the average geometry shifts due to thermal expansion, and the "effective" harmonic frequencies shift correspondingly. This is usually a small effect (few cm^{-1}), and the pipeline ignores it. A user who needs precise comparison with high-T experiment should consider supplying a temperature-corrected DFT structure as input.

21.5 Practical regime of validity

Anharmonic corrections become important when:

- The mode is near a structural transition (the cluster being close to a bond-rearrangement, e.g. between rhombic and butterfly geometries).
- The mode is in a highly nonlinear coupling regime (multi-quantum overtones).
- Hydrogen-stretching modes ($> 2500\ \mathrm{cm}^{-1}$) – our analysis focuses below $1000\ \mathrm{cm}^{-1}$ where anharmonicity is mild.

For the [2Fe-2S] systems analysed by this pipeline at $T \leq 100$ K, harmonic predictions are typically accurate to ±5% in frequency and ±10% in NRVS-intensity-related observables. The validation tolerances in section 88 are set accordingly.

Part III

Algorithms (complete pseudocodes)

The pseudocodes in this part are intended as a reviewer's high-fidelity reading of the code in each phase, condensed to the control flow and data flow that matter physically. Each pseudocode ends with a $\rightsquigarrow$ marker; the actual implementation can be read verbatim by following the marker. Pseudocode is given in a language-agnostic syntax with the conventions of table 4.

Convention	Meaning
`a <- b`	Assignment. Multiple assignment in tuples permitted.
`a, b <- f(x)`	Tuple-unpacking of the return value of f.
`for x in C`	Iteration over collection C.
`foreach`	As `for` but order is not significant.
`assert P`	Halt with an error if predicate P is false.
`require P`	As `assert` but emits a clear user-facing message.
`yield x`	Generator-style emission of x to the caller.
`return`	Return from the current procedure.
`...`	Ellipsis: code irrelevant to the present logic.

Table 4: Pseudocode conventions used throughout Part III.

22 Algorithm: `scan_log`

⇝ `logio.scan_log`

`scan_log` performs a streaming, byte-level scan of the Gaussian `.log` file to find the byte offsets of three pattern types: the Standard-Orientation headers, the normal-coordinate-block headers, and the Frequencies headers. It must be `O`(N) in file size and constant in memory; the only state carried between chunks is a small rolling tail buffer that catches patterns spanning chunk boundaries.

22.1 Byte-level pattern definitions

The three regex patterns used by `scan_log` are:

```
pat_so = re.compile(b"Standard orientation:") pat_nc = re.compile(b"and normal coordinates:") pat_fr =
re.compile(b"\s*Frequencies\s*-{2,}", re.MULTILINE)
```

A few subtleties worth noting:

- `pat_so` and `pat_nc` are *plain substring matches* – no regex meta-characters. The Gaussian writer is deterministic about how these lines are formatted, so a substring match is both faster and more robust than a regex.
- `pat_fr` requires *at least two* dashes after "Frequencies" (with optional whitespace). This matters because `hpmodes` blocks write three dashes (`Frequencies --`) while standard blocks write two (`Frequencies -`). The `-{2,}` quantifier captures both. Less than two dashes would be a Gaussian format violation; we have not observed such logs in the wild.
- `pat_fr` uses `re.MULTILINE` so the `\s*` anchor at the start matches the per-line leading whitespace, not just the file start.
- All patterns are byte-strings (`rb"..."`), not unicode strings, so they apply directly to the raw bytes of the file – no codec conversion needed during the scan.

22.2 The 200-byte tail-overlap

Each chunk read from disk has size `cfg.scan_chunk_mb` · `1024` · `1024` bytes (default 16 MiB). At chunk boundaries, a pattern split across the boundary would be missed if each chunk were processed independently. The remedy is to retain the last 200 bytes of every chunk as a *tail buffer*, prepended to the next chunk:

Listing 2: Streaming byte-offset scan of the Gaussian log with tail-overlap deduplication. Implementation: `logio.scan_log`.

```
input:  filepath, chunk_size_bytes (16 MiB default), tail_overlap (200 B)
output: so_off, nc_off, fr_off    (sorted lists of byte offsets)

seen_so, seen_nc, seen_fr <- {}, {}, {}    # dedup sets
so_off, nc_off, fr_off <- [], [], []

tail   <- b""
offset <- 0          # bytes consumed so far (not counting tail)

with open(filepath, "rb") as fh:
    while True:
        block <- fh.read(chunk_size_bytes)
        if len(block) == 0: break
        data <- tail + block
        base <- offset - len(tail)
            # base = file-absolute byte index of data[0]

        foreach pattern, target_list, seen_set in
                [(pat_so, so_off, seen_so),
                 (pat_nc, nc_off, seen_nc),
                 (pat_fr, fr_off, seen_fr)]:
            foreach match in pattern.finditer(data):
                abs_off <- base + match.start()
                if abs_off not in seen_set:
                    seen_set.add(abs_off)
                    target_list.append(abs_off)

        tail   <- data[-200:]
        offset <- offset + len(block)

so_off.sort(); nc_off.sort(); fr_off.sort()
return so_off, nc_off, fr_off
```

Why 200 bytes? The longest of the three patterns is "`and normal coordinates:`" (22 characters); the `pat_fr` regex with `\s*Frequencies\s*-{2,}` matches at most a few dozen bytes. 200 bytes is generously above the longest possible pattern, leaving headroom for future pattern additions.

Why dedup with sets? Without deduplication, every pattern that falls inside the tail-overlap window of two adjacent chunks would be reported twice (once at the end of chunk N, once at the start of chunk $N+1$). The per-pattern `seen_*` set discards duplicates by absolute file offset.

22.3 Worked example: byte layout of a Gaussian log

For a typical [2Fe-2S] `hpmodes` Gaussian log, the first 4 KiB after the SCF-convergence point contain the equilibrium geometry and its surrounding metadata. A representative excerpt from byte offset $\approx 280\,000$ (immediately after the SCF block) is:

```
Standard orientation:
---------------------------------
Center Atomic Atomic Coordinates (Angstroms)
```

```
Number Number Type X Y Z
-----------------------------------
1 7 0 -4.679206 2.130175 2.693054
2 6 0 -4.196084 1.896548 1.322426
3 6 0 -5.362864 1.844689 0.374943
```

The pattern `pat_so` matches the literal text `"Standard orientation:"` at the start of line 1 of this excerpt. The byte offset reported by `scan_log` for this match is the position of the `S` in `"Standard"`, *not* the start of the line (which has 26 leading spaces). `read_std_orient` (section 23) relies on this offset and seeks to it before parsing.

For a frequency block, the layout is:

```
1 2 3
A A A
Frequencies -- 11.3016 17.0158 19.3550 24.8910 25.6887
Reduced masses -- 8.0483 8.4516 6.1589 8.1501 7.3141
Force constants -- 0.0006 0.0014 0.0014 0.0030 0.0028
IR Intensities -- 0.4340 0.0239 0.0809 0.7555 0.6085
Coord Atom Element:
1 1 7 -0.14255 0.17791 -0.01001 0.02676 0.09199
2 1 7 -0.02809 0.05164 -0.03976 0.09096 -0.03367
```

Several Reviewer-relevant observations:

- The mode numbers (top: "`1 2 3`") appear on the line *before* the symmetry labels ("`A A A A A`") – separated by the columns of 5 mode numbers (for hpmodes) or 3 (for standard). `_read_block_at_freq_line` seeks 300 bytes back from the `Frequencies --` match and scans forward to recover the mode numbers from the all-digit line.
- The `Frequencies --` pattern has *three* dashes, not two; the `pat_fr` regex matches both via `-{2,}`.
- The number of modes per block (5 in hpmodes, 3 in standard) is inferred from the count of frequencies parsed from the `Frequencies --` line.
- The line `"Coord Atom Element:"` is the `hpmodes` signature: its presence sets `is_hp` = True. Standard blocks use the line `"Atom AN X Y Z"` (or omit the column header entirely).
- Within the eigenvector body, columns 1-3 are `Coord` (1=x, 2=y, 3=z), `Atom number` and `Element atomic number`. The eigenvector components start in column 4.

22.4 Edge cases handled by the implementation

(E1) Duplicate matches from the tail-overlap region. Prevented by the `seen_*` sets, as explained above.

(E2) Pattern at the very start of the file. The first chunk has `tail = b""`, so a match at byte 0 is reported with offset 0 (no negative-offset bug).

(E3) Pattern in the very last 200 bytes of the file. The final chunk read is shorter than `chunk_size_bytes`; the loop reads as much as available and the final `tail` update is harmless because the next `fh.read` returns

empty.

(E4) Empty file. `fh.read` returns `b""` immediately; the loop exits; empty lists are returned. The caller (section 23) handles the all-empty case as a fatal error.

(E5) Missing `Frequencies` blocks entirely. If the user supplies a Gaussian log without a frequency calculation, `fr_off` comes back empty. The caller raises a clear error "no normal modes found in the log – did the frequency job converge?".

(E6) Truncated file (terminated mid-block). The scan completes (no error), but `read_all_meta` may produce `BlockInfo`s with incomplete frequencies lists. `_read_block_at_freq_line` returns `None` for any block with zero frequencies, and the offending offsets are dropped. The user is alerted in the run-log: "N frequency offsets found but $M < N$ blocks parsable – log may be truncated."

(E7) Patterns inside arbitrary strings (false positives). The pattern `"Standard orientation:"` is unique to the expected coordinate-block header in normal Gaussian output. A user-supplied geometry comment line containing this substring would falsely match, but Gaussian's writer never emits user comments inside the SCF output region, and we have not encountered a false positive in practice. The pattern `pat_fr` requires the leading-whitespace anchor plus the two-dash terminator, which is a strong structural signature.

(E8) Multi-byte UTF-8 sequences in comments. The patterns are byte-strings, so they match byte-for-byte. A UTF-8 encoded German umlaut in a user-supplied job title (e.g. "Mosbauer" written as "Mössbauer" with `0xC3 0xB6`) takes two bytes – but again, Gaussian never emits these in the relevant blocks. The byte-pattern scan therefore never produces a false negative.

(E9) Compressed input (`.log.gz`, `.log.xz`). `scan_log` does *not* transparently handle compressed logs. The user is expected to decompress first (this is documented in the manual). A future extension could wrap the input with `gzip.open`/`lzma.open`, but the resulting random- access seek operations (needed by `get_eigvec`) would no longer be O(1).

22.5 Computational complexity and performance

Time. $O(N_{\text{bytes}} \cdot N_{\text{patterns}})$, with `bytes.find`/`re.finditer` both running at near memory-bandwidth on modern hardware. The default 16 MiB chunk size empirically gives the best throughput on NVMe SSDs ($\sim$ 2–3 GiB/s). HDDs benefit from larger chunks (32-64 MiB) to amortise seek overhead.

Memory. $O(\text{chunk\_size} + \text{tail\_overlap} + N_{\text{matches}})$. For a 1 GiB log with $\sim$ 10 000 modes (i.e. $\sim$ 10 000 / 5 = 2000 hpmodes blocks, 1 Standard-Orientation, 1 normal-coordinates section), peak RAM is $\sim 16 + 0.0002 + 0.024$ MiB ≈ 16 MiB.

Live progress. The implementation prints a progress line every chunk:

```
Scan:  38.5% 2614 MB/s ETA 7s
```

The throughput is computed as `offset / elapsed`; the ETA is `(total_size - offset) / (offset / elapsed)`. The progress is also logged to the `RunLog` every 10% for the post-run audit trail.

22.6 Cache integration

The byte-offset triple `(so_off, nc_off, fr_off)` is the *sole* cached artefact from the scan phase. The cache key is the SHA-256 hash of:

```
key = sha256(
    filepath +
    str(file_size_bytes) +
    str(file_mtime_ns) +
    f"py{sys.version_info.major}.{sys.version_info.minor}" +
    _CACHE_VERSION
)
```

The cache value is the pickled `(so_off, nc_off, fr_off, _CACHE_VERSION)` tuple, written to `.scan_cache_{key}.pkl` in the current working directory. On the next `scan_log` call:

- If the cache file exists and the stored `_CACHE_VERSION` matches the current `_CACHE_VERSION`, the offsets are loaded directly.
- If the file mtime, file size, or Python version changes, the cache key changes – the old cache file is simply ignored. (Disk-space cleanup is the user's responsibility; in practice the cache files are 50–500 KiB.)
- If `use_cache` = False, the cache is bypassed entirely (neither read nor written).

This cache layer was the source of FUND 2 (section 65): pre-v1.0.4 the file mtime was not included in the key, so re-running an analysis after replacing the log would silently use stale offsets.

23 Algorithm: `read_all_meta`

⇝ `logio.read_all_meta`

`read_all_meta` iterates over the `fr_off` list from section 22 and parses the per-mode metadata of each mode block. No eigenvector data are loaded here – only header information. The function also performs the assignment of mode numbers to blocks (which is non-trivial because Gaussian's mode numbering inside hpmodes blocks restarts from 1 in every section).

23.1 Three-stage pipeline

1. **Per-block parsing.** For each `Frequencies --` offset, call `_read_block_at_freq_line` to extract that block's frequencies, reduced masses, force constants, IR intensities, mode numbers, and the byte offset of the eigenvector data.
2. **Mode-number resolution.** Recover correct global mode numbers using the section structure of the log (one section per "`and normal coordinates:`" marker).
3. **HP/standard preference map.** Build the `best_block` mapping that prefers hpmodes blocks over standard blocks for any mode that appears in both.

23.2 Per-block parsing (_read_block_at_freq_line)

The byte structure of one frequency block, as it appears in a Gaussian hpmodes log, is:

```
 offset f                                                          (matched by pat_fr)
Frequencies -- 11.3016 17.0158 ...
Reduced masses -- 8.0483 8.4516 ...
Force constants -- 0.0006 0.0014 ...
IR Intensities -- 0.4340 0.0239 ...
Coord Atom Element:  ← HP-mode marker; data_offset starts after this line
 offset d                                                  (eigenvector body begins here)
1 1 7 -0.14255 0.17791 ...
2 1 7 -0.02809 0.05164 ...
```

For a standard (non-hpmodes) block, the equivalent layout is:

```
 offset f
Frequencies - 11.30 17.02 19.36
Red.  masses - 8.05 8.45 6.16
Frc consts - 0.00 0.00 0.00
IR Inten - 0.43 0.02 0.08
Atom AN X Y Z X Y Z X Y Z
1 7 -0.05 -0.05 -0.07 -0.06 -0.02 0.02 0.00 -0.01 0.03
```

_read_block_at_freq_line parses this by the following procedure:

Listing 3: Per-block metadata parser. Implementation: logio._read_block_at_freq_line.

```
input:  filepath, fr_offset
output: BlockInfo or None

bi <- BlockInfo(offset=fr_offset)

# (a) parse the Frequencies line itself
with open(filepath, "rb") as fh:
    fh.seek(fr_offset)
    line <- fh.readline().decode("utf-8", errors="replace")
    bi.is_hp <- bool(re.match(r"\s*Frequencies\s*---", line))
        # 3+ dashes = hpmodes; 2 dashes = standard
    bi.freqs <- _parse_dashes(line)
        # _parse_dashes strips "Frequencies ---" prefix and returns
        # a list of floats; missing or malformed values become NaN
    n <- len(bi.freqs)
    if n == 0:
        return None     # malformed block

    # (b) recover mode numbers from the line BEFORE fr_offset
    # Gaussian writes them as:    1     2     3     4     5
    # immediately preceding the symmetry labels (A A A ...)
    fh.seek(max(0, fr_offset - 300))
    prev_lines <- read all complete lines until fh.tell() > fr_offset
    foreach pline in reversed(prev_lines):
        parts <- pline.strip().split()
        if all parts are digit strings:
            bi.mode_nums <- [int(p) for p in parts]
            break
```

```
    if no all-digit line found or count mismatches n:
        bi.mode_nums <- [1, 2, ..., n]      # local fallback

    # (c) parse Red masses / Frc consts / IR ints + find data_offset
    fh.seek(fr_offset)
    fh.readline()    # skip Frequencies line
    for _ in range(60):           # 60-line window
        prev_pos <- fh.tell()     # B4-fix: record pos BEFORE readline
        line <- fh.readline()
        ll <- line.lower()
        if "red" in ll and "mass" in ll:
            bi.red_masses <- _parse_dashes(line)
        elif "frc" in ll or ("force" in ll and "const" in ll):
            bi.frc_consts <- _parse_dashes(line)
        elif "coord atom" in ll:
            bi.is_hp <- True
            bi.data_offset <- fh.tell()
            break
        elif "atom" in ll and (" an" in ll or "an " in ll):
            bi.is_hp <- False
            bi.data_offset <- fh.tell()
            break
        elif re.match(r"\s+\d+\s+\d+\s+[-\d.]", line):
            # first data row encountered without a column header
            bi.is_hp <- False
            bi.data_offset <- prev_pos    # B4-fix
            break

    # (d) pad missing optional fields to length n
    for lst in (bi.red_masses, bi.frc_consts):
        while len(lst) < n: lst.append(0.0)
    while len(bi.syms) < n: bi.syms.append("A")

return bi
```

The 60-line window. Gaussian can include optional extra header rows between the `Frequencies --` line and the eigenvector body, such as "`Raman Activ -`", "`Depolar (P)`", NMR shielding, etc. The 60-line window is empirically large enough to cover all observed Gaussian-16 output variants. If 60 lines pass without finding either marker or a first data row, the block is treated as malformed (`None` returned).

Bugfix B4: position before `readline`. The `prev_pos` is recorded *before* the `readline` call. This is critical for the no-header standard-block case (the `elif re.match` branch): when the first data row is encountered without a preceding `Atom AN` header line, the `data_offset` must point at the start of that data row, not at the position after reading it. The previous version of the code used `fh.tell() - len(line.encode())`, which subtly mis-counted by the encoded-vs-raw byte-length difference in the presence of non-ASCII characters (rare but possible).

Bugfix B5: `data_offset = -1` sentinel. If neither marker nor data row is found within the 60-line window, `data_offset` remains at its default -1. The downstream `get_eigvec` explicitly checks this sentinel and raises a clear error rather than seeking to byte -1 (which would silently `seek_end` on some operating systems).

23.3 Mode-number resolution and the section structure

Each `and normal coordinates:` marker in the log opens a new *section*. Inside a section, Gaussian numbers the modes starting from 1 again. To recover global mode numbers across the whole log, `read_all_meta` performs this remapping:

Listing 4: Mode-number resolution across sections. Implementation: `logio.read_all_meta`.

```
input:  raw_blocks (per-block parsed by _read_block_at_freq_line),
        nc_sorted  (sorted list of "and normal coordinates:" offsets)
output: blocks      (same blocks, mode_nums corrected)

section_count <- {}      # nc_offset -> number of blocks already seen
                         # in that section
foreach bi in raw_blocks:
    sec <- max(nc for nc in nc_sorted if nc <= bi.offset)
        # which section does this block belong to?
    section_count.setdefault(sec, 0)
    nm <- len(bi.mode_nums)

    # Heuristic: if the parsed mode numbers are [1, 2, ..., nm] AND
    # we've already seen blocks in this section, then the parser
    # found local mode numbers, not global. Renumber globally.
    fallback <- (bi.mode_nums == [1, 2, ..., nm]
                 and section_count[sec] > 0)

    # Same fallback if parser failed altogether (mode_nums empty
    # or starts with <= 0)
    if fallback or not bi.mode_nums or bi.mode_nums[0] <= 0:
        start <- section_count[sec] * nm + 1
        bi.mode_nums <- [start, start+1, ..., start+nm-1]

    section_count[sec] += 1
    blocks.append(bi)
```

Why the section-aware remapping. A Gaussian job that writes both an `hpmodes` frequency analysis and a separate `Freq=Anharm` job will have two `and normal coordinates:` markers, two restart-at-1 mode numberings, and possibly overlapping global mode numbers. The remapping ensures that the global mode numbers are unique across the whole log, which is what `best_block` and `cand_map` require.

23.4 HP/standard preference

The final step builds the two output maps:

```
best     <- {}    # mode_num -> preferred BlockInfo (HP wins)
cand_map <- {}    # mode_num -> list of all candidate BlockInfos
foreach bi in blocks:
    foreach mn in bi.mode_nums:
        if mn not in best:
            best[mn] <- bi
        elif bi.is_hp and not best[mn].is_hp:
            best[mn] <- bi           # promote HP over standard
        cand_map.setdefault(mn, []).append(bi)
```

`best_block` is used by the main mode-loop as the fast path; `cand_map` is used by `analyze_mode_with_fallback` when the preferred block's eigenvector data fail to parse (e.g. truncated file, corrupt block) – the loop then tries the next- best candidate from `cand_map`.

23.5 Sentinels and missing data

A missing reduced mass (Gaussian prints "`********`" when the value overflows the fixed-width format – happens for some very soft modes) is parsed as 0.0 by `_parse_dashes` (which returns the empty list on parse failure and is then padded with zeros up to length n). Downstream code treats `red_mass` $= 0$ as fatal at the mode-selection stage – the mode is skipped with a warning, and the user is alerted in the run-log.

Why both `best_block` and `cand_map`: `best_block` is the fast path for the common case (one block per mode). `cand_map` is the fallback list used by `analyze_mode_with_fallback` (section 27) if the preferred block's eigenvector data are unreadable (truncated file, etc.).

23.6 Live progress reporting

The function prints progress every 500 blocks and logs to `RunLog` at the same cadence. For a 200-MiB log with $\sim$ 10 000 frequency blocks, the metadata phase typically completes in 1–3 seconds on an NVMe SSD.

23.7 Edge cases

(M1) Block at byte 0. `_read_block_at_freq_line` clamps the back-scan to `max(0, fr_offset - 300)` so it does not seek before the file start. If the block is at byte 0 there is no preceding mode- number line; the fallback list `[1, 2, ..., n]` is used.

(M2) Frequency line with NaN values. A line containing "`NaN`" is parsed as float NaN (Python's `float("nan")` accepts the string "nan"). NaN frequencies are filtered out at the mode-selection stage (section 27).

(M3) Standard block without column header. Some Gaussian versions (older) omit the `Atom AN` line and go straight from the IR-Intensities line to the eigenvector body. The fallback regex `\s+\d+ \s+\d+\s+[-\d.]` detects a first data row by pattern (Coord, Atom, Element, then numeric) and sets `is_hp = False`, `data_offset = prev_pos`.

(M4) Concatenated logs. A user who concatenates two separate Gaussian frequency-job logs (e.g. `cat job1.log job2.log > combined.log`) will have two "`and normal coordinates:`" sections. The section-aware remapping handles this correctly – global mode numbers are unique across the combined log.

24 Algorithm: cluster discovery

⇝ `geometry.find_cluster, geometry.find_all_clusters`

The cluster-discovery algorithm finds every [2Fe-2S] cluster in a molecular system using two cutoff parameters: a Fe-S bond cutoff `fe_s_cutoff` (default 3.0 Å) and an Fe-Fe distance cutoff `fe_fe_cutoff` (default 3.5 Å).

The output is a list of clusters; each cluster is a 4-tuple of Gaussian centres $(\mathrm{Fe}_1, \mathrm{Fe}_2, \mathrm{S}_1, \mathrm{S}_2)$ in canonical order.

Listing 5: Multi-cluster discovery. Implementation: `geometry.find_all_clusters`.

```
input:  atoms (list of {symbol, x, y, z, center}),
        fe_s_cutoff, fe_fe_cutoff
output: clusters : list of (fe1_c, fe2_c, s1_c, s2_c)

# Step 1: collect all Fe and S indices
fe_indices <- [i for i, a in enumerate(atoms) if a.symbol == "Fe"]
s_indices  <- [i for i, a in enumerate(atoms) if a.symbol == "S"]

# Step 2: build the Fe-Fe adjacency by distance
fe_pairs <- []
foreach (i, j) in unordered_pairs(fe_indices):
    if euclidean(atoms[i], atoms[j]) <= fe_fe_cutoff:
        fe_pairs.append((i, j))

# Step 3: for each Fe-Fe pair, find the bridging sulfurs
clusters_raw <- []
foreach (i, j) in fe_pairs:
    bridging <- []
    foreach k in s_indices:
        d_ik <- euclidean(atoms[i], atoms[k])
        d_jk <- euclidean(atoms[j], atoms[k])
        if d_ik <= fe_s_cutoff and d_jk <= fe_s_cutoff:
            bridging.append((k, d_ik + d_jk))
    if len(bridging) >= 2:
        # Pick the two closest-bridging sulfurs
        bridging.sort(by=second-element)
        s1, s2 <- bridging[0][0], bridging[1][0]
        clusters_raw.append((i, j, s1, s2))

# Step 4: canonicalize ordering (smaller Gaussian centre first
# within Fe pair and within S pair)
clusters <- []
foreach (i, j, s1, s2) in clusters_raw:
    fe1_c, fe2_c <- canonical_order(atoms[i].center, atoms[j].center)
    s1_c,  s2_c  <- canonical_order(atoms[s1].center, atoms[s2].center)
    clusters.append((fe1_c, fe2_c, s1_c, s2_c))

# Step 5: deduplicate (the same cluster may have been discovered
# via multiple Fe-Fe orderings)
clusters <- unique(clusters, key=sorted-tuple-of-all-four-centres)

# Step 6: sort by smallest Gaussian centre across the cluster
# (deterministic cluster numbering across runs)
clusters.sort(by=min-centre-in-tuple)

return clusters
```

The `strict_cluster` option. With `strict_cluster` = true, additional sanity checks are applied:

- The two bridging S atoms must form an $\mathrm{Fe}_2\mathrm{S}_2$ rhombus whose two diagonals (Fe-Fe and S-S) are within $\pm 30\%$ of the standard rhombic-cluster geometry.

- The four atoms must be approximately coplanar (smallest SVD singular value ≤ 0.1 Å).

This guards against false-positive cluster detection in systems with two adjacent [Fe(SCys)$_4$]-like centres that incidentally pass the distance-only test but are not rhombic [2Fe-2S].

Cluster ordering for multi-cluster systems. For systems with $K > 1$ clusters (e.g. a Rieske + MobisMGD enzyme where two [2Fe-2S] clusters are present), the resulting list of clusters is sorted by the smallest Gaussian centre of each tuple. This makes the cluster index reproducible: across runs, "cluster 1" refers to the same physical cluster.

24.1 Worked example: cluster discovery on a 2-cluster system

We trace the algorithm on a synthetic 2-cluster geometry with 4 Fe and 6 S atoms in the QM region:

Centre	Sym	x	y	z
12	Fe	+1.35	0	0
13	Fe	−1.35	0	0
22	Fe	+1.35	0	8.00
23	Fe	−1.35	0	8.00
14	S	0	+1.08	0
15	S	0	−1.08	0
24	S	0	+1.08	8.00
25	S	0	−1.08	8.00
99	S	+5.00	0	4.00

Step 2 – Fe-Fe pairs: All $\binom{4}{2} = 6$ pairwise Fe distances:

$$d(12,13) = 2.70, \quad d(22,23) = 2.70, \quad d(12,22) = d(13,23) = 8.00, \ldots$$

With `fe_fe_cutoff` = 3.5 Å, only the in-plane pairs $(12,13)$ and $(22,23)$ pass. The cross-cluster pairs are filtered.

Step 3 – bridging S per Fe-Fe pair: For $(12,13)$ check each S:

- S 14: $d(12,14) = \sqrt{1.35^2 + 1.08^2} = 1.73$, $d(13,14) = 1.73$. Both ≤ 3.0. ✓
- S 15: $d(12,15) = 1.73$, $d(13,15) = 1.73$. ✓
- S 24: $d(12,24) = 8.14$. Rejected.
- S 99: $d(12,99) = 5.66$. Rejected.

Bridging: $\{14,15\}$ with $d_{ik} + d_{jk} = 3.46$ each. Two closest: 14, 15. Cluster raw: $(12,13,14,15)$.

Same procedure for $(22,23)$ yields $(22,23,24,25)$.

Step 4 – canonicalize: For first cluster, Fe centres sorted: $(12,13) \to (12,13)$ (already sorted); S centres sorted: $(14,15) \to (14,15)$. Tuple: $(12,13,14,15)$.

Step 5 – deduplicate: No duplicates.

Step 6 – sort by min centre: Cluster 1 has min-centre 12, cluster 2 has min-centre 22. Final order:

$$\boxed{[(12, 13, 14, 15),\ (22, 23, 24, 25)]}.$$

S atom 99 (a spectator S not bridging any Fe-Fe) is correctly excluded.

24.2 Edge cases handled

(C1) Single isolated Fe atom. If only one Fe is present, the Fe-Fe-pair list is empty ($\binom{1}{2} = 0$). The function returns the empty list and the caller raises "no [2Fe-2S] clusters found in the geometry".

(C2) Three Fe atoms in a triangle. A linear or near-linear arrangement of 3 Fe atoms (e.g. a partial [3Fe-4S] cluster's projection) would yield 3 Fe-Fe pairs. Each pair is independently checked for bridging S. The `strict_cluster` option filters out non-rhombic results.

(C3) Fe-Fe distance just above cutoff. A cluster with $d_{\mathrm{FeFe}} = 3.51$ Å would be missed at the default cutoff. The cutoff is configurable; the user can raise it to 4.0 Å for unusually stretched clusters (e.g. super-reduced $[2\text{Fe-}2\text{S}]^0$ in some model compounds). The `strict_cluster` mode then catches false positives at the rhombicity stage.

(C4) Shared bridging sulfur between two Fe-Fe pairs. In a [4Fe-4S] cluster, the same S atom would bridge multiple Fe-Fe pairs. The algorithm produces 4 candidate "[2Fe-2S]" clusters (one per Fe-Fe pair), each sharing 2 of its S atoms with its neighbours. Step 5 (dedup by sorted-tuple) does *not* remove these because the tuples are not identical. The `strict_cluster` mode's rhombicity check filters them out (a [4Fe-4S] sub-rhombus is highly distorted from ideal). For systems containing *both* [4Fe-4S] and [2Fe-2S] clusters, the user should run with `strict_cluster` = True.

(C5) Hydrogenated μ-S. A protonated bridging sulfur (a μ-SH state, observed e.g. in some Rieske-type and ferredoxin model studies of PCET) has the same S atom, just with one more H bound. Cluster discovery is unaware of H atoms and treats this S identically to its non-protonated version. (Bridging-S protonation status is not explicitly classified by the pipeline; it manifests indirectly via the PCET-channel modulations whenever an H atom sits within bonding distance of μ-S.)

24.3 Computational complexity

Time. $O(N_{\mathrm{Fe}}^2 \cdot N_{\mathrm{S}})$ for the cluster scan; for a typical QM region with $N_{\mathrm{Fe}} \leq 4$ and $N_{\mathrm{S}} \leq 8$, this is ≤ 128 pairwise distance computations – negligible. For larger QM regions, the scan remains polynomial and well below 1% of total runtime.

Memory. $O(K)$ where K is the number of detected clusters; typically $K \leq 2$.

25 Algorithm: PDB-Gaussian matching (Kabsch alignment)

↝ `geometry.kabsch_align`

When a PDB file is supplied, each PDB atom must be matched to the Gaussian centre that represents it. The PDB atoms are usually a subset of the Gaussian atoms (a typical QM/MM setup has the [2Fe-2S]-binding pocket as QM atoms only). The matching is done in two stages: first the rotation R and translation t that optimally align the PDB heavy-atom positions onto the Gaussian heavy-atom positions are computed via Kabsch [27], then each PDB atom is mapped to its nearest Gaussian neighbour under the aligned coordinates.

25.1 Stage 1: optimal alignment

The Kabsch algorithm computes the rotation R that minimises the RMSD between two sets of N points $\{\mathbf{p}_i\}$ and $\{\mathbf{q}_i\}$:

$$R^\star \;=\; \arg\min_{R \in SO(3)} \frac{1}{N} \sum_{i=1}^{N} \|R\mathbf{p}_i - \mathbf{q}_i\|^2. \tag{77}$$

The closed-form solution uses an SVD of the cross-covariance matrix $\mathbf{H} = \tilde{\mathbf{P}}^\top \tilde{\mathbf{Q}}$ (after centring both sets):

$$\mathbf{H} = \mathbf{U}\,\mathbf{\Sigma}\,\mathbf{V}^\top, \qquad d \;=\; \operatorname{sign}(\det(\mathbf{V}\,\mathbf{U}^\top)), \qquad R^\star = \mathbf{V}\,\operatorname{diag}(1, 1, d)\,\mathbf{U}^\top. \tag{78}$$

The d factor enforces $\det(R^\star) = +1$, ruling out improper rotations (which would correspond to a chirality flip – not physical for a protein backbone).

Listing 6: Kabsch optimal-rotation algorithm. Implementation: `geometry.kabsch_align`.

```
input:  pdb_points : (N, 3) array  -- PDB heavy-atom positions
        gauss_points : (N, 3) array -- corresponding Gaussian positions
output: R : (3, 3) rotation matrix
        t : (3,)   translation vector
        rmsd : float -- post-alignment RMSD

mean_p <- mean(pdb_points, axis=0)
mean_q <- mean(gauss_points, axis=0)
P_centered <- pdb_points - mean_p
Q_centered <- gauss_points - mean_q

H <- P_centered.T @ Q_centered    # (3, 3) cross-covariance

U, S, Vt <- np.linalg.svd(H)
d <- sign(det(Vt.T @ U.T))
correction <- diag([1, 1, d])
R <- Vt.T @ correction @ U.T

t <- mean_q - R @ mean_p

# diagnostics
aligned <- (R @ pdb_points.T).T + t
rmsd <- sqrt(mean(sum((aligned - gauss_points)**2, axis=1)))
return R, t, rmsd
```

25.2 Stage 2: nearest-neighbour mapping

After alignment, each PDB atom is mapped to its nearest Gaussian neighbour, subject to the tolerance `coord_match_tol` (default 1.0 Å). Matches beyond the tolerance are reported as unmatched and excluded from the residue-label mapping.

Listing 7: Nearest-neighbour mapping post-Kabsch.

```
input:  pdb_atoms (aligned), gauss_atoms, tol
output: pdb_to_center : dict[pdb_idx -> gaussian_center] or None

pdb_to_center <- {}
foreach pdb_idx, p in enumerate(pdb_atoms):
    best_g_c    <- None
    best_dist   <- +inf
    foreach g in gauss_atoms:
        d <- euclidean(p, g)
        if d < best_dist:
            best_dist <- d
            best_g_c  <- g.center
    if best_dist <= tol:
        pdb_to_center[pdb_idx] <- best_g_c
    else:
        # log a warning, keep mapping as None
        pdb_to_center[pdb_idx] <- None

return pdb_to_center
```

Multi-cluster behaviour. For multi-cluster systems, the Kabsch alignment is performed once on the full heavy-atom set (typical proteins, where the QM region covers all clusters), not per-cluster. This is important because otherwise the alignment would over-fit to a single cluster and mislabel atoms far from it.

Match-tolerance diagnostic. The post-Kabsch RMSD is recorded in the run-log. RMSD values larger than 0.5 Å usually indicate either a wrong PDB file (different chain or different protomer of the protein), a PDB with engineered mutations, or a Gaussian geometry that diverged substantially from the crystal structure during optimisation. The user is alerted in the run-log.

25.3 The chirality-flip determinant detail

The factor $d = \text{sign}(\det(\mathbf{V}\mathbf{U}^\top))$ is critical and subtle. Without it, the SVD-based solution would sometimes return an *improper* rotation (a rotation composed with a reflection), depending on the relative chirality of the two point clouds. Such an improper rotation has $\det = -1$ and physically corresponds to mirroring the molecule.

When does $d = -1$ occur? Mathematically: when the two point clouds are so different that the optimal alignment is closer to a mirror image than to any rotation. In practice this happens in two scenarios:

1. Numerical: when the point cloud is near-coplanar and the smallest singular value of $\mathbf{H}$ is comparable to machine epsilon. The sign of the determinant becomes random under FP arithmetic.
2. Physical: the PDB and Gaussian structures genuinely have opposite chirality, e.g. if the user supplied the wrong enantiomer or if the PDB was processed through a chirality-flipping tool.

In either case, the d factor enforces $\det(R) = +1$ by flipping the sign of the last column of $\mathbf{V}$, choosing the proper-rotation branch of the optimisation. The cost is a slightly larger residual RMSD; the benefit is a physically valid rotation matrix.

25.4 Worked example: 4-atom alignment with two flips

Two synthetic point clouds:

Atom	PDB x	PDB y	PDB z	Gauss x	Gauss y	Gauss z
Fe_1	$+1.0$	0	0	0	$+1.0$	0
Fe_2	-1.0	0	0	0	-1.0	0
S_1	0	$+1.0$	0	-1.0	0	0
S_2	0	-1.0	0	$+1.0$	0	0

The PDB is the Gaussian geometry rotated by $-90°$ around the z-axis. Both centroids are at the origin.

Step 1 – cross-covariance:

$$\mathbf{H} = \tilde{\mathbf{P}}^\top \tilde{\mathbf{Q}} = \begin{pmatrix} (1)(0)+(-1)(0)+(0)(-1)+(0)(1) & \dots & \dots \\ & \dots & \dots & \dots \end{pmatrix}$$

Computing all 9 entries:

$$\mathbf{H} = \begin{pmatrix} 0 & 0 & 0 \\ -2 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Wait – this can't be right. Let me recompute more carefully for the H_{12} entry (column 2 of $\tilde{\mathbf{P}}^\top$ times column 1 of $\tilde{\mathbf{Q}}$):

$$H_{21} = \sum_i \tilde{P}_{i,2}\tilde{Q}_{i,1} = (0)(0)+(0)(0)+(1)(-1)+(-1)(1) = -2.$$

$$H_{12} = \sum_i \tilde{P}_{i,1}\tilde{Q}_{i,2} = (1)(1)+(-1)(-1)+(0)(0)+(0)(0) = 2.$$

So $\mathbf{H} = \begin{pmatrix} 0 & 2 & 0 \\ -2 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$.

Step 2 – SVD: $\mathbf{H}$ has singular values $\sigma_1 = \sigma_2 = 2$, $\sigma_3 = 0$. $\mathbf{U}$, $\mathbf{V}$ are rotation matrices that diagonalise it. A valid decomposition is

$$\mathbf{U} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{\Sigma} = \mathrm{diag}(2,2,0), \quad \mathbf{V}^\top = \begin{pmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Step 3 – determinant correction: $\det(\mathbf{V}\mathbf{U}^\top) = \det(\mathbf{V}) = +1$ (in this case), so $d = +1$ – no correction needed.

Step 4 – rotation: $R = \mathbf{V}\,\mathbf{U}^\top = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$.

This is the rotation matrix for a $+90°$ rotation about $\hat{\mathbf{z}}$, which is the inverse of the $-90°$ that produced the PDB – correct!

Step 5 – residual RMSD: After applying R to each PDB point and comparing to Gaussian: $R \cdot (1,0,0) = (0,1,0)$ ✓matches Gauss Fe_1. Similarly for the others. RMSD = 0 exactly.

Variant with $d = -1$. If we had Gauss $S_1 = (+1, 0, 0)$ and $S_2 = (-1, 0, 0)$ instead (swapped), the cross-covariance would yield $\det(\mathbf{V}\mathbf{U}^\top) = -1$. The correction would flip the sign of the last column of the SVD product, giving a proper rotation with a residual RMSD of $\sqrt{2}$ (the mirror cannot be fixed by rotation alone; the algorithm picks the best-rotation-only fit). The user would see "`coord_match_rmsd = 1.41 Å - likely wrong PDB chirality`" in the run-log.

25.5 Edge cases

(K1) Single-point or 2-point alignment. With $N < 3$ points, the alignment is geometrically underdetermined (a 2-point set has a free rotation around the joining axis). The function refuses to align with $N < 3$ and returns identity $R = \mathbf{I}, t = 0$, with a warning.

(K2) Co-linear points. With all points along a line, $\mathbf{H}$ has rank 1 and the alignment is determined only up to rotation around the line. The SVD returns one large and two small singular values. The algorithm proceeds (the resulting R is one of the infinitely many valid alignments) but the run-log warns "`coord_match singular values: 2.0, 1e-15, 1e-15`", alerting the user.

(K3) Co-planar points. With all points in a plane, σ_3 is small but $\mathbf{H}$ has rank 2; the alignment is well-determined. The chirality-flip correction d is the relevant safeguard.

(K4) Tolerance failure for one or two atoms. A single PDB atom that fails the tolerance check (e.g. because of a renaming inconsistency or because the QM optimisation moved it by > 1 Å) is reported as unmatched. The function does not abort – it continues mapping the other atoms. The user is alerted; downstream code treats the unmatched atom as having no residue label.

(K5) Many-to-one matching. After Kabsch alignment, two PDB atoms can be nearest-neighbour-matched to the same Gaussian centre if they are both within `coord_match_tol` of it. This is the "ambiguous match" warning of FUND 9. The default behaviour is to log the ambiguity and keep both mappings (the duplicate is post-filtered in the residue-label resolution stage).

26 Algorithm: secondary-structure detection

`⇝ geometry.detect_sse_dssp, geometry.detect_sse_phipsi`

Two parallel secondary-structure (SSE) classifiers are implemented. DSSP is the gold standard but requires either the executable installed locally or pre-computed records in the PDB file. As a fallback, the lightweight ϕ/ψ dihedral-only classifier is used.

26.1 Algorithm: DSSP records from PDB

`⇝ geometry.detect_sse_dssp`

DSSP [29] classifies each residue into one of 8 categories (H, B, E, G, I, T, S, blank), based on hydrogen-bond patterns in the backbone. The implementation reads the SSE records directly from the PDB header (most modern PDB files carry these in the `HELIX` and `SHEET` records):

Listing 8: DSSP-record extraction from PDB. Implementation: `geometry.detect_sse_dssp`.

```
input:  pdb_path
output: sse_per_residue : dict[(chain, resnum) -> sse_type]

sse_per_residue <- {}
with open(pdb_path) as f:
    foreach line in f:
        if line starts with "HELIX":
            chain     <- line[19]
            resnum_a  <- parse_int(line[21:25])
            resnum_b  <- parse_int(line[33:37])
            foreach r in range(resnum_a, resnum_b + 1):
                sse_per_residue[(chain, r)] <- "H"
        if line starts with "SHEET":
            chain     <- line[21]
            resnum_a  <- parse_int(line[22:26])
            resnum_b  <- parse_int(line[33:37])
            foreach r in range(resnum_a, resnum_b + 1):
                sse_per_residue[(chain, r)] <- "E"

return sse_per_residue
```

26.2 Algorithm: ϕ/ψ-only classifier

↝ `geometry.detect_sse_phipsi`

If no SSE records are in the PDB and DSSP is not installed, the fallback computes the ϕ and ψ dihedral angles per residue and applies the Ramachandran-region cutoffs:

- α-helix: $\phi \in [-100°, -30°]$, $\psi \in [-90°, +20°]$.
- β-strand: $\phi \in [-180°, -60°]$, $\psi \in [+90°, +180°]$ or $[-180°, -120°]$.
- All others: classified as "loop".

Additionally, a minimum-length filter (≥ 3 consecutive α residues, ≥ 2 consecutive β residues) is applied to avoid spurious single-residue classifications.

Listing 9: ϕ/ψ secondary-structure classifier. Implementation: `geometry.detect_sse_phipsi`.

```
input:  pdb_atoms (with backbone N, CA, C identified)
output: sse_per_residue : dict[(chain, resnum) -> sse_type]

raw_sse <- {}
foreach residue (chain, resnum) in pdb_atoms:
    prev_C  <- atom(chain, resnum - 1, "C") or skip
    this_N  <- atom(chain, resnum, "N")
    this_CA <- atom(chain, resnum, "CA")
    this_C  <- atom(chain, resnum, "C")
    next_N  <- atom(chain, resnum + 1, "N") or skip
    phi <- dihedral(prev_C, this_N, this_CA, this_C)
    psi <- dihedral(this_N, this_CA, this_C, next_N)
    raw_sse[(chain, resnum)] <- classify(phi, psi)

# Apply min-length filter: drop isolated SSE-classified residues
sse_per_residue <- min_length_filter(raw_sse, alpha_min=3, beta_min=2)
return sse_per_residue
```

Comparison of the two classifiers. DSSP is more accurate (especially for distinguishing α-helices from 3_{10}- and π-helices), but the ϕ/ψ classifier captures the majority of α- and β-character correctly for typical > 30-residue chains. Both classifiers feed the same downstream SSE-amplitude analysis; the choice is logged.

26.3 The fallback cascade

The full SSE-detection pipeline tries three sources in order, stopping at the first that produces a non-empty result:

1. **Pre-computed records in the PDB header.** If the PDB contains `HELIX` and/or `SHEET` records, these are parsed directly. This is the fastest path (no computation, no external dependency). Most PDBs deposited in the RCSB Protein Data Bank since the 1990s carry these records.
2. **External DSSP executable.** If the `dssp_executable` path is set and the binary exists and is runnable, DSSP is invoked as a subprocess. The DSSP output (an 8-class assignment) is parsed; H, G, I are merged into "α"; E, B are merged into "β"; the rest are "loop". Failure modes (non-zero exit code, empty stdout, missing residues) trigger the next fallback.
3. **ϕ/ψ classifier on PDB coordinates.** The in-package fallback, requiring only the backbone N, CA, C atoms (which are always present in a standard PDB).

The chosen source is recorded in the run-log as "`sse_source = pdb_records`", "`sse_source = dssp_executable`", or "`sse_source = phipsi_fallback`".

26.4 Edge cases

(S1) PDB with no SSE records and no DSSP. The ϕ/ψ classifier is used. Some residues at the chain termini (missing ϕ at residue 1, missing ψ at the last residue) get classified as "loop" regardless of their actual backbone geometry. This is acceptable because terminal residues are typically already disordered.

(S2) Cis-proline. A cis-Pro at $\omega \approx 0$ has ψ shifted into the "α-like" region; the classifier may mislabel it. This is a known limitation; the DSSP-records source is more accurate.

(S3) Glycine. Gly residues lack a side chain and occupy a larger Ramachandran region than other amino acids. The default cutoffs do not have a Gly-specific branch; Gly residues in the "left-handed α" region (positive ϕ) are classified as loop, which is usually correct.

(S4) Missing backbone atoms. If the PDB is missing a backbone N, CA, or C atom for some residue (e.g. a CA-only minimal PDB), the ϕ/ψ calculation fails for that residue and adjacent residues. They are classified as "unknown" (UNK in the output) and the user is warned.

(S5) Min-length filter removing real short helices. The 3-residue α-min and 2-residue β-min filter deliberately removes singletons. Real 3_{10}-helix turns and β-bulges may be lost. The filter is necessary because the ϕ/ψ classifier produces $\sim$ 10–15% spurious singletons. The user can adjust the filter parameters via `sse_alpha_min_len` and `sse_beta_min_len`.

(S6) Multiple chains. The classifier processes one chain at a time. Inter-chain H-bonds (which DSSP would detect) are not visible to the ϕ/ψ fallback, so cross-chain β-sheets are classified residue-by-residue as if each chain were independent. For most NRVS analyses this is acceptable (the cluster is in one chain).

26.5 Connection to the SSE-amplitude analysis

The output `sse_per_residue` dict feeds two downstream consumers:

1. `export_sse_amplitude_groups`: builds per-mode SSE-aggregated amplitude sheets. Residues classified "α" are summed into the α-amplitude column; "β" into the β-amplitude column; "loop" into the loop column. The sum-of-three equals the total protein amplitude (except for unclassified residues).
2. `geometry.build_sse_center_map`: enriches the per-atom residue labels with their SSE class for the audit log.

For both, an unclassified residue (UNK) is silently dropped from the SSE-sum but kept in the total – the resulting "$\alpha + \beta + \text{loop} < \text{total}$" inequality in the workbook signals to the user that some residues lacked SSE classification.

27 Algorithm: main mode-analysis loop (Phase 4)

↝ `runner._run_analysis_single`

The Phase 4 main loop is the most computationally expensive part of the pipeline. For each mode that passes the frequency-window filter, the loop:

1. streams the mode's eigenvector from the log (via `logio.get_eigvec`),
2. thermally scales it via $u_{\text{rms}}(\omega_l)$,
3. runs the full per-mode analysis chain (OOP/INP, SCSD, kernel-class, all bond-modulations, B-factor accumulation),
4. collects the result-dict and appends it to the master list.

Listing 10: Main mode-analysis loop. Implementation: `runner._run_analysis_single`, inner call: `core.analyze_mode_with_fallback`.

```
input:  cfg, atoms, idx_map, all_blocks, best_block, cand_map,
        cluster_normal, fe_c, s_c, coord_info
output: results : list of per-mode dicts
        b_accum : (N_atoms,) array of B-factor contributions

# Step 1: build the selection list
selected <- []
foreach bi in all_blocks:
    foreach col, (mn, freq) in enumerate(zip(bi.mode_nums, bi.freqs)):
        # only count the preferred block for each mode number
        if best_block[mn] is not bi:
            continue
        # skip imaginary/zero modes
        if freq <= 0:
            log "skipping mode {mn} with freq={freq} cm^-1"
```

```
            continue
        # frequency-window filter
        if cfg.freq_min and freq < cfg.freq_min: continue
        if cfg.freq_max and freq > cfg.freq_max: continue
        if cfg.freq_windows is not None
                and not any(lo <= freq <= hi for (lo, hi) in cfg.freq_windows):
            continue
        selected.append((bi, col, mn, freq))

selected.sort(by=freq)
results  <- []
b_accum  <- zeros(N_atoms)

# Step 2: per-mode analysis
foreach (bi, col, mn, freq) in selected:
    # eigenvector loading with fallback to alternative blocks
    r, used_block <- analyze_mode_with_fallback(
        candidates  = cand_map[mn],
        col         = col,
        log_file    = cfg.log_file,
        atoms       = atoms,
        idx_map     = idx_map,
        normal      = cluster_normal,
        coord_info  = coord_info,
        fe_c        = fe_c,
        s_c         = s_c,
        cfg         = cfg,
        mode_num    = mn,
    )
    if r is None:
        # all blocks failed; log and skip
        log "failed to read eigenvector for mode {mn}, all blocks bad"
        continue

    # Accumulate B-factor: r["evg"] is the thermally scaled eigenvector
    foreach i in range(N_atoms):
        b_accum[i] += sum(r["evg"][i]**2)   # (e_x*u + e_y*u + e_z*u)^2

    results.append(r)

return results, b_accum
```

Inner helper: `analyze_mode_with_fallback`. ⇝ core.analyze_mode_with_fallback

This helper wraps `logio.get_eigvec` with a fallback loop:

Listing 11: Eigenvector loading with fallback. Implementation: `core.analyze_mode_with_fallback`.

```
input:  candidates : list of BlockInfo, col : int, ...
output: result_dict, used_block

foreach bi in candidates_sorted_by_preference (HP first):
    try:
        evec_raw <- logio.get_eigvec(bi, col, ...)
            # raw cartesian-displacement eigenvector, shape (N_atoms, 3)
            # not yet thermally scaled
    except (CorruptDataError, ParseError):
        continue
```

```
    # Thermal scaling: each component of evec_raw becomes A x u_rms(omega_l)
    u_rms <- core.compute_thermal_amplitude(freq, red_mass, cfg.temp_k, cfg.amplitude)
    evg   <- evec_raw * u_rms        # thermally scaled, in Angstrom

    # Run all per-mode analyses
    result_dict <- {
        "number":    mode_num,
        "freq":      freq,
        "red_mass":  red_mass,
        "evec_raw":  evec_raw,
        "evg":       evg,
        ...,
    }
    result_dict.update(core.classify_oop_inp(evg, normal, fe_c, s_c, cfg))
    result_dict.update(core.analyze_fe_ligand(evg, normal, coord_info, cfg))
    result_dict.update(core.analyze_his_hn(evg, coord_info, cfg))
    result_dict.update(core.compute_scsd_for_mode_full(evec_raw, u_rms, atoms, fe_c, s_c,
        cfg))
    result_dict.update(reorganization.compute_mode_modulations(evec_raw, u_rms, freq,
        coord_info, cfg))

    return result_dict, bi

# All candidates failed
return None, None
```

27.1 The candidate ordering

`cand_map[mn]` contains all blocks whose mode-number list includes `mn`. The fallback iterates them in this order:

1. **Preferred block** (`best_block[mn]`): typically the HP block if any, otherwise the first standard block seen during scan.
2. **HP blocks not chosen as preferred**: rare, but happens if two HP blocks have overlapping mode numbers (e.g. a corrupted log with duplicated sections).
3. **Remaining standard blocks**: lowest precision, used only if the HP blocks all failed.

This ordering is implemented by sorting `cand_map[mn]` with `key=(not bi.is_hp, bi.offset)` (`not bi.is_hp` is False for HP blocks, True for standard, putting HP first).

27.2 What can cause a block to fail

`logio.get_eigvec(bi, col, ...)` can raise three exception types:

- `CorruptDataError`: the block's eigenvector body ends before $3N$ rows are read (file truncated, or the block format violates expectations).
- `ParseError`: a row contains non-numeric data where floats are expected (e.g. "`********`" overflow markers in the unlikely case they appear in eigenvector columns).

- `IndexError`: the requested `col` is beyond the actual number of mode columns in the block (Gaussian sometimes writes fewer columns in the last block of a section if the total mode count is not a multiple of 5).

The fallback catches all three and tries the next candidate. If all candidates fail, the mode is logged as "`failed to read eigenvector for mode {mn}, all blocks bad`" and skipped. The final results list excludes failed modes (FUND-13 synthetic-zero modes are added separately).

27.3 The B-factor accumulator

The `b_accum` array is filled in-place during the loop. For each mode l and each atom i, the per-mode contribution is

$$\Delta b_i^{(l)} = \sum_{c \in \{x,y,z\}} \left(\mathbf{e}_{l,i,c} \cdot u_{\mathrm{rms}}(\omega_l)\right)^2 = u_{\mathrm{rms}}^2 \cdot \|\mathbf{e}_{l,i}\|^2.$$

This is the per-atom contribution before the $8\pi^2/3$ scaling (applied at the export stage, section 17).

Because the loop runs over modes that survived the frequency-window filter, the resulting B-factors are window-restricted (see section 17 for the implications).

27.4 Computational complexity

Time per mode. $O(N_{\mathrm{atoms}})$: one pass over the eigenvector for OOP/INP, one for SCSD, one each for the per-channel bond modulations. Total per-mode time scales linearly with the QM region size.

Time per log. $O(N_{\mathrm{modes}} \cdot N_{\mathrm{atoms}})$. For a typical [2Fe-2S] QM/MM system (here ∼1260 modes, ∼422 QM atoms): ∼ 532k atom-operations per pass, ∼ 3 minutes wall-clock on a 2024-era laptop CPU.

Memory. $O(N_{\mathrm{modes}} \cdot N_{\mathrm{atoms}})$: the results list holds one dict per mode containing the eigenvector, plus per-mode observables. For the same example system: ∼ 50 MiB.

27.5 Edge cases

(L1) Imaginary frequencies. Gaussian writes imaginary frequencies as negative numbers (e.g. −23.45). The filter `if freq <= 0: continue` drops these. The user is alerted once with "`N imaginary modes skipped`".

(L2) Translation/rotation modes. The 6 lowest modes of a free molecule are translations and rotations with $\omega \approx 0$. For QM/MM-optimised structures with the QM region embedded in the MM region, these modes are partially constrained and have small but non-zero frequencies. They are not filtered out by the $\omega \leq 0$ check; if they fall in the user's frequency window, they will be analysed (typically yielding mostly skeletal displacement with little spectroscopic significance).

(L3) Multi-window scenarios. If `freq_windows` is a list of (lo, hi) tuples instead of a single window, a mode passes if it falls in *any* window. The export stage produces one workbook per window.

(L4) Mode-number collisions across HP and standard blocks. If both an HP and a standard block claim mode 100, `best_block` stores the HP version; the loop processes the HP version. The standard version is available via `cand_map` as a fallback.

28 Algorithm: context-mode loading + FUND 12 + FUND 13

↝ `runner._run_analysis_single, runner._make_synthetic_zero_mode`

After the main loop has produced `results`, the interpolation workbook (`*_analysis_interp##.xlsx`) requires that the piecewise-linear interpolation of each per-mode observable has well-defined anchors at the lower and upper edges of the frequency window. Without anchors, the interpolation produces zero outside the range of the modes inside the window – which is generally not the true spectral behaviour just above/below the window.

The remedy is to load a small number of *context modes* just outside the window. Three loading scenarios are distinguished:

Scenario A: at least one real mode within `interp_context_cm1` of the boundary. Load every such mode normally (`analyze_mode_with_fallback`). This is the standard case for densely sampled DFT spectra.

Scenario B (FUND 12): no real mode within `interp_context_cm1`, but real modes exist beyond. The nearest real mode beyond `interp_context_cm1` is loaded as a single "boundary anchor". This guards against the failure mode where the DFT spectrum happens to be sparse exactly at the window boundary, which would leave the interpolation unanchored.

Scenario C (FUND 13): no real mode above the upper window boundary at all. A synthetic null-mode is inserted at $\omega_{\text{anchor}} = \texttt{freq\_max} + \texttt{interp\_context\_cm1}$ with all observables set to zero. This is the case when the analysis window includes the highest frequency in the spectrum, and the natural right-hand boundary of the spectrum is being asked to serve as the interpolation's right boundary.

Listing 12: Context-mode loading with FUND 12 + FUND 13 fallbacks. Implementation: `runner._run_analysis_single`, helper `runner._make_synthetic_zero_mode`.

```
input:  cfg, all_blocks, best_block, cand_map, ..., real_results
output: context_results_left, context_results_right

# RIGHT-EDGE (upper-frequency) CONTEXT
right_window_lo <- cfg.freq_max
right_window_hi <- cfg.freq_max + cfg.interp_context_cm1
context_right_modes <- [m for m in all_modes if right_window_lo < m.freq <= right_window_hi
    ]

if len(context_right_modes) > 0:
    # Scenario A
    foreach m in context_right_modes:
        load_and_analyze(m); append to context_results_right
else:
    # Look for a single fallback above interp_context_cm1
    fallback <- min((m for m in all_modes if m.freq > right_window_hi), key=lambda m: m.
        freq, default=None)
```

```
    if fallback is not None:
        # Scenario B -- FUND 12
        load_and_analyze(fallback); append to context_results_right
        log "FUND-12 fallback: loaded mode {fallback.number} at {fallback.freq:.1f} cm^-1
            as right boundary anchor (no mode within +{cfg.interp_context_cm1} cm^-1)"
    else:
        # Scenario C -- FUND 13
        synth <- _make_synthetic_zero_mode(omega_anchor=right_window_hi, atoms=atoms, ...)
        context_results_right.append(synth)
        log "FUND-13 fallback: inserted synthetic zero anchor at {right_window_hi:.1f} cm
            ^-1 (no real mode above {cfg.freq_max})"

# LEFT-EDGE (lower-frequency) CONTEXT  -- only Scenarios A and B
left_window_lo <- cfg.freq_min - cfg.interp_context_cm1
left_window_hi <- cfg.freq_min
context_left_modes <- [m for m in all_modes if left_window_lo <= m.freq < left_window_hi]

if len(context_left_modes) > 0:
    # Scenario A
    foreach m in context_left_modes:
        load_and_analyze(m); append to context_results_left
else:
    fallback <- max((m for m in all_modes if m.freq < left_window_lo), key=lambda m: m.freq
        , default=None)
    if fallback is not None:
        # Scenario B -- FUND 12
        load_and_analyze(fallback); append to context_results_left
        log "FUND-12 fallback: loaded mode {fallback.number} at {fallback.freq:.1f} cm^-1
            as left boundary anchor"
    # else: no left synthetic anchor -- interpolation's natural
    #       left=0 boundary is already physically correct

return context_results_left, context_results_right
```

Why no synthetic left anchor. The interpolation at the lower edge naturally extrapolates to $\Delta r = 0$ (zero observables at zero frequency), because every spectroscopic observable in the code vanishes at $\omega \to 0$ (the modulation is the displacement, which vanishes for a translation/rotation pseudomode). There is therefore no physically wrong behaviour at the lower edge that requires a synthetic anchor; only the upper edge benefits from one.

Synthetic zero mode shape. The synthetic mode returned by `_make_synthetic_zero_mode` has the same dictionary shape as a real mode's result, with all observable values set to zero and sentinel metadata `number` $= -1$, `mode_type` $=$ `"synthetic_zero"`, `precision` $=$ `"synthetic"`. The eigenvector matrix `evg` and the SCSD coefficient list are deliberately *omitted* (set to `None`) so that the downstream B-factor and SSE-amplitude loops detect the sentinel and skip the mode – it must not contribute to system-wide aggregates, only to the interpolation anchor.

28.1 The decision order in detail

When the main loop finishes and the context-loading phase begins, the implementation processes the right-edge first. The complete decision tree:

1. Build `candidates_right = [m for m in all_modes if cfg.freq_max < m.freq <= cfg.freq_max + cfg.interp_context_cm1]`. If non-empty, this is Scenario A; analyse each candidate and store as `context_results_right`. *Done.*
2. Otherwise build `fallback_right = min(all_modes with freq > cfg.freq_max + cfg.interp_context_cm1, key = freq)`. If not None, this is Scenario B (FUND 12); load and analyse the single fallback mode; log the FUND-12 fallback message. *Done.*
3. Otherwise this is Scenario C (FUND 13); call `_make_synthetic_zero_mode` at $\omega_{\text{anchor}} =$ `freq_max` + `interp_context_cm1`; log the FUND-13 fallback message. *Done.*

The left-edge processing is identical except for the missing Scenario C (no left synthetic anchor).

The full FUND 12 + FUND 13 logic is exercised by three E2E regression tests (`test_e2e_terminal_spectrum_fund_13`, `test_synthetic_zero_mode_has_zero_observables`, `test_synthetic_zero_mode_marked_as_synthetic`; section 82).

28.2 Edge cases

(X1) Window includes the full DFT spectrum. If `freq_min` $= 0$ and `freq_max` $= \omega_{\text{max}}^{\text{DFT}}$, both edges are at the natural spectrum boundaries. Right-edge: Scenario C (synthetic). Left-edge: empty (no synthetic; the natural left boundary is correct).

(X2) Window of zero width. If `freq_min` $=$ `freq_max` (a degenerate window), the main loop produces no real modes. The context loader still runs and may produce 1–2 anchors. The resulting interpolation is constant zero. This is documented but not strictly disallowed.

(X3) Negative `interp_context_cm1`. The context-window construction `[freq_max, freq_max + interp_context_cm1` becomes inverted (`freq_max` $>$ endpoint). The list of context candidates is then empty. The fallback Scenario B/C is triggered. A user who set `interp_context_cm1` $= 0$ will reliably land in Scenario B (FUND 12) for any spectrum with modes above `freq_max`, and in Scenario C for a window that includes the spectrum's maximum.

(X4) Multiple clusters, multiple windows. The context-loading is per-cluster, per-window. Each per-cluster, per-window result has its own synthetic anchor if needed; the sentinels do not bleed across clusters or windows.

29 Algorithm: multi-window loop

↝ `runner.run_analysis`

When the user supplies `freq_windows` (a list of disjoint [lo, hi] intervals), Phase 4 is invoked once per window, producing separate output workbooks (and separate `Reorganisation` sheets). Each window's Λ_X^{total} is independent.

Listing 13: Multi-window dispatching. Implementation: `runner.run_analysis`.

```
input:  cfg
output: dict[window -> workbook paths]
```

```
if cfg.freq_windows is None:
    # Single implicit window from (freq_min, freq_max)
    return _run_analysis_single(cfg)

results_per_window <- {}
foreach (lo, hi) in cfg.freq_windows:
    cfg_sub <- replace(cfg, freq_min=lo, freq_max=hi, freq_windows=None)
    cfg_sub.logname_suffix <- f"_{lo:.0f}_{hi:.0f}_cm"
    results_per_window[(lo, hi)] <- _run_analysis_single(cfg_sub)
return results_per_window
```

Window-boundary convention. The implementation uses *half-open* intervals $[\texttt{lo}, \texttt{hi})$: a mode at exactly $\omega = \texttt{hi}$ is *not* included in the window, but *is* included in the window that starts at `hi`. This avoids double-counting when windows are contiguous (e.g. $[0, 100), [100, 300)$). The convention is documented in the manual and verified by `tests/test_v104_audit_fixes.py::test_freq_window_half_open`.

Cache reuse across windows. Because the byte-offset scan and the `read_all_meta` structures are window-independent, the cache (section 2.8) is shared between window invocations: only the first window pays the disk-scan cost.

29.1 Edge cases for the multi-window loop

(W1) Overlapping windows. The half-open convention only prevents double-counting at adjacent window boundaries ($\texttt{hi}_1 = \texttt{lo}_2$). If the user supplies overlapping windows ($\texttt{lo}_2 < \texttt{hi}_1$), modes in the overlap appear in *both* output workbooks. This is intentional – sometimes a user wants two overlapping windows to compare (Λ^{total} in $[0, 500]$ vs. $[0, 300]$ for example). A warning "`windows {w1} and {w2} overlap`" is logged once.

(W2) Window outside the spectrum. A window entirely above $\omega_{\max}^{\text{DFT}}$ produces zero real modes. The synthetic-zero anchor handles the right boundary; the left boundary triggers Scenario B/C from the previous section. The output workbook is well-defined but contains only the anchor's zero observables.

(W3) Negative or zero-width windows. A window with $\texttt{lo} \geq \texttt{hi}$ is rejected at config-validation time; the run aborts.

(W4) Five or more windows. The loop scales linearly with the number of windows. The byte- offset scan is reused; only the per-mode analysis (Phase 4) is repeated. For 5 windows on a 1260-mode spectrum, the total runtime is ~ 15 minutes instead of ~ 3.

30 Algorithm: multi-cluster loop

↝ `runner.run_analysis`

When `analyze_all_clusters` = true, the pipeline iterates over every cluster found by `find_all_clusters`. Each cluster produces an independent output directory (suffix `_cluster##`) with its own set of workbooks.

Inter-cluster aggregation is not currently performed.

Listing 14: Multi-cluster dispatching. Implementation: `runner.run_analysis`.

```
input:  cfg
output: dict[cluster_index -> window-result-dict]

clusters <- find_all_clusters(atoms, cfg.fe_s_cutoff, cfg.fe_fe_cutoff,
                              strict=cfg.strict_cluster)
if not cfg.analyze_all_clusters:
    cluster_subset <- [clusters[cfg.cluster_index]]
else:
    cluster_subset <- clusters

results_per_cluster <- {}
foreach (idx, cluster) in enumerate(cluster_subset):
    fe1_c, fe2_c, s1_c, s2_c <- cluster
    cfg_sub <- replace(cfg,
        output_dir = cfg.output_dir + f"_cluster{idx:02d}",
    )
    # Reset per-cluster warning state so each cluster gets a clean
    # set of WARNED_EMPTY_GROUP / WARNED_EMPTY_LIG / WARNED_HN_SKIP
    core.reset_warning_state()
    results_per_cluster[idx] <- _run_analysis_single(
        cfg_sub, fe_c=(fe1_c, fe2_c), s_c=(s1_c, s2_c), ...)
return results_per_cluster
```

Per-cluster state reset. The deduplication sets `_WARNED_EMPTY_GROUP`, `_WARNED_EMPTY_LIG`, `_WARNED_HN_SKIP` in `core.py` are module-level and used to suppress duplicate `UserWarning` emissions during the inner per-mode analyses. They must be reset at the start of each cluster run, otherwise a warning that fired during cluster 1's analysis would silence the corresponding warning in cluster 2 – which would be misleading. `core.reset_warning_state` performs this reset; it is invoked unconditionally at the start of every cluster, regardless of whether the system actually has multiple clusters.

30.1 Per-cluster output naming

Each cluster's output is placed in a sibling directory of the main output directory, with suffix `_cluster##` where ## is the zero-padded cluster index (`_cluster00`, `_cluster01`, ...). The cluster index follows the canonical ordering of `find_all_clusters` (sorted by smallest Gaussian centre).

30.2 Edge cases

(M1) Single cluster with `analyze_all_clusters`. Setting `analyze_all_clusters` = True when only one cluster exists is equivalent to setting it False with `cluster_index` = 0, except that the output directory suffix is added. This is documented; the resulting outputs are identical apart from the directory name.

(M2) No clusters found. `find_all_clusters` returns an empty list. The runner raises `NoClustersFoundError` with a clear message and exits without producing any workbook.

(M3) Cluster index out of range. `cluster_index` larger than `len(clusters) - 1` raises `IndexError` at the cluster-subset construction step. The error message includes the actual number of clusters detected.

(M4) Cluster-shared atoms. A user may construct a system where one S atom is shared between two "adjacent" [2Fe-2S] clusters (e.g. a [3Fe-4S]-like arrangement projected through cluster discovery). The shared atom appears in both per-cluster outputs; the per-cluster Λ^{total} values double-count it. The strict-cluster mode of `find_all_clusters` (section 24) filters this case at the discovery stage.

Part IV

Data Structures

This part enumerates every non-trivial data structure passed between the phases of section 2. Every field of every dataclass is documented: its type, what it represents, and which downstream code consumes it. A reviewer who has read this part should be able to write a test that constructs a synthetic instance of any data structure and feeds it into the consuming function.

31 Dataclass: `BlockInfo`

↝ `logio.scan_log`

Defined in `logio.py`. Carries the per-frequency-block metadata read by `read_all_meta` from each `Frequencies --` section in the Gaussian log.

Field	Type	Meaning
`offset`	`int`	Byte offset of the `Frequencies --` header. Used to `seek()` the file before reading the block.
`mode_nums`	`List[int]`	Mode numbers contained in this block (3 modes per row for standard blocks, 5 modes per row for `hpmodes` blocks).
`freqs`	`List[float]`	Frequencies of the modes in cm^{-1}. Same length as `mode_nums`.
`red_masses`	`List[float]`	Reduced masses in amu. NaN for modes where Gaussian printed `********` (overflow).
`frc_consts`	`List[float]`	Force constants in mDyne/Å. Currently unused.
`syms`	`List[str]`	Irreducible-representation labels (`A`, `A'`, A_g, etc.). Used by the kernel-classification scoring.
`is_hp`	`bool`	True for `hpmodes` blocks (5-decimal precision), False for standard 3-decimal blocks. Used by `best_block` to prefer HP over standard for any mode that appears in both.
`data_offset`	`int`	Byte offset of the first eigenvector data row in this block. Used by `get_eigvec` to stream the eigenvector matrix without re-parsing the header.

32 Dataclass: `LigandInfo`

⇝ `geometry.find_coordinating_residues`

Defined in `geometry.py`. Carries per-ligand coordination information for each Fe-ligand bond.

Field	Type	Meaning
`fe_idx`	`int`	Index (0 or 1) of which Fe in the cluster this ligand binds.
`fe_center`	`int`	Gaussian centre of that Fe atom.
`lig_center`	`int`	Gaussian centre of the ligand donor atom.
`lig_element`	`str`	Element symbol of the donor atom ("N", "S", or "O").
`lig_aname`	`str`	PDB atom name of the donor (`SG`, `ND1`, `NE2`, etc.).
`res_num`	`int`	PDB residue number of the donor.
`res_name`	`str`	PDB residue name (`CYS`, `HIS`, `ASP`, `GLU`, etc.).
`res_label`	`str`	Human-readable label (e.g. `Cys11`, `His87`). Used as the column-key in the output spreadsheets.
`bond_vec`	`np.ndarray`	3-vector $\mathbf{r}_L - \mathbf{r}_{\mathrm{Fe}}$. Used for the Fe-ligand stretch/bend axis.
`bond_len`	`float`	$\|$`bond_vec`$\|$ in Å.
`h_center`	`Optional[int]`	For histidines: Gaussian centre of the protonated-N-H proton, if any. `None` for non-histidines or unprotonated histidines.
`hn_vec`	`Optional[np.ndarray]`	N-H bond vector $\mathbf{r}_H - \mathbf{r}_N$.
`hn_len`	`Optional[float]`	N-H bond length in Å.
`his_protonated`	`bool`	True if this is a protonated histidine ligand. Drives the inclusion of this ligand in the His-NH and HA channels.
`his_hn_center`	`Optional[int]`	Synonym for `h_center`; kept for backward compatibility with pre-v1.0.2 result dicts.
`coord_n_aname`	`Optional[str]`	For histidines: which side-chain N coordinates Fe (`ND1` or `NE2`).

33 Dataclass: `CoordInfo`

⇝ `geometry.find_coordinating_residues`

Defined in `geometry.py`. The master container for all coordination information that the per-mode analysis needs. Built once per cluster.

Field	Type	Meaning

ligands	List[LigandInfo]	Every Fe-coordinating ligand, one entry per Fe-L bond. Length is typically 4 (Cys_4 cluster), 4 (Cys_2His_2 Rieske, two Cys_2 + two His_2), or 5 (mixed-coordination systems).
group_map	Dict[str, List[int]]	Pre-defined or auto-generated atom groups, key = group name, value = list of Gaussian centres in the group. Used by the per-group OOP/INP analysis. Default groups: Cluster, ProteinBackbone, one Res_### per residue.
pdb_to_center	Dict[int, int]	Mapping from PDB atom index to Gaussian centre, populated by the Kabsch matching of section 25.
his_ligand_labels	List[str]	Convenience list of res_label strings for all protonated histidines. Drives the order of columns in the His-NH and HA output sheets.
pcet_info	Optional[object]	Carries the PCET acceptor candidates per protonated histidine (built by reorganization.build_channels_from_coord_info). Per-mode HA analysis reads this.
et_info	Optional[object]	Carries the FeFe channel geometry (one per pair of Fe atoms in the cluster).
atoms_h	Optional[List[Dict]]	Hydrogen-inclusive atom list. None when include_hn_vibration is False.
idx_map_h	Optional[Dict[int, int]]	Gaussian-centre → index in atoms_h.

34 Dataclass: ChannelGeometry

⇝ reorganization.build_channels_from_coord_info

Defined in reorganization.py. Carries the geometric input to the per-mode reorganization calculation for one bond sub-channel.

Field	Type	Meaning
name	str	Unique sub-channel name (e.g. FeS_Fe1_S1, FeN_His87, HA_His87_Asp42-OD1). Used as column-key.
idx_a	int	Position of atom a in the heavy-atom (or all-atom) array.
idx_b	int	Position of atom b.
r_a	np.ndarray	Equilibrium position of atom a, 3-vector in Å.
r_b	np.ndarray	Equilibrium position of atom b.
elem_a	str	Element symbol of atom a (used for μ_{pair}).
elem_b	str	Element symbol of atom b.
mu_pair_amu	float	Reduced mass $\mu = m_a m_b/(m_a + m_b)$ in amu. Used directly in lambda_pair_cm1.

`parent_channel`	`str`	Which of the five parent channels this sub-channel feeds: `FeFe`, `FeN`, `FeS`, `NH`, or `HA`.
`weight`	`float`	Aggregation weight w_s for the RSS aggregation (section 12). For HA channels this is the gaussian-distance weight; for all others $w_s = 1$.
`idx_donor`	`Optional[int]`	For HA channels: index of the donor N_{His} atom. Used by equation (22) as the displacement subtraction reference.
`r_donor`	`Optional[np.ndarray]`	Equilibrium position of the donor; defines the $\hat{\mathbf{n}}_{NA}$ projection axis.

35 Dataclass: `ChannelResult`

↝ `reorganization.compute_mode_modulations`

Defined in `reorganization.py`. Carries the per-mode, per-sub-channel result of the reorganization calculation, one instance per (mode, sub-channel) pair.

Field	Type	Meaning
`name`	`str`	Sub-channel name, copied from the corresponding `ChannelGeometry`.
`parent_channel`	`str`	One of FeFe, FeN, FeS, NH, HA.
`dr_signed_a`	`float`	Signed bond modulation $\Delta r_X^{(s)}$ in Å, computed by `signed_dr_along_axis` (classical formula equation (21)) or `compute_mode_modulations` (reaction-coordinate formula equation (22) for HA).
`lambda_pair_cm1`	`float`	Per-sub-channel reorganization energy in cm^{-1}, computed by `lambda_pair_cm1` equation (35).
`lambda_mode_cm1`	`float`	Per-mode aggregate $\lambda_X(\text{mode})$ via equation (43). Only set on the parent-aggregated instance, NaN on sub-channel instances.
`weight`	`float`	Aggregation weight w_s for this sub-channel.

36 Per-mode result dictionary

↝ `core.analyze_mode_with_fallback`

The per-mode result dictionary is the central data structure of the pipeline – one dict per mode in the frequency window, returned by `analyze_mode_with_fallback` and accumulated by `_run_analysis_single`. Every Excel column in every output workbook traces back to one of these keys. This section enumerates every key.

36.1 Identification and frequency metadata

Key	Type	Meaning
`number`	`int`	Gaussian mode number (1-based). Sentinel -1 for synthetic zero modes (FUND 13).
`freq`	`float`	Frequency in cm^{-1}.
`red_mass`	`float`	Reduced mass in amu.
`frc_const`	`float`	Force constant in mDyne/Å.
`symmetry`	`str`	Irrep label as printed by Gaussian.
`precision`	`str`	One of `"hp"`, `"std"`, `"synthetic"`.
`u_rms_a`	`float`	Thermal RMS amplitude $u_{\mathrm{rms}}(\omega_l)$ in Å, computed by equation (8).

36.2 Eigenvector and per-atom contributions

Key	Type	Meaning
`evec_raw`	`np.ndarray`	Raw (un-thermally-scaled) eigenvector, shape $(N_{\mathrm{atoms}}, 3)$.
`evg`	`np.ndarray`	Thermally scaled eigenvector $\widetilde{\mathbf{e}} = \mathbf{e} \cdot u_{\mathrm{rms}}$, shape $(N_{\mathrm{atoms}}, 3)$, in Å. Used by all downstream geometric observables.

36.3 Cluster-core OOP/INP descriptors

Every key uses the OOP/INP decomposition of section 5.

Key	Type	Meaning
`a_oop_pct`	`float`	Cluster-core α_{OOP} in %.
`a_inp_pct`	`float`	$\alpha_{\mathrm{INP}} = 100 - \alpha_{\mathrm{OOP}}$.
`mode_type`	`str`	Binary classification (OOP / INP / mixed), per equation (14).
`mode_type_detail`	`str`	Eight-bin classification (pure_oop, strong_oop, etc.).
`ca_displacement_a`	`float`	Cluster centre-of-mass displacement $\lVert\frac{1}{4}\sum_{i\in\mathrm{cluster}}\widetilde{\mathbf{e}}_i\rVert$ in Å. Diagnoses translational contamination.
`ca_oop_a`	`float`	Out-of-plane component of the cluster-COM displacement.
`cluster_expansion_a`	`float`	Mean radial cluster expansion (Å). Positive = breathing outward; negative = contracting.
`cluster_rotation_deg`	`float`	In-plane rotation angle of the rhombic Fe_2S_2 unit, in degrees.

36.4 Fe-ligand bond descriptors

For each entry in `coord_info.ligands` the following keys are created (with the ligand's `res_label` as the column suffix in the output sheet – e.g. `stretch_Cys11`).

Key	Type	Meaning
`stretch`	`float`	Fe-L stretch component $\Delta r_{\text{stretch}}$ (equation (25)) in Å.
`bend_inp`	`float`	In-plane bend component, ≥ 0.
`bend_oop`	`float`	Signed out-of-plane bend component.
`bend_inp_pct`	`float`	Percentage of total bend power in plane.
`bend_oop_pct`	`float`	Percentage of total bend power out of plane.
`s_stretch`	`float`	Propagated sigma for `stretch` ($\sqrt{2}\sigma_e u_{\text{rms}}$, equation (31)).
`s_bend_inp`	`float`	Propagated sigma for `bend_inp`.
`s_bend_oop`	`float`	Propagated sigma for `bend_oop`.

36.5 His-NH and HA descriptors

Key	Type	Meaning
`hn_stretch_`*`Hisn`*	`float`	N-H stretch modulation for each protonated histidine, in Å.
`s_hn_stretch_`*`Hisn`*	`float`	Propagated sigma for `hn_stretch`; v1.0.4 fix (section 69) applies the u_{rms} factor.
`ha_dr_`*`Hisn`*`_to_`*`acceptor`*	`float`	Reaction-coordinate HA modulation (equation (22)), in Å.
`s_ha_dr_`*`Hisn`*`_to_`*`acceptor`*	`float`	Propagated sigma for the HA modulation.

36.6 Marcus-Hush reorganization per mode

Key	Type	Meaning
`dr_rss_FeFe`	`float`	RSS-aggregated Δr along the FeFe channel.
`dr_rss_FeN`, `dr_rss_FeS`, `dr_rss_NH`, `dr_rss_HA`	`float`	RSS-aggregated Δr along each parent channel (equation (38)).
`lambda_FeFe`, `lambda_FeN`, `lambda_FeS`, `lambda_NH`, `lambda_HA`	`float`	Per-mode reorganization energy in cm^{-1} per parent channel (equation (43)).
`s_dr_rss_*`	`float`	Propagated sigma for each RSS modulation.
`s_lambda_*`	`float`	Propagated sigma for each per-mode λ.

36.7 SCSD amplitudes

One key per $D_{2\text{h}}$ irrep, for both the reference and the distortion projections (section 15).

Key	Type	Meaning
scsd_Ag, scsd_B1g, scsd_B2g, scsd_B3g, scsd_Au, scsd_B1u, scsd_B2u, scsd_B3u	float	Distortion amplitude in each irrep, in Å.
s_scsd_*	float	Propagated sigma for each irrep amplitude (v1.0.4 fix).

36.8 Kernel-classification scores

Key	Type	Meaning
kern_primary	str	Name of the highest-scoring kernel template.
kern_primary_score	float	Score (%) of the primary kernel.
kern_secondary	str	Name of the second-highest kernel.
kern_secondary_score	float	Score (%) of the secondary.
kern_loc	float	Localisation fraction (%): fraction of the mode's total amplitude residing on the cluster-core atoms.

36.9 Group amplitudes and secondary-structure descriptors

For each group key in coord_info.group_map, the following are appended (suffixed by group name – e.g. a_oop_pct_Cys11).

Key	Type	Meaning
a_oop_pct_*group*	float	Per-group α_{OOP}.
a_inp_pct_*group*	float	Per-group α_{INP}.
amplitude_a_*group*	float	Total amplitude (RMS over atoms in the group).
sse_amplitude_*	float	Per-SSE-element amplitudes (only when SSE analysis is enabled).

37 The Config dataclass

⇝ config.Config

The Config dataclass is the single source of truth for every configurable parameter. All 48 fields are enumerated in part V. The dataclass is built either from a TOML file (via Config.from_toml, the classmethod) or constructed in Python directly (useful for tests). Field defaults are listed alongside the field definitions in config.py and reproduced in part V.

Validation. After construction, Config.validate runs a small set of internal-consistency checks:

- freq_max > freq_min (if both set).

- All cutoff distances positive.
- `temp_k` either `None` or positive.
- Mutually exclusive: `analyze_all_clusters` and `cluster_index` > 0 are not both meaningful.

Validation failures raise `ConfigError` before any analysis begins, ensuring the user sees the problem before the long disk-scan phase.

Part V

Configuration Reference

This part enumerates every field of the `Config` dataclass (section 37) with its type, default, effect on the analysis, valid range or set of accepted values, and which functions consume it. The fields are grouped by topic, in the order they appear in `src/modenanalyse_2fe2s/config.py`; the table at the start of each section gives the line-range in `config.py` for cross-reference.

38 Input/output paths and frequency window

Field	Type	Default	Effect
`log_file`	`str`	`""`	Path to the Gaussian `.log` or ORCA `.hess` input. Required. The choice between Gaussian and ORCA is automatic via filename extension.
`pdb_file`	`str`	`""`	Path to a PDB file. Optional. Required for residue labels and SSE analysis.
`output_dir`	`str`	`""`	Directory for all output workbooks and the run-log. Created if missing.
`freq_min`	`Optional[float]`	`None`	Lower frequency bound of the analysis window, in cm^{-1}. Half-open: a mode at exactly $\omega =$ `freq_min` is *included*.
`freq_max`	`Optional[float]`	`None`	Upper bound. Half-open: a mode at exactly $\omega =$ `freq_max` is *excluded* (so contiguous windows can be defined without overlap).
`freq_windows`	`Optional[List[Tuple]]`	`None`	Multi-window analysis: a list of (lo, hi) tuples. When set, `freq_min`/`freq_max` are ignored. Each window triggers a separate Phase-4 run.
`temp_k`	`Optional[float]`	`None`	Temperature in K for u_{rms} computation. `None` triggers classical-amplitude mode (uses `amplitude` instead, see below).

Notes

- `freq_min` and `freq_max` are *half-open*; this convention is documented in the Manual and tested by `test_v104_audit_fixes::test_freq_window_half_open`.

- `freq_windows` is mutually exclusive with the single-window pair: the runner picks `freq_windows` if set, otherwise the `freq_min`/`freq_max` pair.
- For `temp_k` = `None`, the `amplitude` value (default 1.0 Å) is used in place of u_{rms} throughout the per-mode analysis. This is intended for benchmarking against tools that don't apply thermal scaling – production runs should set `temp_k` explicitly.

39 Coordination detection thresholds

Field	Type	Default	Effect
`pdb_chain`	`str`	`"A"`	PDB chain ID for residue-label mapping. Affects `geometry.find_coordinating_residues`.
`fe_coord_cutoff_n`	`float`	`2.50`	Maximum Fe-N distance (Å) for a residue's N to be considered an Fe-binding ligand. Used in `geometry.find_coordinating_residues`.
`fe_coord_cutoff_s`	`float`	`2.70`	Maximum Fe-S distance for Cys-S binding.
`fe_coord_cutoff_o`	`float`	`2.50`	Maximum Fe-O distance for Asp/Glu-O binding. Rare; supports mixed-coordination clusters.
`include_hn_vibration`	`bool`	`True`	Whether to compute the His-NH stretch and HA channel. When False, the hydrogen-inclusive eigenvector loader is skipped and the NH/HA sheets are filled with zeros.

40 Classification thresholds

Field	Type	Default	Effect
`mode_type_threshold`	`float`	`60.0`	α_{OOP} (%) threshold for the binary OOP/INP/mixed classification (equation (14)).
`mode_type_detail_thresholds`	`tuple[3]`	`(60, 75, 90)`	Three thresholds for the eight-bin detailed classification: boundary between mixed and majority, between majority and strong, between strong and pure.
`significance_threshold_low`	`float`	`1.0`	Value-to-sigma ratio threshold for the "below noise" colouring in Excel output.
`significance_threshold_high`	`float`	`3.0`	Ratio threshold for "clearly above noise".

41 Analysis switches

Field	Type	Default	Effect

analysis_compact	bool	True	Produce the compact main workbook with the most-used sheets.
analysis_full	bool	False	Produce the full workbook with every available descriptor. Doubles output size; off by default.
analyze_sse	bool	True	Run secondary-structure analysis if a PDB is supplied.
analyze_scsd	bool	True	Run SCSD projection per mode. Setting False saves $\sim 10\%$ runtime on large logs.
sse_chain	str	""	Restrict SSE analysis to a single chain. Empty = all chains.

42 Cluster discovery

Field	Type	Default	Effect
fe_s_cutoff	float	3.0	Maximum Fe-S distance for cluster discovery. Used by geometry.find_cluster.
fe_fe_cutoff	float	3.5	Maximum Fe-Fe distance for cluster discovery.
cluster_index	int	0	Which cluster to analyse (0 = first). Ignored when analyze_all_clusters = True.
analyze_all_clusters	bool	False	Iterate Phase 4 over every detected cluster, producing independent output directories with the _cluster## suffix.
strict_cluster	bool	False	Apply additional rhombic-geometry sanity checks during cluster discovery (section 24).

43 PCET acceptor heuristic

Field	Type	Default	Effect
pcet_enabled	bool	True	Whether to compute the HA channel at all. Setting False disables all PCET-related descriptors.
pcet_hbond_cutoff_a	float	4.0	Maximum H$\cdots$A distance (Å) for an acceptor candidate.
pcet_acceptor_r0_a	float	2.8	Gaussian weight centre d_0 for H$\cdots$A distance weighting (section 8).
pcet_acceptor_sigma_a	float	0.4	Gaussian weight width σ_d.

44 Modulation-spectrum broadening

Field	Type	Default	Effect
`reorg_spectrum_sigma_cm1`	`float`	`5.0`	Gaussian broadening width σ_M for the $M_X(\omega)$ spectra (equation (46)).
`reorg_spectrum_step_cm1`	`float`	`0.5`	Uniform ω-grid step for the modulation spectra and cumulative $\Lambda_X(\omega)$ curves.

45 Precision and noise model

Field	Type	Default	Effect
`include_hydrogen`	`bool`	`True`	Whether the cluster-core analysis should include any hydrogen atoms. Reserved for special workflows; mostly equivalent to `include_hn_vibration` = True.
`sigma_eigvec`	`float`	`5e-4`	Per-component eigenvector noise floor. Drives every Gauss- propagation calculation (part VII).
`sigma_coord`	`float`	`1e-3`	Per-component coordinate noise floor (e.g. from Gaussian's geometry-print precision).
`amplitude_threshold`	`float`	`0.001`	Below this thermal amplitude (Å), per-mode percentages are reported as NaN to prevent noise amplification.
`amplitude`	`float`	`1.0`	Classical-mode amplitude in Å when `temp_k` = `None`. Useful only for benchmarking.
`coord_match_tol`	`float`	`1.0`	Distance tolerance (Å) for the post-Kabsch nearest- neighbour PDB-to-Gaussian mapping (section 25).

46 Performance

Field	Type	Default	Effect
`scan_chunk_mb`	`int`	`16`	Disk-scan chunk size in MiB. Larger = faster scan on SSD, more RAM use.
`show_errors_in_excel`	`bool`	`True`	Whether to render mode-loading errors as red-coloured cells in the Excel output. When False, errors are only in the run-log.
`use_cache`	`bool`	`True`	Read/write the scan cache (section 2.8). Set False for one-off runs or to force re-scanning.

47 Interpolation workbook

Field	Type	Default	Effect
`interp_step`	`float`	`0.05`	Frequency grid step (cm^{-1}) for the interp workbook.
`interp_boundary_mode`	`str`	`"context"`	Boundary handling: `"context"` loads context modes (section 28), `"zero"` forces zero outside the window, `"nearest"` flatly extends the boundary value. Any other value is rejected by `Config.validate`.
`interp_edge_extend`	`float`	`0.5`	Grid padding (cm^{-1}) beyond the first/last mode. Since v1.2.2 in effect *only* when no frequency range is requested; with a range the grid spans exactly that range (section 28).
`interp_context_cm1`	`float`	`5.0`	Width of the context-mode search window (section 28).

48 Embedding

Field	Type	Default	Effect
`umap_n_neighbors`	`Optional[int]`	`None`	UMAP neighborhood size. `None` = auto-adapt to N_{modes}.
`export_embedding_plots`	`bool`	`True`	Whether to render and save PNG plots of the UMAP-HDBSCAN clusterings.
`logname_suffix`	`str`	`""`	Optional suffix appended to all output filenames (useful for differentiating windows or systems within one output dir).

Part VI

Output Reference

This part is the column-by-column documentation of every Excel workbook produced by the pipeline. For each workbook the sheets are enumerated; for each sheet, every column is given with its type, units, source function, and any propagated-sigma column that accompanies it. The implementation lives in `export.py`.

49 The main workbook (`*_analysis_main.xlsx`)

⇝ `export.export_all, export.export_main_excel`

The main workbook contains the per-mode descriptors and the system-wide aggregates. Sheets are grouped thematically:

- *Overview* sheets: `Modes_overview` (per mode: number, freq, red mass, mode type, primary kernel, classifier precision) – the index sheet.
- *Cluster-core* sheets: `Cluster_OOP_INP`, `Cluster_displacement`, `Cluster_expansion_rotation`, `SCSD`, `Kernel_class`.
- *Per-bond* sheets: `Fe_S_Cys`, `Fe_N_His` (and `Fe_O_Asp` when present); each contains per-mode `stretch`, `bend_inp`, `bend_oop` per ligand, plus their sigmas.
- *Marcus-Hush* sheets: `Reorganisation_per_mode` (per-mode λ_X and $\Delta r_X^{\mathrm{RSS}}$), `Reorganisation_total` (system-wide $\Lambda_X^{\mathrm{total}}$ per channel), `Modulationsspektrum` (mode-resolved $M_X(\omega)$), `Cumulative_Lambda`.
- *Diagnostic* sheets: `Coordination` (the Gaussian-to-PDB atom mapping used throughout), `B_factors` (the Phase-7 accumulated Debye-Waller factors per atom).

49.1 Sheet: `Modes_overview`

Column	Type	Units	Source
`number`	int	–	`logio.read_all_meta`
`freq`	float	cm^{-1}	`logio.read_all_meta`
`red_mass`	float	amu	`logio.read_all_meta`
`u_rms_a`	float	Å	`core.compute_thermal_amplitude`
`mode_type`	str	–	`core.classify_oop_inp`
`a_oop_pct`	float	%	`core.classify_oop_inp`
`kern_primary`	str	–	`core.classify_kernel_mode_from_evg`
`kern_primary_score`	float	%	`core.classify_kernel_mode_from_evg`
`precision`	str	–	`logio.get_eigvec`

49.2 Sheet: `Fe_S_Cys`

One row per mode. Columns repeated for each Cys ligand (suffix = `res_label`, e.g. `stretch_Cys11`, `stretch_Cys15`, etc.).

Column	Type	Units	Source
`number`	int	–	
`freq`	float	cm^{-1}	
`stretch_`*L*	float	Å	`core.analyze_fe_ligand`, equation (25)
`s_stretch_`*L*	float	Å	equation (31)
`bend_inp_`*L*	float	Å	equation (28)
`s_bend_inp_`*L*	float	Å	equation (31)
`bend_oop_`*L*	float	Å	equation (28)
`s_bend_oop_`*L*	float	Å	equation (31)
`bend_inp_pct_`*L*	float	%	`core.analyze_fe_ligand`
`bend_oop_pct_`*L*	float	%	`core.analyze_fe_ligand`

49.3 Sheet: Reorganisation_per_mode

One row per mode, one column-group per parent channel.

Column	Type	Units	Source
dr_rss_FeFe	float	Å	reorganization.aggregate_by_parent, equation (38)
s_dr_rss_FeFe	float	Å	propagated, part VII
lambda_FeFe	float	cm^{-1}	reorganization.lambda_pair_cm1, equation (43)
s_lambda_FeFe	float	cm^{-1}	propagated
dr_rss_FeN	float	Å	same as FeFe row
..._FeS, ..._NH, ..._HA			

49.4 Sheet: Reorganisation_total

A single row, one column per parent channel.

Column	Type	Units	Source
Lambda_total_FeFe	float	cm^{-1}	equation (44)
Lambda_total_FeN	float	cm^{-1}	
Lambda_total_FeS	float	cm^{-1}	
Lambda_total_NH	float	cm^{-1}	
Lambda_total_HA	float	cm^{-1}	

49.5 Sheet: Modulationsspektrum

One row per ω-grid point, one column per parent channel.

Column	Type	Units	Source
omega	float	cm^{-1}	grid generated from reorg_spectrum_step_cm1
M_FeFe	float	Å	reorganization.compute_modulation_spectra, equation (46)
M_FeN, M_FeS, M_NH, M_HA			
C_PCET	float	Å	reorganization.compute_co_modulation_spect equation (50)
C_PT_FeN, C_ET_FeS			
Lambda_FeFe	float	cm^{-1}	cumulative $\Lambda_X(\omega)$ equation (45)
Lambda_FeN, Lambda_FeS, Lambda_NH, Lambda_HA			

49.6 Sheet: SCSD

One row per mode, one column-pair per D_{2h} irrep.

Column	Type	Units	Source
number	int	–	
freq	float	cm^{-1}	
scsd_Ag, scsd_B1g, scsd_B2g, scsd_B3g, scsd_Au, scsd_B1u, scsd_B2u, scsd_B3u	float	Å	core.compute_scsd_for_mode_full, equation (52)
s_scsd_*	float	Å	propagated (section 15)

49.7 Sheet: Coordination

The Coordination sheet documents the master ligand list used by all per-mode analyses. One row per Fe-ligand bond.

Column	Type	Units	Source
Fe	str	–	e.g. Fe1, Fe2.
Fe_center	int	–	Gaussian centre of Fe.
Lig_center	int	–	Gaussian centre of ligand donor.
Lig_element	str	–	N, S, or O.
Lig_aname	str	–	PDB atom name.
Residue	str	–	e.g. Cys11, His87.
bond_length_a	float	Å	equilibrium Fe-L bond length.
H_center	int / NA	–	if applicable (His-NH protonated).

50 Interpolation workbook (*_analysis_interp##.xlsx)

⇝ export.export_interpolated_excel

Contains the per-mode descriptors re-sampled onto a uniform ω-grid of step interp_step. The boundary-mode behaviour is governed by interp_boundary_mode and interp_context_cm1.

Sheets

- Interpoliert_per_channel: ω vs. per-channel $\Delta r_X^{\mathrm{RSS}}(\omega)$, $\lambda_X(\omega)$.
- Cumulative: ω vs. cumulative $\Lambda_X(\omega_{\mathrm{cut}})$ per channel.
- Interp_bond_metrics: ω vs. per-bond stretch, bend_inp, bend_oop (one column per ligand).

The interp values are computed by linear interpolation between adjacent modes (in ω), anchored at the boundary by the context modes loaded in section 28.

51 SSE workbook (*_analysis_SSE.xlsx)

⇝ export.export_sse_excel

Produced only when SSE analysis is enabled and a PDB with SSE records (or DSSP records) is supplied. Sheets enumerate per-SSE-element amplitudes per mode, plus axial-tilt diagnostics.

Sheets

One sheet per descriptor, each a per-mode × per-SSE-element matrix. The element's principal axis is taken from its C_α backbone; *axial* and *lateral* denote the components parallel and perpendicular to that axis. Displacements use the thermally scaled eigenvector $\widetilde{\mathbf{e}}$; amplitudes are in Å, angles in degrees.

- `SSE_amplitude_mean` / `SSE_amplitude_max`: mean and maximum, over the element's atoms, of the per-atom displacement magnitude.
- `SSE_com_amplitude`: magnitude of the *mass-weighted* centre-of-mass displacement $\|\sum_i m_i \widetilde{\mathbf{e}}_i / \sum_i m_i\|$ (rigid-body translation of the element). *(v1.0.5: now mass-weighted; previously the unweighted atom-mean.)*
- `SSE_axial_amplitude`: mean absolute displacement component along the principal axis.
- `SSE_lateral_amplitude`: mean displacement component perpendicular to the principal axis.
- `SSE_lateral_std`: standard deviation of the lateral component across the atoms (lateral inhomogeneity).
- `SSE_stretching`: standard deviation of the axial component across the atoms (axial elongation/compression).
- `SSE_tilting_angle`: magnitude of the rigid-body rocking rotation perpendicular to the axis. It is the best-fit infinitesimal rotation $\boldsymbol{\omega}$ minimising $\sum_i m_i \|\mathbf{d}_i - \boldsymbol{\omega} \times \mathbf{r}_i\|^2$ ($\mathbf{d}_i$ = displacement after removing the COM translation, $\mathbf{r}_i$ = position about the mass centre), reported as $\|\boldsymbol{\omega}_\perp\|$ in degrees. *(v1.0.5: a genuine rigid-body tilt; previously the near-constant polar angle of the centroid translation.)*
- `SSE_internal_amplitude`: mean magnitude of the residual displacement after removing *both* the rigid translation and the rigid rotation – a TLS-style non-rigid internal deformation *(new in v1.0.5)*.
- `SSE_UMAP_Cluster` (optional): present when the SSE-fingerprint UMAP–HDBSCAN clustering is computed (`compute_sse_umap_cluster`); HDBSCAN cluster ID per mode (-1 = noise). The clustering uses the decorrelated five-feature subset `com_amplitude`, `tilting_angle`, `internal_amplitude`, `axial_amplitude`, `lateral_amplitude`.

Each amplitude sheet carries a matching `s_`-sigma (part VII): the mean-type amplitudes use the standard error $\sigma_e^{\text{therm}}/\sqrt{n}$, the tilt angle uses $\sigma_e^{\text{therm}}/R_g$ converted to degrees.

52 Embedding workbook (`*_analysis_embedding.xlsx`)

⇝ `export.export_embedding_excel`

Three parallel UMAP-HDBSCAN clusterings, on three complementary feature spaces:

- *Reorg-features*: per-mode ($\lambda_{\text{FeFe}}, \lambda_{\text{FeN}}, \lambda_{\text{FeS}}, \lambda_{\text{NH}}, \lambda_{\text{HA}}$).
- *SSE-fingerprint*: per-mode SSE-element amplitudes (only when SSE is enabled).
- *C_α-amplitude* fingerprint: per-mode mean α-carbon amplitudes per residue group.

Each clustering produces 4 sheets: `Projektionen` (per-mode UMAP-x, UMAP-y), `Ca_Amplituden` (the underlying feature matrix), `UMAP_Cluster` (HDBSCAN cluster IDs per mode; -1 = noise), `UMAP_Cluster_Profil` (mean feature vector per cluster). Together these three workbooks form the hierarchical-clustering signature of the system.

Part VII

Uncertainty Propagation Complete

This part derives every sigma field reported in the output workbooks from the two user-controllable noise floors σ_e = `sigma_eigvec` and σ_c = `sigma_coord`, applied to the relevant quantities via standard linearised Gauss propagation.

53 Underlying assumptions

Every linearised sigma in this section assumes:

- Eigenvector components $\mathbf{e}_{l,i}^{(\alpha)}$ are independent Gaussian random variables with zero mean and per-component standard deviation σ_e (default 5×10^{-4}, motivated by the printed precision of Gaussian `hpmodes`).
- Atomic coordinates $r_i^{(\alpha)}$ are independent Gaussian random variables with per-component standard deviation σ_c (default 10^{-3} Å, motivated by the Standard-Orientation print format).
- Frequencies ω_l and reduced masses m_l are treated as exact (their printed precision is sufficient for the linear-noise regime).
- Eigenvector and coordinate noises are uncorrelated.

The general rule for a smooth function $y = f(\mathbf{e}, \mathbf{r})$ is

$$\sigma_y^2 \;\approx\; \sum_i (\partial_{\mathbf{e}_i} f)^2 \sigma_e^2 + \sum_i (\partial_{\mathbf{r}_i} f)^2 \sigma_c^2. \tag{79}$$

This expansion is applied below to each output.

54 Thermal scaling of σ_e

↝ `core.compute_thermal_amplitude`

Every geometric observable consumed downstream uses the thermally scaled eigenvector $\widetilde{\mathbf{e}} = \mathbf{e} \cdot u_{\mathrm{rms}}$. The corresponding scaled per-component noise is

$$\boxed{\sigma_e^{\mathrm{therm}}(\omega_l) \;=\; \sigma_e \cdot u_{\mathrm{rms}}(\omega_l)} \tag{80}$$

This rescaling is the universally applied convention; failure to apply it was the FUND 6/7/8 bug class in v1.0.4 (section 69). For $\omega = 200$ cm^{-1}, $T = 80$ K, $m_{\mathrm{red}} = 30$ amu, $u_{\mathrm{rms}} \approx 0.05$ Å, so $\sigma_e^{\mathrm{therm}} \approx 2.5 \times 10^{-5}$ Å.

55 Bond-modulation sigmas: `s_stretch`, `s_bend_inp`, `s_bend_oop`

⇝ `core.analyze_fe_ligand`

The bond stretch $\Delta r_{\text{stretch}}$ (equation (25)) is the dot product of a 3-vector difference with a unit vector:

$$\Delta r_{\text{stretch}} \;=\; (\widetilde{\mathbf{e}}_L - \widetilde{\mathbf{e}}_{\text{Fe}}) \cdot \hat{\mathbf{b}}. \tag{81}$$

Treating $\hat{\mathbf{b}}$ as constant (since coordinates have their own sigma σ_c, treated separately) and the eigenvector components as independent, the variance of the dot product is

$$\sigma^2_{\text{stretch}} \;=\; \sum_\alpha b_\alpha^2 \cdot \text{Var}[(\widetilde{\mathbf{e}}_L - \widetilde{\mathbf{e}}_{\text{Fe}})_\alpha] \;=\; \sum_\alpha b_\alpha^2 \cdot 2(\sigma_e^{\text{therm}})^2 \;=\; 2(\sigma_e^{\text{therm}})^2, \tag{82}$$

where the last equality uses $\sum_\alpha b_\alpha^2 = 1$ (unit vector). Therefore

$$\boxed{\sigma_{\text{stretch}} \;=\; \sqrt{2}\,\sigma_e \cdot u_{\text{rms}}(\omega_l)} \tag{83}$$

This is the formula in equation (31). The same argument applies to $\Delta r_{\text{bend\_inp}}$ and $\Delta r_{\text{bend\_oop}}$: they are obtained from the same difference vector, projected onto different orthonormal axes; each carries the same $\sqrt{2}\,\sigma_e^{\text{therm}}$.

56 N-H stretch sigma: `s_hn_stretch`

⇝ `core.analyze_his_hn`

Same form as equation (83), with the v1.0.4 FUND 6 fix (section 69) applying u_{rms}:

$$\boxed{\sigma_{\text{NH}} \;=\; \sqrt{2}\,\sigma_e \cdot u_{\text{rms}}(\omega_l)} \tag{84}$$

57 HA reaction-coordinate sigma: `s_ha_dr`

⇝ `reorganization.compute_mode_modulations`

The HA reaction coordinate equation (22) is

$$\Delta r_{HA} \;=\; (\widetilde{\mathbf{e}}_H - \widetilde{\mathbf{e}}_N) \cdot \hat{\mathbf{n}}_{NA}. \tag{85}$$

Again a unit-norm projection of a difference of two independent eigenvectors:

$$\boxed{\sigma_{HA} \;=\; \sqrt{2}\,\sigma_e \cdot u_{\text{rms}}(\omega_l)} \tag{86}$$

58 RSS-aggregate sigma: `s_dr_rss_X`

⇝ `reorganization.aggregate_by_parent`

For the RSS-aggregated parent-channel modulation $\Delta r_X^{\text{RSS}} = \sqrt{\sum_s w_s (\Delta r_s)^2}$, the linearised propagation of

independent Δr_s sigmas (each $= \sqrt{2}\sigma_e u_{\rm rms}$) gives

$$\sigma^2_{\rm RSS} = \sum_s \left(\frac{\partial \Delta r^{\rm RSS}}{\partial \Delta r_s}\right)^2 \sigma^2_{\Delta r_s} \tag{87}$$

$$= \sum_s \left(\frac{w_s \Delta r_s}{\Delta r^{\rm RSS}}\right)^2 \cdot 2(\sigma_e^{\rm therm})^2 \tag{88}$$

$$= \frac{2(\sigma_e^{\rm therm})^2}{(\Delta r^{\rm RSS})^2} \sum_s w_s^2 (\Delta r_s)^2. \tag{89}$$

So

$$\boxed{\sigma^X_{\rm RSS} = \sqrt{2}\,\sigma_e \cdot u_{\rm rms}(\omega_l) \cdot \frac{\sqrt{\sum_s w_s^2 (\Delta r_s)^2}}{\Delta r^{\rm RSS}}} \tag{90}$$

For the single-sub-channel limit ($|s| = 1$, $w = 1$), the square-root factor reduces to 1, and $\sigma_{\rm RSS} = \sqrt{2}\,\sigma_e^{\rm therm}$ as expected.

For the special case of all-equal sub-channels ($w_s = w$, $|\Delta r_s| = \Delta r_0$), the factor becomes $1/\sqrt{|s|}$ – i.e. averaging over $|s|$ sub-channels reduces the relative uncertainty by $\sqrt{|s|}$.

59 Per-mode λ_X sigma: `s_lambda_X`

⇝ `reorganization.compute_mode_modulations`

$\lambda_X = \frac{1}{2}\mu\omega^2(\Delta r^{\rm RSS})^2/(hc)$. For exact μ, ω, hc, the variance is

$$\sigma^2_{\lambda_X} = \left(\frac{\partial \lambda_X}{\partial \Delta r^{\rm RSS}}\right)^2 \cdot \sigma^2_{\rm RSS} = \left(\frac{\mu\omega^2 \Delta r^{\rm RSS}}{hc}\right)^2 \cdot \sigma^2_{\rm RSS}. \tag{91}$$

So

$$\boxed{\sigma_{\lambda_X} = \frac{2\lambda_X}{\Delta r^{\rm RSS}} \cdot \sigma_{\rm RSS}} \tag{92}$$

i.e. the relative uncertainty on λ is twice the relative uncertainty on the RSS modulation – a direct consequence of $\lambda \propto (\Delta r)^2$.

60 SCSD sigmas: `s_scsd_*`

⇝ `core.compute_scsd_for_mode_full`

The SCSD reference projection uses only equilibrium coordinates:

$$\sigma^{(\Gamma)}_{\rm SCSD,ref} = \sqrt{2}\,\sigma_c. \tag{93}$$

The distorted projection uses both coordinates (for the centre- of-mass term) and the thermally scaled eigenvector:

$$\sigma^{(\Gamma)}_{\rm SCSD,dist} = \sqrt{2}\,\sqrt{\sigma_c^2 + (\sigma_e \cdot u_{\rm rms}(\omega_l))^2}. \tag{94}$$

The difference SCSD is the difference of the two; by variance addition (independent),

$$\boxed{\sigma^{(\Gamma)}_{\rm SCSD,diff} = \sqrt{(\sigma^{(\Gamma)}_{\rm SCSD,ref})^2 + (\sigma^{(\Gamma)}_{\rm SCSD,dist})^2}} \tag{95}$$

This is the v1.0.4 FUND 8 fix (section 69): pre-v1.0.4, hard-coded constants $(5 \times 10^{-7}, 5 \times 10^{-6})$ were used instead.

61 Cluster expansion and rotation sigmas

↝ `core.classify_oop_inp`

Both are derived from the cluster-COM displacement plus post-projection cleanup. The COM is an average of 4 atomic displacements, so its sigma is $\sigma_e^{\text{therm}}/2$. The cluster-expansion observable subtracts this COM from each cluster atom and reads the radial component:

$$\sigma_{\text{expansion}} = \sqrt{(\sigma_e^{\text{therm}})^2 + (\sigma_e^{\text{therm}}/2)^2} = \tfrac{\sqrt{5}}{2}\,\sigma_e^{\text{therm}}. \tag{96}$$

The rotation angle is a small-angle approximation of the in-plane projected displacement, so its sigma is the in-plane sigma divided by the mean cluster radius.

62 Aggregate sigmas in totals: `s_Lambda_total_X`

↝ `reorganization.compute_total_reorganization`

The system-wide Λ_X^{total} is a sum of per-mode $\lambda_X(i)$ values. Assuming the per-mode sigmas are independent,

$$\sigma_{\Lambda_X^{\text{total}}} = \sqrt{\sum_i \sigma^2_{\lambda_X(i)}}. \tag{97}$$

For typical analyses with ~ 100 modes per window, the relative sigma on the total is $\sim 10\%$ of the relative sigma on a single mode.

63 The 24 `s_*` fields enumerated

The complete list of propagated sigmas reported in the output:

Sigma field	Formula	Comment
`s_stretch_L`	$\sqrt{2}\sigma_e u_{\text{rms}}$	per Fe-L bond
`s_bend_inp_L`	$\sqrt{2}\sigma_e u_{\text{rms}}$	
`s_bend_oop_L`	$\sqrt{2}\sigma_e u_{\text{rms}}$	
`s_hn_stretch_His`	$\sqrt{2}\sigma_e u_{\text{rms}}$	FUND 6 fix
`s_ha_dr_His_to_A`	$\sqrt{2}\sigma_e u_{\text{rms}}$	FUND 7 fix
`s_dr_rss_FeFe`	equation (90)	
`s_dr_rss_FeN`	equation (90)	
`s_dr_rss_FeS`	equation (90)	
`s_dr_rss_NH`	equation (90)	
`s_dr_rss_HA`	equation (90)	
`s_lambda_FeFe`	equation (92)	
`s_lambda_FeN`	equation (92)	
`s_lambda_FeS`	equation (92)	

`s_lambda_NH`	equation (92)	
`s_lambda_HA`	equation (92)	
`s_scsd_Ag, ..., B3u`	section 15	8 irrep amplitudes (FUND 8 fix)
`s_expansion_a`	$\frac{\sqrt{5}}{2}\sigma_e u_{\mathrm{rms}}$	
`s_rotation_deg`	in-plane sigma / radius	
`s_lambda_total_X` (5 channels)	RSS over modes	system-wide

Significance colouring in Excel. The export layer compares each value to its sigma and assigns a colour: below `significance_threshold_low` σ = grey (below noise); between low and `significance_threshold_high` σ = yellow (marginal); above high σ = white (significant). This makes the noise floor visually apparent in the workbook.

Part VIII

Audit Trail of v1.0.4

This part documents every code-level and documentation-level change between v1.0.3 and v1.0.4. The two principal categories are *fundamental code findings* (`FUND` 1 through `FUND` 13) and *manual drift findings*. Each entry follows a uniform template: *symptom*, *root cause*, *fix*, *regression test reference*, *migration impact*.

64 FUND 1: `evec_raw` mutation in place

Symptom. After Phase 4, an attempt to re-use the original (un-scaled) eigenvector for diagnostics revealed that `r["evec_raw"]` was numerically identical to `r["evg"]` (the scaled one). The raw eigenvector had been clobbered.

Root cause. In `core.analyze_mode_with_fallback`, the line `evg = evec_raw * u_rms` relied on numpy creating a new array. But an earlier in-place line `evec_raw *= u_rms` (left over from a refactor) was modifying the source array in place. The result-dict's `evec_raw` key then pointed to the already-scaled array.

Fix. Replaced the in-place op with `evg = evec_raw * u_rms` (explicit out-of-place multiplication). Added a defensive `.copy()` at the entry to `analyze_mode_with_fallback`.

Regression test. `tests/test_v104_audit_fixes.py::test_evec_raw_preserved_after_scaling`.

Migration impact. None for numerical outputs. Affects only direct consumers of the result-dict's `evec_raw` field (which were always supposed to read the unscaled eigenvector).

65 FUND 2: scan-cache key insensitive to file mtime

Symptom. Re-running an analysis after updating the Gaussian log produced output identical to the previous run – the new log was not being read.

Root cause. The cache key (section 2.8) hashed only the file path and size, not the modification time. A user who replaced their log with a re-run of the same length got a false cache hit.

Fix. Added `os.stat(filepath).st_mtime` to the cache-key hash. Bumped `logio._CACHE_VERSION` to invalidate all pre-existing caches.

Regression test. `tests/test_v104_audit_fixes.py::test_cache_key_depends_on_mtime`.

Migration impact. First run after upgrading to v1.0.4 re-scans every log (no cache hit). Subsequent runs are cached as before.

66 FUND 3: synthetic-zero `evg` causes B-factor crash

Symptom. With `interp_boundary_mode = "context"` and a log whose highest real mode is well within the analysis window, the B-factor accumulation crashed on a NoneType.

Root cause. The FUND-13 synthetic zero mode was being created with `evg = None`, but the Phase-7 B-factor loop unconditionally summed `r["evg"]**2`.

Fix. Two-step guard: `_make_synthetic_zero_mode` omits the `evg` key entirely, and the B-factor loop checks `r.get("evg") is not None` before summing. Same guard applied to the SSE-amplitude and group-amplitude loops.

Regression test. `test_v104_audit_fixes.py::test_synthetic_zero_skipped_by_b_factor`.

67 FUND 4: cluster-normal sign flip between clusters

Symptom. In a multi-cluster system, the same physical $\Delta r_{\text{bend\_oop}}$ for an identical mode had opposite signs in different clusters.

Root cause. The SVD-based cluster-normal was returning $\mathbf{V}_{:,3}$ with SVD's arbitrary sign. For some clusters this happened to point $+\hat{\mathbf{z}}$, for others $-\hat{\mathbf{z}}$.

Fix. After SVD, check the sign of $\mathbf{V}_{:,3} \cdot \hat{\mathbf{z}}$ and flip $\hat{\mathbf{n}}$ if negative (section 3.6). Applied uniformly per cluster.

Regression test. `test_v104_audit_fixes.py::test_cluster_normal_sign_convention`.

Migration impact. For multi-cluster systems with clusters whose $\hat{\mathbf{n}}$ previously pointed $-\hat{\mathbf{z}}$, the sign of `bend_oop` flips. The magnitudes are unchanged.

68 FUND 5: His-NH skip warning fires for unprotonated His

Symptom. `UserWarning` "His-NH not found" fired for every unprotonated His ligand, polluting the run-log.

Root cause. The unprotonated check in `core.analyze_his_hn` happened *after* the warning emission.

Fix. Re-ordered: protonation check first, warning only if the H atom is expected (his.`his_protonated` = True) but not found.

Regression test. `test_v104_audit_fixes.py::test_unprotonated_his_no_warning`.

69 FUND 6, 7, 8: sigma-propagation drift

These three findings share a single root cause: the sigma for geometric observables had not been thermally rescaled by u_{rms}, despite the underlying observables having that scaling applied.

FUND 6 – His-NH stretch sigma. Pre-v1.0.4: `s_hn_stretch` = $\sqrt{2}\,\sigma_e$ (raw, in eigenvector-component units). Post-v1.0.4: $\sqrt{2}\,\sigma_e \cdot u_{\mathrm{rms}}$. For typical $u_{\mathrm{rms}} \approx 0.05$ Å, the old sigma was $\sim 20\times$ too large, forcing many genuine PCET signals below the significance threshold.

FUND 7 – HA reaction-coordinate sigma. Same drift as FUND 6, applied to the `s_ha_dr` column.

FUND 8 – SCSD sigmas. Pre-v1.0.4: hard-coded constants 5×10^{-7} (reference) and 5×10^{-6} (distortion) regardless of mode. Post- v1.0.4: built from σ_e, σ_c, and u_{rms} per the formulas in section 15.

Root cause (all three). Sigma-propagation was added as an afterthought to the code; the authors did not at the time appreciate that the underlying observables were already u_{rms}-scaled. The fix is to apply u_{rms} uniformly to all sigmas of observables derived from $\widetilde{\mathbf{e}}$.

Regression tests. `test_v104_audit_fixes.py::test_sigma_hn_stretch_uses_urms`, `test_v104_audit_fixes.py::test_sigma_ha_dr_uses_urms`, `test_v104_audit_fixes.py::test_sigma_scsd_uses_eigvec_coord_noise`.

Migration impact. `s_hn_stretch`, `s_ha_dr`, `s_scsd_*` columns are now numerically different (typically $\sim 20\times$ smaller for the first two; $\sim 1000\times$ larger for the third relative to the hard-coded constants). The Excel significance- colouring is now meaningful for these columns; previously it was either uniformly above threshold (FUND 6, 7) or uniformly below (FUND 8). The *underlying* observables in the workbook are unchanged.

70 FUND 9: `coord_match_tol` not applied

Symptom. PDB atoms far from any Gaussian atom (e.g. in regions of the protein not in the QM zone) were being mapped to the nearest Gaussian atom regardless of distance, producing nonsense residue-label assignments.

Root cause. The nearest-neighbour loop in `geometry._build_pdb_to_center_map` computed `best_dist` correctly but never checked it against the tolerance.

Fix. Apply `best_dist <= cfg.coord_match_tol`; mark `pdb_to_center[idx] = None` for over-tolerance pairs.

Regression test. `test_v104_audit_fixes.py::test_coord_match_tol_filters_distant_atoms`.

71 FUND 10: `ChannelGeometry.weight` ignored in single-sub-channel parents

Symptom. For a parent channel with a single sub-channel (e.g. NH for a single-His system), the RSS aggregator returned $|\Delta r|$ instead of $\sqrt{w}|\Delta r|$.

Root cause. Fast-path for $|s| = 1$ in `reorganization.aggregate_by_parent` short-circuited the weight multiplication.

Fix. Removed the fast-path; the general formula $\sqrt{\sum w_s(\Delta r_s)^2}$ now applies for all $|s| \geq 1$.

Regression test. `test_v104_audit_fixes.py::test_weight_applied_single_subchannel`.

Migration impact. Only affects channels with $w \neq 1$. In practice this is HA only (gaussian distance weights). The numerical impact is small ($< 3\%$) but the formula is now consistent across all channel cardinalities.

72 FUND 11: Excel mode-overview omits `u_rms_a`

Symptom. The `Modes_overview` sheet did not show the per-mode thermal amplitude u_{rms}, making it hard to relate the sigma values to the noise model.

Fix. Added `u_rms_a` column to `Modes_overview` export.

Regression test. `test_v104_audit_fixes.py::test_overview_includes_urms`.

73 FUND 12: interpolation boundary anchor

Symptom. For an analysis window whose right edge falls in a sparsely sampled region of the DFT spectrum, the interpolation workbook showed an abrupt drop to zero at the right edge – the piecewise-linear curve had no anchor beyond the last mode in the window.

Root cause. The context-mode loader looked only for modes within `interp_context_cm1` of the boundary. When the nearest mode beyond the window was farther than that, the fallback "load a single anchor mode" was missing.

Fix. Added the FUND-12 fallback (section 28 Scenario B): when no real mode lies within `interp_context_cm1` of the upper edge, load *exactly one* real mode – the closest above the upper edge – as the boundary anchor.

Regression test. `test_v104_audit_fixes.py::test_fund_12_single_anchor_fallback`.

Migration impact. Interpolation workbook gains anchor points for affected windows. The interpolated $\Lambda_X(\omega)$ curves no longer have the spurious right-edge drop.

74 FUND 13: synthetic zero anchor

Symptom. For analysis windows whose right edge coincides with the highest frequency in the DFT log (no modes above), even the FUND-12 fallback can't help – there is genuinely no real mode to load. The right-edge of the interpolation remained unanchored.

Root cause. No fallback existed for the case of an absolutely terminal spectrum.

Fix. Introduced `runner._make_synthetic_zero_mode`: when no real mode exists above `freq_max`, inject a synthetic mode at `freq_max` + `interp_context_cm1` with all observables set to zero (the physically expected behaviour of a non-existent mode). Sentinel values `number` $= -1$, `mode_type` $=$ `"synthetic_zero"`, `precision` $=$ `"synthetic"` identify it. The synthetic mode contributes to interpolation anchors but *not* to system-wide aggregates (B-factors, Λ_X^{total}).

Regression tests. `test_v104_audit_fixes.py::test_fund_13_synthetic_zero_inserted_at_correct_freq`, `test_v104_audit_fixes.py::test_fund_13_synthetic_zero_skipped_by_aggregates`, `test_v104_audit_fixes.py::test_fund_13_no_synthetic_at_lower_edge`.

Migration impact. For terminal-spectrum windows the interpolation tail is now well-defined. Headline Λ_X^{total} values are unchanged (synthetic mode contributes zero).

75 FUND 14: doubled `His_HN NICHT erkannt` warning

Symptom. On a real-world [2Fe-2S]-Cys_2His_2 run (May 2026) where both coordinating histidines were deprotonated, the "`His_HN NICHT erkannt: His <num>`" warning appeared *twice* per ligand in the run

log instead of once. The doubled output suggested a duplicate analysis was running, which was alarming on initial inspection. The same bug would trigger on any system whose histidine-N coordinating ligand carries no detectable H atom (Rieske-type Cys_2His_2 in its deprotonated state, or mitoNEET-type Cys_3His after deprotonation).

Root cause. In `geometry._add_his_hn_info`, the "`if not lig.his_protonated:`" warning emission was placed *inside* the outer "`for use_pdb_only in (True, False):`" loop. The outer loop runs twice (PDB-only path, then Gaussian-fallback path); for a deprotonated His, both paths fail to find an H atom, so the unsuccessful warning fired in each iteration – producing two identical warning lines.

The dedup set `_WARNED_HN_SKIP` was not involved, because the warning was a one-shot `print/runlog.warn` rather than a deduplicated `UserWarning`; the dedup set guards a different code path.

Fix. Hoist the "`if not lig.his_protonated:`" block out of the outer loop, so it is executed exactly once per ligand after both paths have been attempted.

Regression test. `test_v104_audit_fixes.py::test_his_hn_warning_emitted_only_once_per_ligand` constructs a synthetic His ligand with no nearby H, runs `_add_his_hn_info`, and asserts that exactly one `"His_HN NICHT"`-warning is emitted, not two.

Migration impact. Numerical output unchanged. Run-log noise reduced by 50% in the specific case of deprotonated His ligands.

76 FUND 15: misleading PCET-disabled message for deprotonated His

Symptom. On the deprotonated-His system from the FUND 14 run, the PCET-decision line in the run log stated:

```
PCET reorg:  NOT active (no His ligands at cluster).
```

But the same run produced `Lambda_FeN` > 0 and `Fe_N_His` sheets populated for both histidines – clearly His ligands were coordinating Fe. The PCET-decision text was actively misleading. The same bug would mislead any analysis of a [2Fe-2S]-Cys/His system where all histidines happen to be deprotonated.

Root cause. In `runner._run_analysis_single`, the PCET-disabled branch selected its message based on `coord_info.pcet_info.n_his == 0`, where `n_his` (from `pcet_et.build_pcet_info`) counts only *protonated* His ligands (`pcet_et.py`, line 243-244, skips non-protonated His). For a deprot system with His ligands but no detectable HN bond, `n_his` is zero, so the "no His ligands at cluster" message fires – even though His ligands are clearly present.

Fix. In the PCET-disabled branch, count protein-total His ligands directly via `coord_info.ligands` (independent of protonation status) and use the count to differentiate the two disjoint cases:

- "no His ligands at cluster" (true Cys_4 system),

- "N His ligand(s) found but none protonated" (deprotonated state).

Regression test. `test_v104_audit_fixes.py::test_pcet_decision_distinguishes_no_his_from_deprotonated_h` constructs two synthetic `LigandInfo` lists (Cys_4 only, Cys_2His_2) and verifies the counting logic.

Migration impact. Numerical output unchanged. Reviewer correctly informed about the actual reason PCET is disabled.

77 FUND 16: spurious `no context modes` warning at full-spectrum runs

Symptom. All systems in the same batch run emitted the warning

```
interp_boundary_mode='context' but no context modes available - boundary values set
to 0.
```

even though the runs had not set `freq_min` or `freq_max` (full-spectrum analysis). For a full-spectrum run there is no interpolation boundary that requires anchoring – the natural spectrum end serves as the upper limit and `np.interp`'s right-extrapolation already returns the value at the highest mode.

Root cause. In `export.export_interpolated_excel`, the warning condition was

```
if bmode == "context" and not (_ctx_l or _ctx_r):
    runlog.warn(...)
```

without checking whether the user actually requested a window. The context-loading code in `runner._run_analysis_single` silently skips its body when `freq_max` is None (line 988), so *_ctx_r* is empty for full-spectrum runs too – and the export-stage warning fires inappropriately.

Fix. Guard the warning with an additional check that the user has explicitly set at least one window parameter:

```
_user_set_window = (cfg.freq_min is not None
                    or cfg.freq_max is not None
                    or cfg.freq_windows is not None)
if bmode == "context" and not (_ctx_l or _ctx_r)
        and _user_set_window:
    runlog.warn(...)
```

Regression test. `test_v104_audit_fixes.py::test_interp_no_context_warning_suppressed_when_no_window` constructs a default Config, verifies that the `_user_set_window` flag is False, and then verifies it flips to True after setting `freq_max`.

Migration impact. Numerical output unchanged. Spurious warning silenced for full-spectrum runs. The warning still fires correctly when the user actually sets a window with no context modes available.

78 FUND 17: REPORT.txt missing `freq_windows`

Symptom. The batch run used `freq_windows = [(0,100), (100,300), (300,500)]` but the resulting REPORT.txt showed

```
freq_filter:  - - - cm-1
```

making the active frequency filter invisible in the run audit trail.

Root cause. In `RunLog.write_befund` (the report-writer method in `logio.py`), the report writer only formatted `freq_min` and `freq_max` (both None for a windows-only run); `freq_windows` was never emitted. The user had to read the original Config to learn what windows were active.

Fix. After the `freq_filter` line, emit an additional `freq_windows:  [...]` line whenever `cfg.freq_windows` is non-empty.

Regression test. `test_v104_audit_fixes.py::test_report_includes_freq_windows` verifies the string-format produced by the report writer.

Migration impact. Numerical output unchanged. REPORT.txt audit trail now fully records which windows were active.

79 Manual drift findings

In addition to the code-level FUND findings, an audit of the v1.0.3 manual identified 9 drift items where the manual's description had diverged from the code:

1. "OOP fraction is kinetic-energy fraction" – the manual claimed mass-weighted aggregation. Reality: geometric- cartesian (section 5.2). Fixed.
2. "HA reaction coordinate is $(\mathbf{e}_H - \mathbf{e}_A) \cdot \hat{\mathbf{n}}_{HA}$" – wrong. Reality: $(\mathbf{e}_H - \mathbf{e}_N) \cdot \hat{\mathbf{n}}_{NA}$ (equation (22)). Fixed with derivation.
3. "σ scaling is constant" – did not flag the FUND 6/7/8 fix. Fixed.
4. "window boundaries are closed" – conflicts with code's half-open convention. Fixed.
5. "cluster normal points away from the protein bulk" – not enforced anywhere. Reality: $+\hat{\mathbf{z}}$ half-space of the Gaussian frame (section 6.2). Fixed.
6. "L1 sum aggregation" – wrong. Reality: RSS (section 12). Fixed with consistency proof.
7. "`interp_boundary_mode = "context"` works for all windows" – did not document the FUND 12/13 fallbacks. Fixed.
8. "α_{OOP} is always in $[0\%, 100\%]$" – the per-atom decomposition is, but aggregates over atom groups can exceed 100% if displacements differ in sign. Fixed.
9. Missing `Coordination` sheet documentation. Fixed.

80 Migration notes for v1.0.4

Columns with numerical differences (output-affecting):

- `s_hn_stretch_*`: $\sim 20\times$ smaller (FUND 6).
- `s_ha_dr_*`: $\sim 20\times$ smaller (FUND 7).
- `s_scsd_*`: $\sim 1000\times$ larger (FUND 8) – previously implausibly small.
- `bend_oop` in some multi-cluster systems: sign flipped (FUND 4). Magnitude unchanged.
- Interpolation workbook tails: now anchored (FUND 12, 13). Headline totals unchanged.

Columns with no numerical difference but improved documentation:

- All RSS-aggregated Δr columns – now derived rigorously from energetic-consistency arguments (section 12).
- All `a_oop_pct_*` columns – now documented as geometric-cartesian fraction (section 5.2).
- HA channel – now documented as reaction-coordinate formula (section 8).

No breaking config changes. The TOML schema is backward-compatible. All existing v1.0.3 configs run unchanged under v1.0.4.

Part IX

Test Suite

This part documents the test suite. As of v1.2.3 it comprises 201 unit tests + 4 end-to-end (E2E) smoke tests (205 total; the E2E tests carry the `slow` marker and are deselected by default). The detailed per-suite sections that follow describe the v1.0.4 regression baseline; the suite was expanded in v1.1.0 (SSE-rename guards), v1.2.0 (critical-review regression tests, including a default-run headline-Λ regression and PCET angle-criterion tests), v1.2.1 (ORCA reduced-mass reconstruction), v1.2.2 (interpolation grid range) and v1.2.3 (export robustness).

81 Overview: 205 tests

Suite	Tests	Purpose
`test_config.py`	26	Config defaults, TOML parsing/coercion, validation, field inventory.
`test_v104_audit_fixes.py`	24	Regression guards for the FUND findings, half-open windows, plus SCSD sigma-scaling and symmetry-validation tests.
`test_core_physics.py`	23	Thermal amplitude, OOP/INP, kernel metrics, classification.

`test_sanity_physics.py`	19	Physical property tests on u_{rms}, OOP/INP, Δr, λ, RSS, and the HA reaction coordinate – run against the implementation without requiring a Gaussian log.
`test_reorganization.py`	18	Marcus–Hush λ per channel, additivity, scaling, non-negativity.
`test_pcet_et.py`	14	H-bond acceptor detection, exclusions, element filter, and the D–H$\cdots$A angle criterion.
`test_interp_grid_range_v122.py`	14	Interpolation grid spans the requested range: explicit bounds, upper-bound-only $\rightarrow$ 0.0, no-range runs stay data-driven, infinite/degenerate bounds fall back safely, exporter keyword parity.
`test_v123_export_robustness.py`	10	Locked-file fallback of `export._save_workbook`, `runner._window_context` selection, REPORT base directory in multi-window mode.
`test_ca_umap_and_exports.py`	10	Cα-UMAP embedding and Excel/plot export paths.
`test_sse_physics_v105.py`	8	SSE rigid-body decomposition (translation/rotation/internal) against ground truth.
`test_multi_cluster.py`	7	Multi-[2Fe-2S] detection and the per-cluster wrapper.
`test_group_map_indexing.py`, `test_manual_code_refs.py`	12	Group/center mapping; every $\rightsquigarrow$ points to a real function.
`test_regression_reorg.py`	6	Default-run headline regression pinning $\Lambda_{\mathrm{FeFe}}/\Lambda_{\mathrm{FeS}}$ for the reference fixture ($\pm 2\,\%$).
`test_helpers_v35.py`	5	Bend-split invariants and NaN guards.
`test_v110_rename.py`	3	SS$\rightarrow$SSE rename / legacy-alias guards.
`test_orca_reduced_mass.py`	2	ORCA reduced-mass reconstruction $\mu_k = \sum_i m_i \lvert l_{i,k}\rvert^2$ (incl. analytic pure-translation case).
`test_smoke.py` (E2E, `slow`)	4	End-to-end smoke tests on the Cys_4 benchmark log (full pipeline to Excel).
Total	**205**	

82 Regression suite: `test_v104_audit_fixes.py`

19 tests, one per FUND finding plus three FUND-13 sub-tests. Each test is named after the finding it guards (`test_fund_##_`*`description`*). The tests construct minimal synthetic inputs (no Gaussian log required) and assert the post-fix behaviour. A regression in any FUND is detected by the corresponding test.

82.1 Test patterns used in this suite

- *Round-trip*: construct a fake mode, run the full `analyze_mode_with_fallback`, assert the result-dict keys and values match the expected post-fix output.
- *Property-based*: assert that the post-fix formula produces the expected limit value (e.g. FUND 13 synthetic zero must have `number` $= -1$).
- *State-isolation*: assert that the per-cluster reset cleans up between cluster runs (FUND 5 dedup state).

82.2 Per-test specifications

The following table specifies, for each test, what it constructs, what it asserts, and which FUND finding it guards. The construction column shows the key synthetic inputs; the assertion column shows the threshold or invariant; the FUND column refers to the audit-trail section.

Test	Construction & assertion	Protects
`test_his_hn_sigma_scales_with_u_rms`	His ligand stub, $\mathbf{e}_H = (0.1, 0, 0)$, run `analyze_his_hn` once with $u_{\mathrm{rms}} = 1.0$ and once with $u_{\mathrm{rms}} = 0.05$. Assert $\sigma_1 = \sqrt{2} \cdot 5 \times 10^{-4}$ and $\sigma_2 = \sqrt{2} \cdot 5 \times 10^{-4} \cdot 0.05$; ratio $\sigma_2/\sigma_1 = 0.05$ exact to 10^{-10}.	FUND 6 (section 69) – σ should scale with u_{rms}. Pre-fix the ratio was 1.0 instead of 0.05.
`test_his_hn_value_and_sigma_have_same_scaling`	Same setup, value unchanged across two u_{rms} values (1.0 and 0.05). Assert $\lvert\Delta r^{(1)}_{\mathrm{NH}}/\Delta r^{(2)}_{\mathrm{NH}}\rvert = \sigma_1/\sigma_2$. Verifies value and sigma scale *together*.	FUND 6 – consistency of value-and-sigma propagation.
`test_sse_element_sigma_responds_to_sigma_eigvec`	Construct a synthetic SSE-element with 3 atoms, run the export-amplitude function with `sigma_eigvec` $= 10^{-4}$ and then 5×10^{-4}. Assert sigma ratio = 5.	FUND 7 – SSE-element sigmas should be user-configurable.
`test_scsd_sigmas_use_cfg`	Set `sigma_eigvec` = 1e-3, `sigma_coord` = 2e-3; compute SCSD for a synthetic 4-atom cluster; assert $\sigma_{\mathrm{SCSD,ref}} = \sqrt{2} \cdot 2 \times 10^{-3}$ and $\sigma_{\mathrm{SCSD,dist}}$ matches section 15.	FUND 8 – SCSD sigmas built from Config (not hard-coded).
`test_analyze_fe_ligand_warns_but_does_not_raise`	Build a ligand whose `fe_center` doesn't appear in the c2l mapping. Run `analyze_fe_ligand` and capture the warning. Assert exactly one `UserWarning` with the substring "Fe center" is emitted.	FUND 5 – warning is correctly raised on missing center.
`test_analyze_fe_ligand_warning_deduplicated`	Same setup, call the function twice. Assert that only *one* warning is emitted across both calls (the second call should be silent thanks to the `_WARNED_EMPTY_LIG` dedup set).	FUND 5 – warning deduplication via module-level set.

`test_analyze_his_hn_warns_on_protonated_lookup_failure`	Protonated His ligand whose `his_hn_center` is not in c2l. Assert exactly one warning emitted with substring "His-NH".	FUND 5 – warning fires only for protonated case.
`test_analyze_his_hn_does_not_warn_for_deprotonated`	Same topology but `is_protonated = False`. Assert *no* warning emitted (the function early-returns before the lookup).	FUND 5 – warning is gated on protonation status.
`test_ws_fe_bindung_warns_on_all_zero_ligand`	Construct a ligand group whose `c2l` maps map all atoms to indices that lie outside the eigvec array (degenerate group). Assert warning emitted.	FUND 5 – empty-group warning.
`test_ws_fe_bindung_no_warning_when_data_present`	Valid data with 2 atoms inside the eigvec array. Assert no warning.	FUND 5 – false-positive guard for empty-group warning.
`test_ws_his_hn_warns_only_for_protonated_case`	Two His ligands, one protonated, one not. Both have $\mathbf{e}_H = 0$. Assert exactly one warning, fired for the protonated case only.	FUND 5 – selective warning based on protonation.
`test_torsion_loop_robust_to_c2l_gaps`	Build a torsion definition (i, j, k, l) where atom k is missing from c2l. Run the analysis loop; assert it does not raise, returns NaN for the affected torsion, and emits one warning.	General robustness against malformed inputs.
`test_build_pcet_info_warns_on_missing_h_index`	PCET ligand with `his_hn_center` not in c2l. Assert warning substring "H index" present.	FUND 5 + PCET path.
`test_pdb_matching_warns_on_high_ambiguity`	Two PDB atoms within `coord_match_tol` = 1.0 Å of the same Gaussian atom. Assert warning fired ("ambiguous match").	FUND 9 – the tolerance check.
`test_pdb_matching_no_warning_for_low_ambiguity`	Two PDB atoms each at distinct Gaussian atoms. Assert no warning.	FUND 9 – false-positive guard.
`test_window_boundaries_are_half_open`	Build synthetic blocks with modes at exactly $\omega = 100$ and $\omega = 200$ cm^{-1}. Run with `freq_min` = 100, `freq_max` = 200. Assert mode-100 is included, mode-200 is excluded.	General half-open-interval convention.

test_synthetic_zero_mode_has_zero_observables	Call runner._make_synthetic_zero_mode(omega=410.0, n_atoms=10). Assert every observable (a_oop_pct, a_inp_pct, all dr_rss_*, all lambda_*) is exactly 0.0.	FUND 13 (section 74) – synthetic mode is physics-free.
test_synthetic_zero_mode_marked_as_synthetic	Call same function. Assert number = −1, mode_type = "synthetic", precision = "synthetic".	FUND 13 – sentinel values.
test_synthetic_zero_mode_safe_in_interp	Run the full interpolation export with a mix of real modes and one synthetic. Assert the interp curves are continuous and the synthetic mode anchors the right edge at zero.	FUND 13 – end-to-end safety.

82.3 Patterns of synthetic input construction

A typical regression-test setup constructs three things:

1. A *minimal coord_info stub*: a Python class with only the attributes needed by the function under test (e.g. ligands, group_map, atoms_h). The stub does not implement CoordInfo fully – it duck-types only enough to satisfy the function's accesses.
2. A *c2l mapping*: a Python dict from Gaussian centre (1-based) to position in the eigenvector array.
3. A *synthetic eigenvector matrix*: a small NumPy array of shape $(N_{\text{atoms}}, 3)$ that encodes the property under test (e.g. pure stretch, pure translation, zero everywhere except one atom).

This pattern decouples the test from the full pipeline: no log file is read, no PDB matching is performed, no I/O happens. The test runs in milliseconds and exercises only the function under test.

82.4 Pytest collection

All 19 tests in this file are picked up by the default pytest test discovery (function names starting with test_). The file is in the tests/ directory and is included by pyproject.toml's tool.pytest.ini_options testpath. The full file is run by:

```
pytest tests/test_v104_audit_fixes.py -v
```

Runtime: ∼ 1.5 seconds for all 19 tests.

83 Sanity-physics suite: test_sanity_physics.py

↪ core.compute_thermal_amplitude, core.classify_oop_inp, reorganization.signed_dr_along_axis, reorganization.lambda_pair_cr

19 tests on the physical properties of the core formulas. These tests run against the implementation, without using any Gaussian log, by constructing synthetic eigenvectors and geometries that exhibit the property under test. Unlike the regression suite above (which protects against bugs that already happened), these tests protect against *future* formula refactors that would silently break a physical property.

83.1 Per-test specifications: OOP/INP physics

Test	Construction	Assertion
`test_pure_oop_mode_yields_alpha_oop_100`	Cluster $\hat{\mathbf{n}} = (0, 0, 1)$, all 4 atoms with $\mathbf{e}_i = (0, 0, +1)$.	$\alpha_{\mathrm{OOP}} = 100.00\%$ exact.
`test_pure_inp_mode_yields_alpha_oop_0`	Cluster $\hat{\mathbf{n}} = (0, 0, 1)$, all 4 atoms with $\mathbf{e}_i = (1, 0, 0)$ (in-plane breathing).	$\alpha_{\mathrm{OOP}} = 0.00\%$ exact.
`test_mixed_oop_inp_satisfies_pythagorean`	Random eigenvector (seed=42), $\hat{\mathbf{n}} = (0, 0, 1)$.	$\alpha_{\mathrm{OOP}} + \alpha_{\mathrm{INP}} = 1.0$ to $< 10^{-12}$ absolute error.

83.2 Per-test specifications: bond modulation physics

Test	Construction	Assertion
`test_pure_translation_mode_yields_zero_bond_modulation`	Atoms at arbitrary positions; $\mathbf{e}_a = \mathbf{e}_b = (1.0, 0.5, 0)$ (pure translation).	$\Delta r_{ab} = 0$ exact.
`test_pure_rotation_mode_yields_zero_bond_stretch`	Atoms aligned along $\hat{\mathbf{x}}$; $\mathbf{e}_a = (0, +\delta, 0)$, $\mathbf{e}_b = (0, -\delta, 0)$ (rigid in-plane rotation).	$\Delta r_{ab} = 0$ to first order.
`test_pure_bond_stretching_mode_yields_antisymmetric`	Atoms aligned along $\hat{\mathbf{x}}$, separation 2 Å; $\mathbf{e}_a = (+\Delta, 0, 0)$, $\mathbf{e}_b = (-\Delta, 0, 0)$ with $\Delta = 0.05$.	$\Delta r_{ab} = +0.10$ Å exact.

83.3 Per-test specifications: u_{rms} physics

Test	Construction	Assertion
`test_urms_zero_T_limit`	$\omega = 500$ cm^{-1}, $m = 30$ amu, $T = 10^{-3}$ K.	u_{rms} matches $\sqrt{\hbar/(2m\omega)} = 0.0327$ Å to $< 10^{-6}$ relative error.
`test_urms_classical_limit`	$\omega = 50$ cm^{-1}, $m = 30$ amu, $T = 2000$ K ($\hbar\omega/k_{\mathrm{B}}T = 0.036 \ll 1$).	u_{rms} matches $\sqrt{k_{\mathrm{B}}T/(m\omega^2)}$ to $< 1\%$ relative error.

test_urms_monotonic_in_T	$\omega = 100$ cm^{-1}, $m = 30$ amu. Compute $u_{\mathrm{rms}}(T)$ at $T = 1, 10, 100, 300, 1000, 3000$ K.	Strictly increasing in T (assert $u_{\mathrm{rms}}(T_i) < u_{\mathrm{rms}}(T_{i+1})$ for all i).

83.4 Per-test specifications: HA reaction-coordinate physics

These three tests jointly verify the PT/skeletal discriminator property (section 8).

Test	Construction	Assertion
test_ha_pure_acceptor_motion_yields_zero	N–H aligned along $\hat{\mathbf{z}}$ at $(0,0,0)$ and $(0,0,1)$, acceptor A at $(0,0,3)$; $\mathbf{e}_N = \mathbf{e}_H = 0$, $\mathbf{e}_A = (0.05, 0, 0)$.	$\Delta r_{HA} = 0$ exact (no PT character).
test_ha_pure_h_motion_toward_acceptor_yields_positive	Same geometry, $\mathbf{e}_N = \mathbf{e}_A = 0$, $\mathbf{e}_H = (0, 0, +0.1)$ (H moves toward A).	$\Delta r_{HA} = +0.1$ Å (positive sign = PT).
test_ha_pure_n_motion_toward_acceptor_yields_negative	Same geometry, $\mathbf{e}_H = \mathbf{e}_A = 0$, $\mathbf{e}_N = (0, 0, +0.1)$ (N moves toward A, skeletal).	$\Delta r_{HA} = -0.1$ Å (negative sign = skeletal). The naive formula $(\mathbf{e}_H - \mathbf{e}_A)\cdot\hat{\mathbf{n}}_{HA}$ would yield -0.1 Å *with the same sign* as the genuine PT case – this test would fail for that formula.

83.5 Per-test specifications: λ and RSS physics

Test	Construction	Assertion
test_lambda_zero_T_pure_stretching	Cluster pair with $\alpha_X = 1$ (pure stretch along bond), $\omega = 320$ cm^{-1}, $\mu = 31.97$ amu, $T = 10^{-4}$ K.	$\lambda^{\mathrm{pair}} = 80.0$ cm^{-1} $(= \omega/4)$ to $< 10^{-3}$ relative error (equation (37)).
test_lambda_zero_for_zero_displacement	$\Delta r = 0$, any ω, μ.	$\lambda^{\mathrm{pair}} = 0$ exact.
test_lambda_positive_for_any_nonzero_displacement	$\Delta r = \pm 0.01$ Å, $\omega = 200$ cm^{-1}, $\mu = 20$ amu.	λ^{pair} identical for $+0.01$ and -0.01; both positive.
test_lambda_quadratic_in_displacement	Two Δr values $0.01, 0.02$ Å, same ω, μ.	$\lambda(0.02)/\lambda(0.01) = 4.0$ to $< 10^{-10}$.
test_lambda_quadratic_in_frequency	Two ω values $100, 200$ cm^{-1}, same $\Delta r, \mu$.	$\lambda(200)/\lambda(100) = 4.0$.
test_rss_aggregation_single_subchannel	One sub-channel with $\lvert\Delta r\rvert = 0.012$ Å, $w = 1$.	RSS aggregate $= 0.012$ Å exact.

`test_rss_aggregation_two_subchannels_consistency_with_lambda`	Synthetic case with $\lambda = 0.008$, $\Delta r_2 = -0.012$ Å, $w_1 = w_2 = 1$, $\omega = 350$ cm^{-1}, $\mu = 31.97$ amu. Compute $\lambda_{\text{sum}} = \lambda(\Delta r_1) + \lambda(\Delta r_2)$ and λ_{RSS}.	$\lambda_{\text{sum}} = \lambda_{\text{RSS}}$ to $< 10^{-10}$ relative. (Energetic consistency, equation (41).)

83.6 The role of sanity-physics tests

These tests are the longest-lived part of the test suite. They do not protect against any specific past bug; they assert *theorems* of the underlying physics. A future developer who refactors `compute_thermal_amplitude` into a more elegant form, or migrates `aggregate_by_parent` to a NumPy-vectorised implementation, will only know they got it right if these tests keep passing. They are the *contract* between the code and the physics; the regression suite above is the contract between the code and yesterday.

84 Manual-code-ref suite: `test_manual_code_refs.py`

5 tests verifying that every `\codref{module.function}` entry in `Manual.tex` and `Supplement.tex` points to a real function in the source tree, and that the module name is a real module under `src/modenanalyse_2fe2s/`.

84.1 Per-test specifications

Test	What it asserts
`test_manual_en_codrefs_exist`	Every `\codref` entry in `Manual.tex` resolves to a function or class definition somewhere in `src/modenanalyse_2fe2s/<module>.py`. Test fails with a list of unresolved entries.
`test_manual_en_codrefs_nonempty`	Manual contains at least 8 `\codref` entries (current count: 14). Test fails if a future refactor of the Manual accidentally removes the formula-to-code annotations.
`test_manual_en_codrefs_use_known_modules`	All Manual `\codref` module names appear in `src/modenanalyse_2fe2s/` as `.py` files. Catches typos like `cores.foo` (instead of `core.foo`).
`test_supplement_codrefs_exist`	Same as Manual test above, applied to `Supplement.tex` (current count: 93). Skipped if `Supplement.tex` is absent.
`test_supplement_codrefs_use_known_modules`	Same as Manual module test, applied to `Supplement.tex`.

84.2 Why this matters

Without these tests, a refactor that renames a function (e.g. `logio.get_eigvec_orca` → `orca_io.get_eigvec_orca`, which actually happened during the v1.0.4 audit) would silently leave the manual referring to a non-existent function. The reader of the manual would chase a phantom reference.

The tests also act as a forcing function: every time a function is renamed or moved, the developer is reminded – via test failure – that the documentation needs updating in lock-step. This is the same principle as a type-checker: a structural error becomes mechanically detectable rather than relying on human discipline.

84.3 Implementation: `_extract_codrefs` and `_function_exists`

The tests share two private helpers. `_extract_codrefs(tex_path)` runs a regex over the TeX source to find all `\codref{...}` occurrences, splits comma- separated entries, unescapes LaTeX `\_` back to `_`, and returns a list of `(module, function)` pairs. LaTeX comment lines (starting with `%`) are skipped so the macro's own definition (which contains an example `\codref{module.function}`) is not counted.

`_function_exists(module, name)` reads `src/modenanalyse_2fe2s/{module}.py` and searches with four patterns: `def {name}(`, `async def {name}(`, `class {name}`, and the private-prefix variant `def _{name}(` (for cases where the manual references the public name but the implementation uses a private prefix).

85 Legacy unit suites

The legacy unit suites cover individual modules and predate the v1.0.4 audit. They are kept running to maintain coverage of the older code paths.

File	Tests	Coverage
`test_config.py`	∼12	TOML round-trip, default values, field validation.
`test_core_physics.py`	∼15	Pre-v1.0.4 physics tests; partly subsumed by `test_sanity_physics.py`.
`test_reorganization.py`	∼18	Per-mode λ_X computations on hand-crafted geometries.
`test_pcet_et.py`	∼8	PCET acceptor-search heuristic edge cases.
`test_multi_cluster.py`	∼10	Cluster discovery and isolation between cluster runs.
`test_group_map_indexing.py`	∼6	Group atom-index resolution and gaps.
`test_helpers_v35.py`	∼8	Helper functions from the v0.3.5 era.
`test_ca_umap_and_exports.py`	∼10	UMAP embedding and Excel export shape tests.
`test_smoke.py`	4	End-to-end smoke runs on the bundled small test log.

The legacy suites do not have per-test documentation in this supplement; they are tested as a group and serve as additional regression coverage. Each test is self-documenting via its docstring – a reviewer interested in any specific test can read the source directly.

86 End-to-end suite

4 E2E tests run the full pipeline on small synthetic systems and assert observable invariants:

Test	What it does

`test_e2e_4fe_dummy_cys4`	Run pipeline on a 4-Fe + 4-Cys-S synthetic log. Verifies that the pipeline produces the expected workbook structure (sheet names, columns), executes Phase 4 over all modes, and writes non-empty outputs.
`test_e2e_4fe_rieske_cys2his2`	2-Cys + 2-His ligation. Verifies that the NH and HA sheets populate only for the His ligands; verifies that the coord_info correctly identifies both protonated histidines.
`test_e2e_two_clusters`	2 independent [2Fe-2S] clusters in one Gaussian log. Verifies multi-cluster dispatch, per-cluster output directories, and correct isolation of the per-cluster warning-dedup state.
`test_e2e_terminal_spectrum_fund_13`	Log whose highest mode falls inside the analysis window. Verifies that FUND 13 synthetic-zero anchor is correctly inserted and that the interpolation workbook is well-defined at the right edge.

E2E tests are marked with the `pytest.mark.e2e` marker and are *not* run by the default `pytest` invocation (because they take several seconds each). They are run explicitly via:

```
pytest tests/ -m e2e -v
```

Part X

Validation Strategy

This part documents the validation strategy of the pipeline – the external (experimental and literature) data against which the pipeline's headline numbers can be compared, the DFT setup used to produce the benchmark inputs, and the explicit acceptance tolerances per benchmark quantity.

Scope of applicability. The pipeline targets any [2Fe-2S] cluster with Cys/His coordination, covering the major biological subclasses:

- Ferredoxin-type Cys_4 coordination (plant-type and bacterial-type ferredoxins; the H87C variant of mitoNEET is a synthetic example).
- Rieske-type Cys_2His_2 coordination (Rieske of cytochrome bc_1 and bc_6f, ARH1, BphA1, IscR, Apd1).
- mitoNEET-type Cys_3His coordination (mitoNEET, NAF-1, MiNT, IscA, BolA-like).

For *all* subclasses, the analysis observables (OOP/INP, Δr_X per channel, λ_X^{pair}, $M_X(\omega)$, SCSD irreps, B-factors, embedding) are computed by the same code path; only the ligand-specific channels populate or remain empty depending on which N/S coordination is present (NH and HA are skipped for Cys_4, FeN is skipped if no His-N coordination exists, etc.).

Choice of benchmark systems below. The DFT setups and acceptance tolerances in sections 87 and 88 cover three example systems that span the three coordination classes above (Apd1 in two protonation states, and the synthetic mitoNEET-H87C Cys_4 variant). These are *example validation systems*, not the only systems for which the pipeline is intended. Any other [2Fe-2S]-Cys/His system can be analysed without

modification; for such systems, the headline numbers below provide order-of-magnitude expectations rather than strict acceptance bounds.

87 DFT calculation setup for benchmark logs

The benchmark Gaussian logs distributed with the package were produced with the following setup. A reviewer who wishes to regenerate them can use these settings; for other [2Fe-2S] systems, the same setup parameters (functional, basis, charge adjusted to ligation, broken-symmetry spin state) are a reasonable default starting point.

87.1 Example 1: Apd1 wild-type (Cys_2His_2, reduced [2Fe-2S]$^{1+}$, $S = 1/2$)

Setting	Value
Software	Gaussian 16 Revision C.02
Functional	B3LYP-D3 (with Becke-Johnson damping)
Basis set, Fe	TZVP (Ahlrichs)
Basis set, S	TZVP with diffuse functions
Basis set, others	6-31G(d,p)
Charge	-2 ($[Fe_2S_2(SCys)_4]^{2-}$ reduced form)
Spin	broken-symmetry singlet ($S_z = 0$ with antiferromagnetic Fe-Fe coupling)
SCF convergence	10^{-8} Ha density
Geometry	QM/MM-optimised; QM region = cluster + coordinating Cys sidechains + 6 Å of backbone; MM region = remainder of protein with TIP3P water
Frequency option	`Freq=hpmodes`
Temperature	Thermochemistry at $T = 80$ K (cryogenic NRVS condition)
Print orientation	Standard orientation (default)

The frequency output contains $3N - 6 = 1260$ modes for $N = 422$ QM atoms. The file size is approximately 200 MiB; the pipeline processes it in ~ 3 minutes on a 2024-era laptop CPU.

87.2 Example 2: Apd1 protonated (μS-protonated Cys_2His_2, $S = 0$)

Same setup as above except:

- Charge $= -1$ (one of the bridging μ-S atoms carries an extra H^+).
- Spin: closed-shell singlet (the protonation removes the antiferromagnetic coupling).
- Geometry: as a separate QM/MM-optimised structure, since protonation re-organises the cluster.

87.3 Example 3: mitoNEET H87C variant (Cys_4, oxidised [2Fe-2S]$^{2+}$)

Setting	Value
Software	ORCA 5.0.4 (yields `.hess` file)
Functional	TPSSh
Basis set	def2-TZVP everywhere
Charge	-2 ($[Fe_2S_2(SCys)_4]^{2-}$ oxidised)
Spin	broken-symmetry singlet
Reference	The mutated mitoNEET structure from [12], with His87 mutated to Cys in silico.

This system uses the ORCA `orca_io.py` parser (section 23 counterpart). The pipeline path is identical from Phase 4 onward.

88 Headline numbers per benchmark system with acceptance tolerances

The following tables specify expected values *and* acceptance tolerances for our three example validation systems. A pipeline release "passes validation" on a benchmark system if every entry in its table is matched within the stated tolerance. The tolerances are chosen to be 2-3σ of the DFT-vs-experiment systematic uncertainty.

Convention. All headline Λ^{total} values are reported in the *pair-μ convention* (`Lambda_total_pair_cm1` column in the `Reorg_total` sheet), which uses the reduced mass of the underlying bond ($\mu_{ab} = m_a m_b/(m_a + m_b)$, e.g. $\mu_{\text{Fe-S}} \approx 20.4$ amu) for each per-mode contribution. This is the convention closest to the spectroscopic reorganization energies reported in the NRVS+DFT literature [8, 9]. The pipeline additionally reports a *mode-μ convention* (`Lambda_total_mode_cm1` column) using the Gaussian-printed reduced mass of the normal mode itself; this gives values typically a factor 5–10 smaller and is useful for cross-checks against vibrational-energy budgets but is not the quantity meant by "the reorganization energy of bond X".

Use of these numbers for other systems. The numerical expectations below are calibrated for our three specific benchmark inputs (analysis window 0–500 cm^{-1}, $T = 5$ K, B3LYP-D3/TZVP-Ahlrichs/QM-MM, broken-symmetry singlet) and should *not* be treated as universal acceptance criteria for arbitrary [2Fe-2S]-Cys/His systems. A user analysing a different protein (e.g. a plant-type Cys_4 ferredoxin, the native mitoNEET Cys_3His ligation, or a non-standard μ-SH state) should expect *order-of-magnitude* agreement with the appropriate coordination class, not exact match. The structural patterns that *must* hold across all systems (cluster planarity, $\Lambda_{\text{NH}} = 0$ for unprotonated states, α_{OOP} near 100% for true umbrella modes, etc.) are noted explicitly in the tables.

Sensitivity to window and temperature. The Λ^{total} values are window-dependent (they accumulate only modes inside the analysis window) and weakly temperature-dependent (via the coth factor in u_{rms}). For the 0–500 cm^{-1} window at $T = 5$ K the values are zero-point-dominated and within 1% of their $T = 0$ limit. A full-spectrum analysis (window $0 - \omega_{\text{max}}^{\text{DFT}}$) would give substantially *larger* values for the protonated case (the N-H stretch at ~ 3300 cm^{-1} contributes strongly to Λ_{NH}); for the deprot/Cys_4 cases the increase is smaller because no high-frequency channel-active mode is missed.

Λ in cm^{-1} is an *energy*, not a frequency bound. A common point of confusion: $\Lambda^{\text{total}}_{\text{FeS}} = 555$ cm^{-1} for an analysis run with `freq_max` $= 500$ cm^{-1} may look paradoxical ("how can Λ exceed the cut-off?"). The answer is that Λ^{total} is not a frequency at all: it is the *cumulative reorganization energy*, expressed in spectroscopic cm^{-1} units (where 1 cm^{-1} $= 1.986 \times 10^{-23}$ J $\equiv 0.124$ meV). It is built up as

$$\Lambda^{\text{total}}_{X} \;=\; \sum_{i=1}^{N_{\text{modes in window}}} \tfrac{1}{2}\,\mu_X\,\omega_i^2\,(\Delta r_i)^2,$$

i.e. a sum over all modes in the window. Even though no single mode contributes more than ~ 20 cm^{-1}, the sum over ~ 5900 modes easily reaches several hundred cm^{-1}. The analogue is total kinetic energy in classical mechanics: if every particle has at most k_BT of kinetic energy, the system total is $\frac{3}{2}Nk_BT$, which can vastly exceed k_BT. Concretely for Apd1_deprot: the strongest single-mode FeS contribution is 18.3 cm^{-1} at 433 cm^{-1}, the average is 0.09 cm^{-1}, summing over 5931 contributing modes gives $\Lambda^{\text{total}}_{\text{FeS}} \approx 555$ cm^{-1}.

88.1 Example 1: Apd1 wild-type (Cys_2His_2, deprotonated)

Calibrated against an actual production run (Apd1_deprot, $3N - 6 \approx 18900$ modes, analysis window 0–500 cm^{-1}, $T = 5$ K, B3LYP-D3/TZVP/QM-MM, ∼5958 modes inside the window).

Quantity (pair-μ convention)	Expected	Tol.	Source / rationale
$\Lambda^{\text{total}}_{\text{FeS}}$	555 cm^{-1}	±50 cm^{-1}	Production-run observation, summed over 6 Fe-S sub-bonds (2 Cys terminal + 4 bridging). Literature comparable values $\sim$ 320–360 cm^{-1} from [8] (different DFT setup, smaller cluster model).
$\Lambda^{\text{total}}_{\text{FeFe}}$	70 cm^{-1}	±15 cm^{-1}	Single Fe-Fe bond contribution. The cumulative $\Lambda(\omega)$ should saturate by $\omega \approx 380$ cm^{-1}.
$\Lambda^{\text{total}}_{\text{FeN}}$	48 cm^{-1}	±15 cm^{-1}	2 Fe-N(His) sub-bonds, comparable magnitudes (~ 24 each).
$\Lambda^{\text{total}}_{\text{NH}}$	0	exact	Deprotonated WT has no N-H bonds.
$\Lambda^{\text{total}}_{\text{HA}}$	0	exact	No protonated donor ⇒ HA channel inactive.
α_{OOP} for the cluster-umbrella mode (~ 80 cm^{-1})	$\geq 92\%$	–	Property of the A_u irrep.
Cluster planarity σ_3/σ_1	≤ 0.02	–	Rhombic Fe_2S_2 is genuinely planar.

88.2 Example 2: Apd1 protonated (μS-protonated Cys_2His_2)

Calibrated against an actual production run (Apd1_prot, analysis window 0–500 cm^{-1}, $T = 5$ K, B3LYP-D3/TZVP/QM-MM, ∼5941 modes inside the window).

Quantity (pair-μ convention)	Expected	Tol.	Source / rationale
$\Lambda_{\mathrm{FeS}}^{\mathrm{total}}$	570 cm^{-1}	± 50 cm^{-1}	Production-run observation, sum over 6 Fe-S sub-bonds. Slightly higher than the deprot state (555 cm^{-1}) due to protonation-induced bond stiffening.
$\Lambda_{\mathrm{FeFe}}^{\mathrm{total}}$	64 cm^{-1}	± 15 cm^{-1}	Lower than deprot WT (70 cm^{-1}) because protonation breaks the antiferromagnetic Fe-Fe coupling.
$\Lambda_{\mathrm{FeN}}^{\mathrm{total}}$	58 cm^{-1}	± 15 cm^{-1}	Higher than deprot WT (48 cm^{-1}), consistent with stiffer Fe-N(His) geometry in the protonated state.
$\Lambda_{\mathrm{NH}}^{\mathrm{total}}$ (in 0–500 cm^{-1} window)	$\lesssim 0.05$ cm^{-1}	–	The full N-H stretch sits at $\sim$3300 cm^{-1}, far outside the analysis window. Only low-frequency librations of the N-H vector contribute; the value is therefore numerically very small.
$\Lambda_{\mathrm{HA}}^{\mathrm{total}}$ (in 0–500 cm^{-1} window)	10–25 cm^{-1}	± 5 cm^{-1}	PCET signature: the H$\cdots$A reaction coordinate modulates with frequencies below 500 cm^{-1}, dominated by one His (here His259 with $\Lambda_{\mathrm{HA}} \approx 11$ cm^{-1} pair-μ or 22 cm^{-1} mode-μ); the second His contributes < 1 cm^{-1}.

PCET diagnostic. The asymmetric distribution between the two histidines (His259 carries $> 90\%$ of Λ_{HA}, His255 carries $< 10\%$) is the expected PCET fingerprint: only one of the two H-bond acceptor environments is geometrically favourable for the proton transfer. The cumulative $\Lambda_{\mathrm{HA}}(\omega)$ curve should show a step in the $\sim$ 200–320 cm^{-1} range where the μ-S-H wagging modes are active.

88.3 Example 3: Cys_4 system (e.g. mitoNEET H87C or Apd1 H255C/H259C)

Calibrated against a Cys_4 production run (Apd1_HH255_259CC_deprot, 4 Cys ligands, no His, analysis window 0–500 cm^{-1}, $T = 5$ K, B3LYP-D3/TZVP/QM-MM, $\sim$5931 modes inside the window).

Quantity (pair-μ convention)	Expected	Tol.	Source / rationale
$\Lambda_{\mathrm{FeN}}^{\mathrm{total}}$	0	exact	Cys_4 has no His ligand; FeN channel is structurally absent.

$\Lambda_{\mathrm{FeS}}^{\mathrm{total}}$	715 cm^{-1}	± 60 cm^{-1}	*Higher* than the Cys_2His_2 cases ($\sim$ 555–570 cm^{-1}) because replacing 2 Fe-N coordinations with 2 additional Fe-S coordinations redistributes the cluster reorganization energy into the FeS channel.
$\Lambda_{\mathrm{FeFe}}^{\mathrm{total}}$	73 cm^{-1}	± 15 cm^{-1}	Comparable to the Cys_2His_2 cases (single Fe-Fe bond, ligand-environment- independent to first order).
$\Lambda_{\mathrm{NH}}^{\mathrm{total}}$	0	exact	No protonated His; NH channel absent.
$\Lambda_{\mathrm{HA}}^{\mathrm{total}}$	0	exact	No PCET donor in Cys_4.
Number of detected clusters	1	exact	Single [2Fe-2S].
Cluster planarity σ_3/σ_1	≤ 0.03	–	Cys_4 systems are typically slightly *less* planar than Cys_2His_2 (folding residual ~ 0.025 Å in the production run vs 0.009 Å for the WT).

88.4 The headline-number test (acceptance criterion)

A pipeline release is considered to "pass validation" on a given benchmark system if all numerical entries above are met within their tolerances. The procedure is identical for each of the three example systems; below we illustrate it for the Cys_2His_2 wild-type case:

```
# Run pipeline on the canonical Cys2His2 deprotonated input
import modenanalyse_2fe2s as mna
result = mna.run_analysis(cfg)

# Read the headline values from the Reorg_total sheet
# (columns: 'Channel', 'Lambda_total_pair_cm1', 'Lambda_total_mode_cm1',
#  'n_modes_contributing'); each row corresponds to one channel.
import openpyxl
wb = openpyxl.load_workbook(result.main_xlsx_path)
ws = wb["Reorg_total"]
lambdas = {row[0]: row[1] for row in ws.iter_rows(min_row=2, values_only=True)}

# Check tolerances against the Cys2His2-WT (deprotonated) reference values
assert abs(lambdas["FeS"]  - 555) <= 50,  f"FeS  off: {lambdas['FeS']}"
assert abs(lambdas["FeFe"] -  70) <= 15,  f"FeFe off: {lambdas['FeFe']}"
assert abs(lambdas["FeN"]  -  48) <= 15,  f"FeN  off: {lambdas['FeN']}"
assert lambdas["NH"] == 0.0   # deprotonated WT has no N-H
assert lambdas["HA"] == 0.0   # no PCET donor
```

This procedure is run as part of the E2E test suite (part IX) before every release.

89 Reproducibility

89.1 Determinism guarantees

With `n_jobs` = 1 (default), every run produces bit- identical output for the same Gaussian log, PDB, and Config. This is enforced by avoiding all sources of non-determinism in the analysis pipeline:

- Python ≥ 3.7 preserves dict insertion order, so iteration over the per-mode result dict is deterministic.
- No hash-based set iteration is used in the analysis pipeline – where a set is needed for deduplication (e.g. `_WARNED_EMPTY_LIG`), the set is only used for `in`-checks, never iterated.
- NumPy operations are deterministic by default on a fixed BLAS configuration. We do not use any `np.random` operations in the pipeline (only in the UMAP embedding, which has a fixed seed; see below).
- Floating-point summations are done in a fixed order (sum over modes is in ascending frequency order; sum over atoms is in Gaussian-centre order).

89.2 UMAP and HDBSCAN determinism

The UMAP embedding uses a fixed `random_state = 42` for reproducibility. With this seed, the umap-learn library produces deterministic output across runs (provided the input feature matrix is identical and the umap-learn version is identical).

Version sensitivity. UMAP's internal algorithm has changed between umap-learn versions: 0.4.x, 0.5.x, and (when released) 0.6.x will produce different embeddings even with the same seed and inputs. We pin `umap-learn == 0.5.5` in `pyproject.toml` to avoid this. A reviewer can verify the version with:

```
import umap; print(umap.__version__)   # should be 0.5.5
```

The HDBSCAN clustering is deterministic given a fixed UMAP output, so once UMAP is pinned, the cluster assignments are reproducible.

89.3 Cache integrity

Cache invalidation by mtime + file-size + cache-format version (section 2.8) ensures that updating the source Gaussian log or upgrading the package version triggers a re-scan. The cache key is robust against:

- File replacement (mtime changes).
- File extension (the cache key is keyed by absolute path).
- Python major-version upgrades (cache key includes Python version).
- Cache-format changes within the package (cache key includes `logio._CACHE_VERSION`, bumped manually on format changes).

89.4 Version pinning

The exact dependency versions used to produce the example benchmark results above are pinned in `pyproject.toml`:

Package	Version	Determinism note
`numpy`	≥ 1.24	BLAS configuration affects bit- level reproducibility across machines (Intel MKL vs OpenBLAS can differ in the last bit).
`scipy`	≥ 1.10	Used for SVD; same BLAS considerations apply.
`openpyxl`	≥ 3.1	Output format only.
`scikit-learn`	≥ 1.3	Not used directly; pulled in by umap-learn.
`umap-learn`	$= 0.5.5$	Hard-pinned for embedding reproducibility.
`hdbscan`	$= 0.8.40$	Hard-pinned.

89.5 Floating-point reproducibility caveats

Bit-identical output across machines requires the same BLAS configuration. In practice we observe:

- Linux x86-64 (Intel MKL, Anaconda): bit-identical across Intel and AMD CPUs of the same generation.
- Linux x86-64 (OpenBLAS): differs from MKL in the last bit of summations involving long sequences (> 100 terms).
- macOS arm64 (Apple Accelerate): differs from x86-64 in accumulated SVD operations beyond the last 2-3 bits.

For the validation acceptance criteria above, the tolerances are chosen to be far larger than any BLAS-induced differences (typical floating-point differences are at the 10^{-12} level; tolerances are at the cm^{-1} level, i.e. relative 10^{-2}).

89.6 Logging and audit trail

Every run produces a `run-log.txt` that records:

- Package version and dependency versions at run time.
- Path and mtime of the Gaussian log.
- Cache hit or miss.
- Number of modes scanned, selected, and analysed.
- All warnings emitted by the analysis (deduplication notwithstanding).
- FUND-12 and FUND-13 fallbacks if triggered.
- Final headline numbers (a copy of the `Reorganisation_total` row).

A reviewer auditing a pipeline output should consult the run-log first – it contains all the information needed to reconstruct the pipeline's decisions on a given input.

Part XI

References and Glossary

Bibliography

The references are grouped thematically: (i) vibrational and quantum-mechanical foundations, (ii) NRVS theory and experimental work on iron-sulfur proteins, (iii) Marcus-Hush electron-transfer theory, (iv) proton-coupled electron transfer (PCET) theory and biology, (v) symmetry-coordinate decomposition and group theory, (vi) geometry algorithms and structural-biology data structures, (vii) dimension-reduction and clustering methods.

References

[1] E. B. Wilson, J. C. Decius, P. C. Cross, *Molecular Vibrations: The Theory of Infrared and Raman Vibrational Spectra*, McGraw-Hill, New York, 1955. The canonical reference for the harmonic-oscillator basis of vibrational spectroscopy and for the position-variance derivation underlying equation (8).

[2] L. D. Landau, E. M. Lifshitz, *Quantum Mechanics: Non-Relativistic Theory* (3rd ed.), Pergamon Press, Oxford, 1977. § 23 derives $\langle q^2 \rangle_T$ for the canonical-ensemble quantum harmonic oscillator.

[3] D. A. McQuarrie, *Statistical Mechanics*, University Science Books, 2000. Chapter 6 derives the harmonic-oscillator partition function and thermal averages used in section 4.

[4] E. B. Wilson, J. C. Decius, P. C. Cross, *Decius rules for normal-mode analysis under symmetry*, in *Annual Review of Physical Chemistry* **31**, 1980. Foundational discussion of how irrep decomposition organises normal-mode amplitudes.

[5] K. Achterhold, C. Keppler, A. Ostermann, U. van Bürck, W. Sturhahn, E. E. Alp, F. G. Parak, *Vibrational dynamics of myoglobin determined by the phonon- assisted Mößbauer effect*, **Phys. Rev. E 65**, 051916 (2002). Foundational application of NRVS to an Fe-protein system.

[6] W. Sturhahn, *Nuclear resonant spectroscopy*, **J. Phys.: Condens. Matter 16**, S497–S530 (2004). Theory of NRVS and connection to the Fe-projected partial density of states (pDOS).

[7] C. J. Kingsbury, F. Neese, M. O. Senge, *The Fe-projected vibrational density of states from nuclear-resonance vibrational spectroscopy and quantum chemistry*, **J. Chem. Theory Comput. 20**, 7212–7228 (2024). Recent benchmark study connecting computed Fe-projected pDOS to NRVS spectra of [2Fe-2S] proteins.

[8] L. B. Gee, C. Y. Lin, F. E. Jenney Jr., M. W. W. Adams, Y. Yoda, R. Masuda, T. Mitsui, M. Hu, J. Zhao, J. T. Sage, S. P. Cramer, *Comparison of Iron Hydrogenase Vibrational Spectra to those of [2Fe-2S] Model Compounds via NRVS and DFT*, **Inorg. Chem. 60**, 9714–9727 (2021). Cluster-vibration assignments in the 200–420 cm^{-1} window used as benchmark for the Cys_2His_2 wild-type validation (part X).

[9] D. Mitra, V. Pelmenschikov, Y. Guo, D. A. Case, H. Wang, W. Dong, M.-L. Tan, T. Ichiye, F. E. Jenney Jr., M. W. W. Adams, Y. Yoda, J. Zhao, S. P. Cramer, *Dynamics of the [4Fe-4S] cluster in*

Pyrococcus furiosus *ferredoxin via NRVS and DFT*, **Biochemistry 50**, 5220–5235 (2017). Comprehensive NRVS/DFT comparison for iron-sulfur cluster modes.

[10] C. E. Tinberg, R. Y. Tonzetich, H. Wang, L. H. Do, Y. Yoda, S. P. Cramer, S. J. Lippard, *Characterization of iron dinitrosyl species formed in the reaction of nitric oxide with a biological Rieske center*, **J. Am. Chem. Soc. 130**, 14550–14551 (2008). Rieske [2Fe-2S] benchmark.

[11] S. Iwata, M. Saynovits, T. A. Link, H. Michel, *Structure of a water soluble fragment of the 'Rieske' iron-sulfur protein of the bovine heart mitochondrial cytochrome bc_1 complex determined by MAD phasing at 1.5 Å resolution*, **Structure 4**, 567–579 (1996). The structural reference for the Cys_2His_2 Rieske coordination mode.

[12] M. L. Paddock, S. E. Wiley, A. T. Axelrod, A. K. Cohen, M. Roy, E. C. Abresch, D. Capraro, A. N. Murphy, R. Nechushtai, J. E. Dixon, P. A. Jennings, *MitoNEET is a uniquely folded 2Fe-2S outer mitochondrial membrane protein stabilized by pioglitazone*, **Proc. Natl. Acad. Sci. USA 104**, 14342–14347 (2007). The Cys_3His Mn coordination of mitoNEET used as a benchmark system.

[13] R. A. Marcus, *On the theory of oxidation-reduction reactions involving electron transfer. I.*, **J. Chem. Phys. 24**, 966–978 (1956). The foundational Marcus paper. Introduces the reorganization- energy concept that equation (35) specialises to a single vibrational mode.

[14] R. A. Marcus, *Chemical and electrochemical electron-transfer theory*, **Annu. Rev. Phys. Chem. 15**, 155–196 (1964).

[15] R. A. Marcus, N. Sutin, *Electron transfers in chemistry and biology*, **Biochim. Biophys. Acta 811**, 265–322 (1985). Definitive review covering the biological context of intramolecular ET. The inner-sphere reorganization λ_i is the closest analogue to the spectroscopic $\Lambda_X^{\mathrm{total}}$ computed by this code.

[16] N. S. Hush, *Adiabatic rate processes at electrodes*, **J. Chem. Phys. 28**, 962–972 (1961). Hush's analogous derivation; the Marcus-Hush nomenclature.

[17] J. J. Hopfield, *Electron transfer between biological molecules by thermally activated tunneling*, **Proc. Natl. Acad. Sci. USA 71**, 3640–3644 (1974). Quantum vibrational version of the Marcus rate including Huang-Rhys factors – the link from this code's Λ_X to biological ET rates.

[18] J. Jortner, *Temperature dependent activation energy for electron transfer between biological molecules*, **J. Chem. Phys. 64**, 4860–4867 (1976). Theoretical extension covering the quantum regime $\hbar\omega \gg k_{\mathrm{B}}T$ relevant for low-temperature ET in iron-sulfur proteins.

[19] S. Hammes-Schiffer, A. A. Stuchebrukhov, *Theory of coupled electron and proton transfer reactions*, **Chem. Rev. 110**, 6939–6960 (2010). Comprehensive PCET-theory review. The HA reaction-coordinate formula (equation (22)) is derived in the spirit of § III.B.

[20] D. R. Cukier, D. G. Nocera, *Proton-coupled electron transfer*, **Annu. Rev. Phys. Chem. 49**, 337–369 (1998). The earlier conceptual framework that separates PT and ET reaction coordinates – the basis for our HA-vs-FeFe co-modulation descriptor.

[21] S. Y. Reece, D. G. Nocera, *Proton-coupled electron transfer in biology: results from synergistic studies in natural and model systems*, **Annu. Rev. Biochem. 78**, 673–699 (2009). PCET in iron-sulfur and other metalloproteins.

[22] J. Hirst, *Mitochondrial complex I*, **Annu. Rev. Biochem. 82**, 551–575 (2013). PCET-driven ET via [2Fe-2S] and [4Fe-4S] clusters in complex I.

[23] C. J. Kingsbury, M. O. Senge, *Quantifying near-symmetric molecular distortion using symmetry-coordinate structural decomposition*, **Chem. Sci. 15**, 13638–13649 (2024). DOI: 10.1039/D4SC01670J. Web implementation: `https://www.kingsbury.id.au/scsd`. The reference for equation (52).

[24] C. J. Kingsbury, M. O. Senge, *Quantifying near-symmetric molecular distortion using symmetry-coordinate structural decomposition*, ChemRxiv preprint, doi:10.26434/chemrxiv-2024-6b25s (2024).

[25] C. J. Bradley, A. P. Cracknell, *The Mathematical Theory of Symmetry in Solids*, Clarendon Press, Oxford, 1972. Group-theoretic reference for the D_{2h} character table and irrep notation used in section 15.

[26] F. A. Cotton, *Chemical Applications of Group Theory* (3rd ed.), Wiley, New York, 1990. Practitioner-friendly reference for normal-mode irreps in molecular spectroscopy.

[27] W. Kabsch, *A solution for the best rotation to relate two sets of vectors*, **Acta Cryst. A32**, 922–923 (1976). The SVD-based optimal-rotation algorithm used in section 25.

[28] W. Kabsch, *A discussion of the solution for the best rotation to relate two sets of vectors*, **Acta Cryst. A34**, 827–828 (1978). Follow-up addressing the chirality sign-correction.

[29] W. Kabsch, C. Sander, *Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features*, **Biopolymers 22**, 2577–2637 (1983). The DSSP algorithm referenced in section 26.

[30] G. N. Ramachandran, C. Ramakrishnan, V. Sasisekharan, *Stereochemistry of polypeptide chain configurations*, **J. Mol. Biol. 7**, 95–99 (1963). The ϕ/ψ classification used in the SSE-detection fallback.

[31] G. H. Golub, C. F. Van Loan, *Matrix Computations* (4th ed.), Johns Hopkins University Press, 2013. Reference for the SVD algorithms used throughout.

[32] K. Pearson, *On lines and planes of closest fit to systems of points in space*, **Philos. Mag. 2**, 559–572 (1901). PCA, used as a baseline reduction technique in the embedding workbook.

[33] L. van der Maaten, G. Hinton, *Visualizing data using t-SNE*, **J. Mach. Learn. Res. 9**, 2579–2605 (2008).

[34] L. McInnes, J. Healy, J. Melville, *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*, arXiv:1802.03426 (2018).

[35] E. Becht, L. McInnes, J. Healy, C.-A. Dutertre, I. W. H. Kwok, L. G. Ng, F. Ginhoux, E. W. Newell, *Dimensionality reduction for visualizing single-cell data using UMAP*, **Nat. Biotechnol. 37**, 38–44 (2019).

[36] R. J. G. B. Campello, D. Moulavi, J. Sander, *Density-based clustering based on hierarchical density estimates*, in *Advances in Knowledge Discovery and Data Mining*, LNCS **7819**, 160–172 (2013). The HDBSCAN algorithm.

[37] C. R. Harris et al., *Array programming with NumPy*, **Nature 585**, 357–362 (2020).

[38] P. Virtanen et al., *SciPy 1.0: fundamental algorithms for scientific computing in Python*, **Nat. Methods 17**, 261–272 (2020).

[39] J. D. Hunter, *Matplotlib: A 2D graphics environment*, **Comput. Sci. Eng. 9**, 90–95 (2007).

[40] *openpyxl – A Python library to read/write Excel 2010 xlsx files*, `https://openpyxl.readthedocs.io`. The output backend used by `export.py`.

[41] M. J. Frisch et al., *Gaussian 16*, Revisions A.03–C.02, Gaussian, Inc., Wallingford CT, 2016. The primary frequency-job source. The `Freq=hpmodes` option produces the 5-decimal eigenvectors that the parser expects.

[42] F. Neese, *Software update: the ORCA program system, version 5.0*, **WIREs Comput. Mol. Sci. 12**, e1606 (2022). The alternative input format (`.hess`) handled by `orca_io.py`.

Symbol glossary

Physical and mathematical symbols

Symbol	Units	Meaning
$\hbar$	J s	Reduced Planck constant.
k_{B}	J/K	Boltzmann constant.
hc	J cm	$1.986\,446 \times 10^{-23}$ J cm; used for cm^{-1}-to-Joule conversion.
T	K	Temperature. Defaults to none (classical mode); for production analysis, set via `temp_k`.
$\omega_l, \tilde{\nu}_l$	rad/s, cm^{-1}	Angular and wavenumber frequency of mode l.
$m_l, m_{\mathrm{red}}, \mu$	amu, kg	Reduced mass: `red_mass` in the code.
$\mathbf{e}_{l,i}$	dimensionless	Cartesian-displacement eigenvector of mode l on atom i.
$\widetilde{\mathbf{e}}_{l,i}$	Å	Thermally scaled eigenvector, $\mathbf{e}_{l,i} \cdot u_{\mathrm{rms}}(\omega_l)$.
$u_{\mathrm{rms}}(\omega, m, T)$	Å	QHO thermal RMS amplitude, equation (8).
$\hat{\mathbf{n}}$	dimensionless	Unit cluster-normal, equation (16).
$\hat{\mathbf{b}}$	dimensionless	Unit bond vector.
$\alpha_{\mathrm{OOP}}, \alpha_{\mathrm{INP}}$	%	OOP/INP fractions, equation (12).
Δr_X	Å	Bond modulation along channel X, equation (21) or equation (22).
$\Delta r_X^{\mathrm{RSS}}$	Å	RSS aggregate over sub-channels, equation (38).
$\lambda_X(\mathrm{mode})$	cm^{-1}	Per-mode Marcus-Hush reorganization, equation (43).
$\Lambda_X^{\mathrm{total}}$	cm^{-1}	System-wide total, equation (44).
$\Lambda_X(\omega)$	cm^{-1}	Cumulative-Lambda curve, equation (45).
$M_X(\omega)$	Å	Modulation spectrum, equation (46).
$C_X(\omega)$	Å	Co-modulation spectrum, equation (50).
B_i	Å^2	Debye-Waller / B-factor for atom i, equation (60).
σ_e	dimensionless	Per-component eigenvector noise, `sigma_eigvec`.
σ_c	Å	Per-component coordinate noise, `sigma_coord`.
$\sigma_e^{\mathrm{therm}}$	Å	$\sigma_e \cdot u_{\mathrm{rms}}$, equation (80).

Subscripts and superscripts

Notation	Meaning
$X \in \{\mathrm{FeFe}, \mathrm{FeN}, \mathrm{FeS}, \mathrm{NH}, \mathrm{HA}\}$	The five Marcus-Hush channels (electron-transfer reaction coordinate, two metal- ligand stretches, proton-donor coordinate, proton-acceptor coordinate).
l	Mode index.
i	Atom index.

$\alpha \in \{x, y, z\}$	Cartesian-component index.
s	Sub-channel index within a parent channel X.
OOP, INP	Out-of-plane / in-plane component, relative to $\hat{\mathbf{n}}$.
G	Atom group (e.g. cluster, residue, SSE-element).

Code identifiers

Identifier	Meaning
`evec_raw`	Raw eigenvector, not thermally scaled. Carried unchanged through the result-dict (FUND 1 fix).
`evg`	Thermally scaled eigenvector, `evec_raw` $\cdot\, u_{\mathrm{rms}}$, in Å.
`fe_c, s_c`	Tuples of Gaussian centres for the two Fe atoms and two S atoms of a cluster, in canonical order.
`c2l, c2l_h`	Cluster-to-ligand atom centre lists (heavy-only vs hydrogen-inclusive).
`coord_info`	The `CoordInfo` dataclass instance, section 33.
`normal`	The cluster normal $\hat{\mathbf{n}}$, equation (16).
`u_rms`	The u_{rms} value for the current mode.
`HC_JCM`	The constant $hc = 1.986446 \times 10^{-23}$ J cm, used for cm^{-1} conversion.
`b_accum`	B-factor accumulator array, shape $(N_{\mathrm{atoms}},)$.

Glossary of terms

Term	Definition
Cluster-core mode	A normal mode whose dominant displacement amplitude resides on the four [2Fe-2S] cluster atoms. `kern_loc` quantifies this.
Context mode	A mode just outside the analysis window, loaded for the sole purpose of anchoring the interpolation workbook (section 28).
Synthetic zero mode	A FUND 13 artefact mode at the upper window edge, all observables = 0, sentinel `number` $= -1$. Used only for interpolation anchoring; never contributes to aggregates.
Half-open frequency window	[`lo`, `hi`). A mode at $\omega =$ hi is excluded; this prevents double-counting in contiguous-window analyses.
RSS aggregation	Root-sum-square, $\sqrt{\sum_s w_s (\Delta r_s)^2}$, the energetically consistent aggregation rule for sub-channels (section 12).
PCET	Proton-coupled electron transfer. Detected in the pipeline by simultaneous activity of the HA and FeFe (or FeS) channels at the same frequency.
SCSD	Symmetry-coordinate structural decomposition (Kingsbury & Senge 2024); projection onto $D_{2\mathrm{h}}$ irreps for a planar 4-atom cluster.
Kernel classification	The 10-template heuristic that names cluster-core modes by their classical normal-mode character ("Fe-Fe sym stretch", "cluster umbrella", etc.).
Significance colouring	Per-cell colour code in the Excel output that flags values below `significance_threshold_low` σ (grey), between low and high σ (yellow), or above high σ (white).

Code-function index

Alphabetical index of every function referenced via ⇝ in this document, with the file and line range as of v1.0.4. The companion test `tests/test_manual_code_refs.py` verifies that each entry resolves to a real function definition; this table is generated semi-automatically and refreshed for every release. Subsequent releases may shift the line numbers; the function names themselves are stable.

Reference	File	Lines
config.Config	config.py	34–699
core._oop	core.py	193–197
core._oop_ring_metrics	core.py	318–368
core.analyze_fe_ligand	core.py	467–634
core.analyze_his_hn	core.py	635–738
core.analyze_mode_with_fallback	core.py	1701–1841
core.classify_oop_inp	core.py	924–1026
core.compute_scsd_for_mode_full	core.py	1995–2122
core.compute_thermal_amplitude	core.py	131–192
core.reset_warning_state	core.py	88–130
export.export_all	export.py	118–328
export.export_embedding_excel	export.py	645–708
export.export_interpolated_excel	export.py	709–883
export.export_main_excel	export.py	329–592
export.export_sse_excel	export.py	593–644
geometry.cluster_normal	geometry.py	336–371
geometry.detect_sse_dssp	geometry.py	1240–1422
geometry.detect_sse_phipsi	geometry.py	1423–1566
geometry.find_all_clusters	geometry.py	80–192
geometry.find_cluster	geometry.py	193–279
geometry.find_coordinating_residues	geometry.py	752–917
geometry.kabsch_align	geometry.py	423–573
logio.get_eigvec	logio.py	1209–1255
logio.load_scan_cache	logio.py	549–600
logio.read_all_meta	logio.py	878–971
logio.read_std_orient	logio.py	658–766
logio.save_scan_cache	logio.py	601–657
logio.scan_log	logio.py	421–520
orca_io.get_eigvec_orca	orca_io.py	298–320
orca_io.parseresult_to_atoms	orca_io.py	249–274
reorganization.aggregate_by_parent	reorganization.py	491–540
reorganization.build_channels_from_coord	reorganization.py	971–1096
reorganization.compute_co_modulation_spectra	reorganization.py	854–891
reorganization.compute_cumulative_reorganization	reorganization.py	892–970
reorganization.compute_mode_modulations	reorganization.py	356–440
reorganization.compute_modulation_spectra	reorganization.py	799–853
reorganization.compute_total_reorganization	reorganization.py	744–798
reorganization.lambda_pair_cm1	reorganization.py	246–291

`reorganization.signed_dr_along_axis`	`reorganization.py`	177–211
`runner._make_synthetic_zero_mode`	`runner.py`	132–228
`runner._run_analysis_single`	`runner.py`	229–1577
`runner.run_analysis`	`runner.py`	1578–1712

How this index is maintained. The function names are stable across patch releases (v1.0.x). Line ranges are refreshed at every release by re-running `tools/build_codref_index.py` against the source tree; the test `tests/test_manual_code_refs.py::test_supplement_- codrefs_exist` ensures that every reference resolves to a real function regardless of line numbers, providing a stable hyperlink even when the underlying line numbers shift.